

VII

FlyClient Guide

Merkle mountain ranges, the chain-history commitment, and the FlyClient sampling protocol

Marek
m@rek.onl

Abstract

This volume of *The Zcash Arboretum* states the protocol by which a light client decides which of several claimed chains carries the most work while receiving a number of block headers logarithmic in the chain length. It constructs Merkle mountain ranges under the two bagging orders in use, with inclusion paths, prefix roots, position binding and aggregate binding; fixes the security model of the FlyClient paper of Bünz, Kiffer, Luu and Zamani; constructs the FlyClient protocol from the header chain commitment and the per-sample check through the optimal sampling distribution to the non-interactive protocol and its security theorem, with every hypothesis named; and states the chain-history commitment of ZIPs 221, 244 and 258, to which every Zcash block header after the Heartwood activation block commits. Each component of a Zcash FlyClient is classified as specified, designed but unspecified, or an open problem. The volume describes the protocol as specified for NU7, whose deployment ZIP, ZIP 259, has status Draft. It assumes the *Math Guide*, the *Crypto Guide* and the *Consensus Guide*, and cites the *Ironwood Guide* for network upgrades, ZIP statuses and transaction-level objects. It is non-normative: the Zcash Protocol Specification and the Zcash Improvement Proposals are authoritative.

Contents

1	Introduction	5
1.1	Scope and status	5
1.2	Sources and conventions	7
2	The verification problem	9
2.1	Header-only verification	9
2.2	The light-client model of ZIP 307	11
2.3	Statement of the problem	12
3	Merkle mountain ranges	13
3.1	The recursive definition	13
3.2	Peaks and bagging	15
3.3	Appending a leaf	18
3.4	Inclusion paths and prefix roots	19
3.5	Aggregating ranges	22
4	The FlyClient security model	24
4.1	The variable-difficulty backbone model	24
4.2	Retargeting and the difficulty-raising attack	25
4.3	The (c, L) -adversary	27
4.4	Security and succinctness of chain proofs	30
5	The FlyClient protocol	32
5.1	Header chain commitments and per-sample checks	32
5.2	Forks against a committed chain	33
5.3	The difficulty Merkle mountain range	35
5.4	The sampling distribution	38
5.5	The interactive protocol	43
5.6	The non-interactive protocol	43
5.7	The security theorem	45
5.8	Verification after acceptance	47
6	The chain-history commitment	48
6.1	The deployed chain commitment	48
6.2	Upgrade epochs and epoch trees	49
6.3	The node vector	50
6.4	The epoch tree and its root commitment	54
6.5	The authorising-data root	57
6.6	The block commitment hash	58
6.7	The header consensus rules	59
6.8	The committed range	60

7	The deployment boundary	61
7.1	Classification of the security hypotheses	62
7.2	The specified commitment and its consumers	63
7.3	The paper's commitment and the deployed commitment	65
7.4	A Zcash verifier	66
7.5	Open problems	68

1 Introduction

This section fixes the subject of the volume, its status, its sources and its notation before any construction: the scope; the named network upgrades and ZIP statuses on which later rules depend; the three classes in which the deployment boundary places each component of a Zcash FlyClient; the primary paper and the policy for its printed defects; the normative sources; and one table of conventions closing with a symbol index.

1.1 Scope and status

This volume of *The Zcash Arboretum* states three objects and the relation between them. The first is the FlyClient protocol of Bünz, Kiffer, Luu and Zamani (ePrint 2019/226). The second is the chain-history commitment of ZIP 221 (Final), to which every block header after the Heartwood activation block (Definition 1.1) commits: directly, as the header field `hashLightClientRoot`, in blocks validated under the Heartwood or Canopy rules, and, from NU5, as the first input of the hash `hashBlockCommitments` of ZIP 244 (Final). The header fields are named in “The header consensus rules” (§6.7), and their values are defined in “The epoch tree and its root commitment” (§6.4) and “The block commitment hash” (§6.6). The third is the boundary between the protocol and the commitment: each component of a Zcash FlyClient is classified as specified, designed but unspecified, or an open problem (Definition 1.3). The volume is non-normative: the Zcash Protocol Specification, cited as the protocol specification with the title of a section, and the Zcash Improvement Proposals are authoritative.

The volume assumes the following, each cited by italic volume name and section title. From the *Math Guide*: probability, densities and asymptotics (§“Probability, asymptotics, and computation”, with §“Beyond finite spaces: countable additivity, limits, and densities” and §“Asymptotic notation”). From the *Crypto Guide*: hash functions and their security notions (§“Hash functions and the random oracle model”, with §“Security notions: preimage, second-preimage, and collision resistance” and §“The random oracle model”); the provable-security model (§“The provable-security model”, with §“Adversaries and the security parameter”, §“Security as a game” and §“Reductions”); interactive arguments (§“Interactive protocols, proofs, and arguments”); the Fiat–Shamir transform (§“The Fiat–Shamir transform: from interactive to non-interactive”); and the binding and inclusion soundness of Merkle trees (§“Merkle trees and commitments to sets”, with its subsections §“Binary Merkle trees”, §“Authentication paths and membership proofs”, §“Soundness: forging a path implies a collision”, §“Layer and domain separation”, §“Append-only and incrementally updatable trees” and §“Vector commitments”). From the *Consensus Guide*: work and the most-work rule (§“The block tree and the most-work rule” and §“Work, and the best chain”), memoryless clocks (§“Memoryless clocks and the attribution sequence”), the retarget rule under NU7 (§“Difficulty in motion”) and Equihash (§“Equihash and the memoryless model”). From the *Ironwood Guide*, only for the objects it constructs: block chain, height, network upgrades, ZIPs and the status Draft (§“Scope and status”); the Orchard pool and the Ironwood pool (§“The Orchard protocol and its two pools”); the encoding LEBS2OSP (§“Fields, groups, and encodings”); pool roots, final treestates and anchors (§“The note commitment tree” and §“Anchors”); branch IDs and the valid transaction formats, version 5 and version 6 under NU7 (§“The transaction format”; ZIP 2003 (Draft); ZIP

259 (Draft)); and transaction identifiers, the personalised hash h and the branch-ID encoding β (§“Transaction digests and signatures”). The light-client model is cited directly to ZIP 307 (Draft), “Security Model” and “Block header validation”.

Definition 1.1 (Named network upgrades). A *network upgrade*, its *activation height* and the fixed order in which network upgrades activate are those of the *Ironwood Guide*, §“Scope and status”, Definition “Network upgrade” (ZIP 200 (Final), “Terminology” and “Activation mechanism”). This volume names the following network upgrades, in activation order, each with its deployment ZIP and the status of that ZIP:

Upgrade	ZIP	Status	Upgrade	ZIP	Status
Sapling	205	Final	NU6	253	Final
Blossom	206	Final	NU6.1	255	Final
Heartwood	250	Final	NU6.2	257	Final
Canopy	251	Final	NU6.3	258	Draft
NU5	252	Final	NU7	259	Draft

An upgrade is named only where its name fixes which header rule or tree-node rule applies. The *Heartwood activation block* is the block whose height is the activation height of Heartwood. No activation height, date or branch ID value is given here; the branch IDs of the upgrades from Heartwood on, whose upgrade epochs bear an epoch tree (“Upgrade epochs and epoch trees”, §6.2), are tabulated in Table 3.

Definition 1.2 (ZIP statuses). A ZIP and its status *Draft*, the status of every ZIP on submission, are those of the *Ironwood Guide*, §“Scope and status”, Definition “ZIP”. Two further statuses occur in this volume (ZIP 0 (Active), “ZIP Status Field”):

- (i) *Final*, the status of a Consensus or Standards ZIP that is both implemented and activated on the Zcash network;
- (ii) *Active*, the status, typically of a Process or Informational ZIP, reached once rough consensus on it has been reached; ZIP 0 itself has this status.

The status is a property of the document and does not decide whether its rules are in force. The rules of ZIP 258 (Draft) and ZIP 229 (Draft) apply from NU6.3 activation: ZIP 258, “NU6.3 deployment”, lists ZIP 229 among the primary sources of the NU6.3 consensus changes, and ZIP 258, “Changes to ZIP 221”, amends the history-tree node of ZIP 221. Every ZIP is cited with its status at its first citation in the volume.

The volume describes the protocol as specified for NU7, whose deployment ZIP, ZIP 259, has status Draft (ZIP 259, preamble). NU6.3 is active.

Definition 1.3 (Epistemic classes). A *component* of a Zcash FlyClient is a data structure, rule, check or proof format of the protocol, or a hypothesis of its security theorem. A component is

- (i) *specified* when normative text of the protocol specification or of a ZIP fixes it, whatever the status of that ZIP (Definition 1.2);

- (ii) *designed but unspecified* when a published design with an analysis exists but no ZIP or section of the protocol specification fixes it for Zcash;
- (iii) an *open problem* when no design exists.

A property proved in this volume of a specified object is classified by its object: the object is specified, and the property is proved here.

The class *specified* is that of the *Ironwood Guide*, §“Scope and status”, Definition “Epistemic classes”; that volume applies *designed but unspecified* to a property that a deployed construction is designed to satisfy but that no specification, ZIP or proof establishes, whereas this volume applies it to a component that exists only as a published design; the two uses share the condition that no normative text fixes the object, and the sense of the *Ironwood Guide* is not used here.

1.2 Sources and conventions

The primary source is B. Bünz, L. Kiffer, L. Luu and M. Zamani, “FlyClient: Super-Light Clients for Cryptocurrencies”, IACR ePrint 2019/226, in its sixth revision, the last revision on the ePrint page and the revision of record; the paper also appeared at the IEEE Symposium on Security and Privacy. Every statement taken from the paper is cited by the section, definition, theorem, lemma, algorithm or printed page of that revision. A section of the paper is written “Section 5.3”, without the section sign, which this volume reserves for sections of the protocol specification and of the *Arboretum*.

Where a statement of this volume rests on a printed statement of the paper that is defective, the volume states the corrected form at that point and names the departure. Only defects that touch a stated result are mentioned, and there is no separate register of them. Claims of the paper that this volume does not prove are stated as the paper’s, with their gaps: for example Assumption 5.11 (“Transition rewriting”) and Remark 4.11 (“Lemma 1 of the paper”).

The normative sources are the following. The protocol specification, in the sections §“The Block Chain”, §“Integers, Bit Sequences, and Endianness”, §“Constants”, §“BLAKE2 Hash Functions”, §“Transaction Encoding and Consensus”, §“Block Header Encoding and Consensus”, §“Difficulty filter”, §“Difficulty adjustment”, §“nBits conversion” and §“Definition of Work”. ZIP 0 (Active), which defines the ZIP process and its statuses; ZIP 200 (Final); ZIP 204 (Draft); ZIPs 205, 206, 250, 251, 252, 253, 255 and 257 (Final), the deployment ZIPs of the upgrades named in Definition 1.1 before NU6.3; ZIP 218 (Draft); ZIP 221 (Final); ZIP 229 (Draft); ZIP 244 (Final); ZIP 258 (Draft); ZIP 259 (Draft); ZIP 307 (Draft); and ZIP 2003 (Draft). A ZIP is cited by its number and the title of a section, the title following the number in the same sentence; the protocol specification is cited as protocol specification, §“Title”.

The conventions of the volume and the symbols of later sections are fixed in Table 1. The paper overloads several symbols: n denotes both the honest participation and the chain length; k the persistence depth, the samples per interval and $\log_c$; λ both the security parameter and a statistical exponent; δ both the weight fraction checked in full and the backbone parameter for which $\mu = 1 - \delta$ bounds the adversary’s mining power; L both the fork threshold and the number of blocks of the checked suffix; and c both the weight bound and a ratio of mining power. The table assigns one meaning to each symbol, and the index that closes it records where each is defined.

Conventions	
Logarithms	The logarithm $\log x$ is natural and $\log_b x$ is to base 2 (<i>Math Guide</i> , §“Symbols and their pronunciation”). For real $b > 0$, $b \neq 1$, $\log_b x := \log x / \log b$, as in $\log_{1/c}$: a real-base logarithm, distinct from the discrete logarithm $\log_g b$, whose base is a group element.
Floor, ceiling	The floor $\lfloor x \rfloor$ and ceiling $\lceil x \rceil$ of a real x (<i>Math Guide</i> , §“Symbols and their pronunciation”; §“The division algorithm”).
Asymptotics	The symbols O and Θ and negligible functions are as in the <i>Math Guide</i> , §“Asymptotic notation” and §“Polynomial, exponential, and negligible functions”; and $\text{polylog}(N) := (\log N)^{O(1)}$. With several parameters, n or N grows while λ , σ , c and L are fixed; the budget R is either fixed or, as in the paper, $R = c n$, and each bound states which.
Three parameters	λ the security parameter (<i>Math Guide</i> , §“The security parameter”); σ the statistical exponent of a failure target $2^{-\sigma}$; κ the output length in bits of the paper’s hash H . The paper writes λ for all three.
Node fields	Fields of a node v of the difficulty Merkle mountain range are sans-serif and accessed with a dot, as $v.w$, $v.t$.
Personalised hash	The hash h and the encoding β are those of the <i>Ironwood Guide</i> , §“Transaction digests and signatures”; the hash is always written with its personalisation and input in full.
Two epochs	The <i>retarget epoch</i> (“Retargeting and the difficulty-raising attack”) and the <i>upgrade epoch</i> (“Upgrade epochs and epoch trees”) are distinct objects; “epoch” is not used alone where it is ambiguous.
Unused symbols	The <i>Crypto Guide</i> ’s c (completeness error) and L (language) do not occur.
Symbol index (pointers to the defining subsection; nothing is defined here)	
Header-only verification	N the length of a claimed chain; η a block height in the sense of the <i>Ironwood Guide</i> , §“Scope and status”; $\omega(C)$ the total work of a chain C ; hdr a block header; every chain-length quantity before “Merkle mountain ranges” is written N or “the chain length”, never n .
Merkle mountain ranges (section preamble)	the hash $H: \{0, 1\}^* \rightarrow \{0, 1\}^*$ and its output length κ .
The recursive definition	n the leaf count of a Merkle mountain range, hence of the range a head commits to, never a block height; j a leaf index and x_j the value of leaf j ; M_n the range over n values and rt_n its root; $\text{alt}(v)$ the altitude of a node.
Peaks and bagging	$P_1, \dots, P_d$ the peaks, $d = \text{popcount}(n)$; Bag_R and Bag_L the two bagging orders.
Aggregating ranges	ser the serialisation of a node tuple; W the cumulative weight before a leaf.
The variable-difficulty backbone model	D a difficulty level (larger is harder); T a target (smaller is harder); ω , restated, never written D ; q the queries per player per round; r a round; ν_r and t_r the honest and adversarial players of round r ; (γ, s) the participation parameters; μ the adversary’s power bound; ℓ the persistence depth.
Retargeting and the difficulty-raising attack	E the retarget-epoch length; Δ the rounds a retarget epoch takes; τ the dampening factor; f the expected blocks per round.
The (c, L) -adversary	c and L the parameters of the adversary; ρ the power ratio of the paper’s evaluation.
The difficulty Merkle mountain range	y and z the level exponent and the retarget-epoch count of the feasibility bounds; the dotted node fields.
The sampling distribution	δ the weight fraction checked in full; k the catch parameter, $\delta = c^k$, its only meaning; a a fork point; ξ a point of $[0, 1]$; φ a density on $[0, 1]$; K the constant of the indifference density; g the FlyClient density and G its distribution function; p the single-draw catch probability; Q the sample count; μ the samples per dyadic interval; U a uniform variable on $[0, 1]$.
The non-interactive protocol	R the head-recomputation budget.
Header chain commitments and per-sample checks	B_η the block at height η ; heights are written η , never i .

Table 1. Conventions and symbol index. Each symbol has the one meaning listed, fixed in the subsection named; a letter bound inside a single definition, construction or proof, such as a node or an index, keeps its meaning only there.

Upgrade epochs and epoch trees	$\mathcal{A}$ an activation height; $\mathcal{A}_H$ that of Heartwood, distinct from the hash H ; $\alpha(\eta)$ the greatest activation height below η ; $\mathcal{T}_\eta$ the epoch tree committed by B_η , distinct from a target T , with $n = \eta - \alpha(\eta)$ leaves.
The epoch tree and its root commitment	$b(\mathcal{T})$ the branch ID of an epoch tree $\mathcal{T}$.
The header consensus rules	$\mathcal{A}_{\text{NU5}}$ the activation height of NU5.
The committed range	$\mathcal{A}_{\text{NU7}}$ the activation height of NU7.

2 The verification problem

This section fixes the object a light client verifies, the Zcash block header, and the baseline procedure that verifies a chain of headers in full. It states the trust model of the light-client protocol specified for Zcash and the problem the volume solves, with each term of its solution defined in one clause and constructed in a later section.

2.1 Header-only verification

Definition 2.1 (Light client). A *light client* is a client that is not a full participant in the network of Zcash peers. It can send and receive payments, but does not store or validate a copy of the block chain (ZIP 221, “Terminology”).

The verifier of the FlyClient paper is a light client given the additional inputs fixed in Definition 4.16 (“Security and succinctness of chain proofs”, §4.4).

Byte strings and integers are related by the maps of the *Ironwood Guide*, §“Fields, groups, and encodings”, Definition “Integer and byte encodings”: the map I2LEOSP_k sends an integer in $\{0, \dots, 2^k - 1\}$ to its $\lceil k/8 \rceil$ -byte little-endian encoding, and the map LEOSP2IP_k sends a $k/8$ -byte string to the integer it encodes in little-endian order.

Definition 2.2 (CompactSize). The *CompactSize encoding* of an integer $v \in \{0, \dots, 2^{64} - 1\}$ is the byte string

$$\text{CompactSize}(v) := \begin{cases} \text{I2LEOSP}_8(v) & \text{if } 0 \leq v \leq 252, \\ 0xFD \parallel \text{I2LEOSP}_{16}(v) & \text{if } 253 \leq v \leq 0xFFFF, \\ 0xFE \parallel \text{I2LEOSP}_{32}(v) & \text{if } 0x10000 \leq v \leq 0xFFFFFFFF, \\ 0xFF \parallel \text{I2LEOSP}_{64}(v) & \text{if } 0x100000000 \leq v \leq 0xFFFFFFFFFFFFFFFF, \end{cases}$$

of length 1, 3, 5 or 9 bytes respectively. Only this encoding, the shortest, is valid for v : a byte string that decodes to v under one of the other three prefixes **MUST** be rejected (ZIP 204, “CompactSize”).

ZIP 204 has status Draft; the minimal-encoding rule binds block headers independently,

through the protocol specification, §“Block Header Encoding and Consensus”, which requires the header field `solutionSize` to be encoded with the minimum number of bytes. That rule prevents a miner from testing several encodings of one Equihash solution against the difficulty filter.

The upgrade epochs delimited by the network upgrades of Definition 1.1 are fixed in “Upgrade epochs and epoch trees” (§6.2).

Definition 2.3 (Block header). A Zcash *block* consists of a block header and a sequence of transactions. A *block header* `hdr` is the concatenation, in the following order, of the fields

Bytes	Field	Content
4	<code>nVersion</code>	signed 32-bit integer; the consensus rules require $nVersion \geq 4$
32	<code>hashPrevBlock</code>	the block hash of the previous block
32	<code>hashMerkleRoot</code>	the root of the transaction Merkle tree over the identifiers of the block’s transactions, stated in “The authorising-data root” (§6.5)
32	commitment field	named as below
4	<code>nTime</code>	unsigned 32-bit integer, the block timestamp
4	<code>nBits</code>	unsigned 32-bit integer, an encoding of the target threshold
32	<code>nNonce</code>	32 arbitrary bytes
3	<code>solutionSize</code>	CompactSize(1344)
1344	<code>solution</code>	an Equihash solution

with integers encoded little-endian. The commitment field is named `hashReserved` before Sapling, `hashFinalSaplingRoot` in Sapling and Blossom, `hashLightClientRoot` in Heartwood and Canopy, and `hashBlockCommitments` from NU5; its value in each upgrade epoch is fixed in “The header consensus rules” (§6.7). The *block hash* of `hdr` is the 32-byte string

$$\text{BlockHash}(\text{hdr}) := \text{SHA256d}(\text{hdr}), \quad \text{SHA256d}(x) := \text{SHA256}(\text{SHA256}(x)),$$

taken over the whole header serialisation, `solutionSize` and `solution` included, in internal byte order, the order in which the hash function outputs its bytes. Let $\text{ToTarget}: \{0, \dots, 2^{32} - 1\} \rightarrow \mathbb{N}$ be the conversion of protocol specification, §“nBits conversion”. The header `hdr` *satisfies proof of work* when `solution` is a valid Equihash solution for `hdr` and

$$\text{LEOS2IP}_{256}(\text{BlockHash}(\text{hdr})) \leq \text{ToTarget}(\text{nBits})$$

(protocol specification, §“Difficulty filter”). The consensus rules further require, for a block at height η , that $\text{nBits} = \text{ThresholdBits}(\eta)$, the value the difficulty-adjustment rule of protocol specification, §“Difficulty adjustment” assigns to that height.

The definition follows protocol specification, §“Block Header Encoding and Consensus” for the field table and its consensus rules, and ZIP 221, “Block header semantics and consensus rules” and ZIP 244 for the renaming of the commitment field. The hash `SHA256d` is registered in the *Crypto Guide*, §“The transparent layer’s hashes”. The Equihash condition is cited, not reconstructed: *Consensus Guide*, §“Equihash and the memoryless model”. The *work* of a block whose

header carries nBits is

$$\text{work}(\text{hdr}) := \left\lfloor \frac{2^{256}}{\text{ToTarget}(\text{nBits}) + 1} \right\rfloor,$$

and the *total work* $\omega(C)$ of a chain C is the sum of the work of its blocks; a node considers the valid chain of greatest total work to be best, breaking ties between leaf blocks in favour of the block received first (*Consensus Guide*, §“The block tree and the most-work rule”, and its subsection §“Work, and the best chain”; protocol specification, §“The Block Chain” and §“Definition of Work”).

The fixed-width fields of Definition 2.3 occupy $4 + 32 + 32 + 32 + 4 + 4 + 32 = 140$ bytes. Since $253 \leq 1344 \leq 0xFFFF$, the field `solutionSize` occupies 3 bytes, and with the 1,344-byte solution a Zcash block header is 1,487 bytes, of which the solution is $1344/1487$, about 90.4 per cent.

Definition 2.4 (Simplified payment verification). A *claimed chain* of length $N \geq 1$ is a sequence of block headers $C = (\text{hdr}_0, \dots, \text{hdr}_{N-1})$ in which hdr_0 is the header of the genesis block; header hdr_η is the header at height η . Given one or more claimed chains, a *simplified-payment-verification* (SPV) client downloads every header of every claimed chain and checks, for each chain C :

- (S1) *linkage*: for $1 \leq \eta \leq N - 1$, the field `hashPrevBlock` of hdr_η equals $\text{BlockHash}(\text{hdr}_{\eta-1})$;
- (S2) *proof of work*: for $0 \leq \eta \leq N - 1$, header hdr_η satisfies proof of work in the sense of Definition 2.3, and its field `nBits` equals $\text{ThresholdBits}(\eta)$, computed from $\text{hdr}_0, \dots, \text{hdr}_{\eta-1}$.

It rejects every chain that fails a check and accepts, among the rest, the chain C of greatest total work $\omega(C)$, under the best-chain rule of the *Consensus Guide*, §“Work, and the best chain”.

The definition is that of ePrint 2019/226, Section 1, p. 2, instantiated with the header checks of protocol specification, §“Block Header Encoding and Consensus”. For a claimed chain of N blocks the client receives N headers, 1,487 N bytes, so its communication is $\Theta(N)$. A block is at most 2,000,000 bytes (same section), so the saving over downloading full blocks is at most a constant factor, not an asymptotic one.

An SPV client checks neither the validity of transactions nor any consensus rule beyond those of the header, and therefore relies on the following assumption.

Assumption 2.5 (SPV assumption). “The chain with the most PoW solutions follows the rules of the network and will eventually be accepted by the majority of miners.” (ePrint 2019/226, Section 1, p. 2, Assumption 1.)

The paper formalises the property an SPV client checks as Definition 4.14, and strengthens Assumption 2.5 quantitatively as Assumption 4.8 (“The (c, L) -adversary”, §4.3).

2.2 The light-client model of ZIP 307

ZIP 307 (Draft) specifies a light-client protocol in which a light client obtains, from a Zcash node and a proxy, *compact blocks*: per-block messages that may carry the block header. The trust model

is stated in ZIP 307, “Security Model”: the node and proxy combination is assumed to be “honest but curious” and “is trusted to provide a correct view of the current best chain state and to faithfully transmit queries and responses”.

ZIP 307, “Block header validation” describes, as a proposed enhancement that the ZIP records as only partially implemented (only check (Z2) is made), checks of the SPV kind on each compact block that carries a header. With the fields named as in Definition 2.3, the checks are:

- (Z1) the field `nVersion` is at least the minimum block version;
- (Z2) the field `hashPrevBlock` equals the block hash of the previous compact block;
- (Z3) the Equihash solution is valid;
- (Z4) the target `ToTarget(nBits)` is non-zero and at most the proof-of-work limit `PoWLimit` (protocol specification, §“Constants”);
- (Z5) when the last 27 compact blocks all carry headers, the field `nBits` is correct under the difficulty-adjustment rule;
- (Z6) the block hash, read as a little-endian integer, is at most `ToTarget(nBits)`.

In that proposal, a compact block that fails a check **MUST** be discarded. The same section of ZIP 307 further describes a check that the commitment field equals the root of the Sapling note commitment tree recomputed from the compact block; from Heartwood that check no longer applies, since the field changes meaning (Definition 2.3; ZIP 221, “Block header semantics and consensus rules”). Checks (Z1) to (Z6), where made on every header, cost communication linear in the chain length N , as in Definition 2.4.

Of the trust that ZIP 307 places in the node and proxy, a FlyClient verifier would replace only the trust in which header chain carries the most work, under Assumption 4.8 (“The (c, L) -adversary”). It would replace the linear header checks by the logarithmic sampling of “The FlyClient protocol” (§5); it does not replace faithful transmission of queries and of compact-block contents. Such a verifier is designed but unspecified in the sense of Definition 1.3; its status is fixed in “A Zcash verifier” (§7.4).

2.3 Statement of the problem

The problem is the following. Under the SPV assumption (Assumption 2.5), strengthened quantitatively as Assumption 4.8 in “The (c, L) -adversary” (§4.3), a light client is to accept the head of the chain of greatest total work after communication sublinear in the chain length N . The inputs and the chain to be selected are those of Definition 2.4; the assumption is strengthened to Assumption 4.8, the selection holds except with a failure probability fixed in “The FlyClient protocol” (§5), and the communication falls from $\Theta(N)$ to sublinear.

A client that does not check every proof of work “can be tricked into accepting an invalid chain by a malicious prover who can precompute a sufficiently-long chain using its limited computational power” (ePrint 2019/226, Section 1, p. 3); the adversary is fixed in “The FlyClient security model” (§4).

The solution uses four objects. A *Merkle mountain range* is a binary hash tree over a sequence that admits appending a leaf without changing its peaks or any node beneath them (“Merkle mountain ranges”, §3). A *header chain commitment* is a root, carried in every header after the genesis header, of such a range over all blocks strictly before that header (“Header chain commitments and per-sample checks”, §5.1). A *work-weighted sample* is a block drawn at a random point of cumulative work and checked against the commitment of the head (“The sampling distribution”, §5.4). A *suffix* of the chain is downloaded and checked in full, its extent fixed in the same subsection. *FlyClient* is the protocol that verifies a proof-of-work chain by drawing logarithmically many such samples and checking each against the commitment of the head (ePrint 2019/226, Section 1). The commitment of the paper covers the chain from the genesis block, whereas the commitment of ZIP 221 covers one upgrade epoch: the blocks from the activation height of the preceding network upgrade up to the committing block, exclusive (ZIP 221, “Specification”; “Upgrade epochs and epoch trees”, §6.2).

3 Merkle mountain ranges

This section constructs the commitment on which the FlyClient protocol and the chain-history commitment of ZIP 221 rest: a binary hash tree over a sequence of values whose shape is fixed by the length of the sequence, extended by one value at logarithmic cost, and opened at one position j by a path from which the root over the prefix of length $j - 1$ is recomputed, and, under the right fold, also the root over the prefix of length j . The construction is parametrised by the order in which its perfect subtrees are joined: the right fold of ePrint 2019/226 and the left fold of ZIP 221, “Background”. The fixed-depth trees of the *Crypto Guide*, §“Binary Merkle trees” and §“Append-only and incrementally updatable trees”, are the special case of a sequence whose length is a power of two.

Throughout, $H: \{0, 1\}^* \rightarrow \{0, 1\}^k$ is a hash function drawn from a family that is collision resistant in the sense of the *Crypto Guide*, Definition “Collision resistance” (§“Security notions: preimage, second-preimage, and collision resistance”); ePrint 2019/226, Definition 8, states the same assumption. A statement holds *barring a collision* when a deterministic procedure, given any input on which the statement fails, outputs two distinct strings with equal images under H , using the number of evaluations of H stated with it. Such a procedure is a tight reduction in the sense of the *Crypto Guide*, Definition “Reduction loss and tightness” (§“Reductions”), so a probabilistic polynomial-time adversary makes the statement fail with negligible probability.

3.1 The recursive definition

Definition 3.1 (Merkle mountain range). Let $n \geq 1$ and $x_1, \dots, x_n \in \{0, 1\}^k$. The *Merkle mountain range* M_n over $(x_1, \dots, x_n)$ is the rooted ordered binary tree, every internal node of which has exactly two children, a left and a right one, with a value in $\{0, 1\}^k$ at every node, defined by recursion on n :

1. if $n = 1$, M_1 is the single leaf with value x_1 ;

2. if $n > 1$, let $i = \lfloor \log_2(n-1) \rfloor$, so that $2^i < n \leq 2^{i+1}$ and 2^i is the largest power of two strictly below n ; the root has as left child the range M_{2^i} over $(x_1, \dots, x_{2^i})$ and as right child the range M_{n-2^i} over $(x_{2^i+1}, \dots, x_n)$, where $1 \leq n - 2^i \leq 2^i$.

Every internal node with children of values v_ℓ and v_r has value $H(v_\ell \parallel v_r)$, the image of the 2κ -bit concatenation of its children's values. The leaves are ordered from left to right and leaf j holds x_j . The root rt_n is the value of the root node; *range* abbreviates Merkle mountain range.

The *depth* of a node is the number of edges from the root to it, and the depth of a tree the maximal depth of a leaf. A *perfect subtree of altitude a* is a subtree with 2^a leaves, all at depth a below its root; its root node v has *altitude* $\text{alt}(v) = a$. The *right spine* is the sequence of nodes that starts at the root and continues from each internal node to its right child, ending at a leaf.

ZIP 221, “Terminology”, calls a Merkle mountain range a binary hash tree that admits appends of new leaves without changing the value of existing nodes; Theorem 3.11 and Proposition 3.12 below establish it for Definition 3.1 for the peaks and every node beneath them. An append rehashes only the right-spine nodes above the first perfect one, the counterparts of the bagging nodes of ZIP 221, “Background”. Definition 3.1 is ePrint 2019/226, Definition 11, with the base case $n = 1$, implicit there, made explicit. Every node of M_n is the root of a range over a run of consecutive values, by the recursion. Its depth is fixed in Lemma 3.2.

Lemma 3.2 (Left children are perfect). *Let $n \geq 1$.*

1. *A range over 2^a values is a perfect subtree of altitude a .*
2. *Every left child in M_n is perfect: the left child of a node over $m \geq 2$ leaves is the perfect range over $2^{\lfloor \log_2(m-1) \rfloor}$ leaves.*
3. *Descend the right spine of M_n from the root, stopping at the first node whose leaf count is a power of two. The left children of the nodes passed, followed by the node where the descent stops, are perfect subtrees of pairwise distinct power-of-two sizes, decreasing from the root downwards, covering consecutive runs of leaves from left to right; their sizes are the terms of the binary expansion of n .*
4. *The range M_n has depth exactly $\lfloor \log_2 n \rfloor$, which is 0 for $n = 1$.*

Proof. (1) By induction on a . For $a = 0$ the range is a leaf. For $a \geq 1$, $\lfloor \log_2(2^a - 1) \rfloor = a - 1$, so the root has two children over 2^{a-1} values each, perfect of altitude $a - 1$ by hypothesis; all leaves lie at depth a .

(2) Every node over $m \geq 2$ leaves is a range M_m , whose left child is a range over $2^{\lfloor \log_2(m-1) \rfloor}$ values, perfect by (1).

(3) Let a right-spine node have m leaves. If m is a power of two, the node is perfect by (1) and the descent stops. Otherwise $2^{\lfloor \log_2 m \rfloor} < m$, hence $m - 1 \geq 2^{\lfloor \log_2 m \rfloor}$ and $\lfloor \log_2(m - 1) \rfloor = \lfloor \log_2 m \rfloor$: the left child takes the highest term $2^{\lfloor \log_2 m \rfloor}$ of the binary expansion of m and the right child the remaining terms. By induction along the spine, the left children passed take the terms of

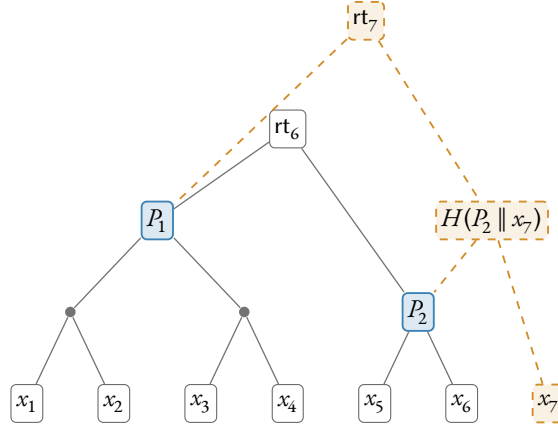


Figure 1. The range M_6 over $x_1, \dots, x_6$ in single-rooted form (solid): the root rt_6 has left child P_1 , the perfect range over $x_1, \dots, x_4$, and right child $P_2 = H(x_5 \parallel x_6)$, the perfect range over x_5, x_6 . The nodes P_1 and P_2 are the peaks of Definition 3.4; P_2 is the first perfect node on the right spine of M_6 , whose last node is the leaf x_6 . Dashed: appending x_7 creates only the nodes $H(P_2 \parallel x_7)$ and $rt_7 = H(P_1 \parallel H(P_2 \parallel x_7))$; the nodes P_1, P_2 and every node beneath them are shared by M_6 and M_7 .

the expansion of n from the highest downwards, and the descent stops when one term is left. The runs are consecutive because each left child precedes its sibling's leaves.

(4) By induction on n . For $n = 1$ the depth is 0. For $n > 1$ let $i = \lfloor \log_2(n - 1) \rfloor$. The left child is perfect over 2^i leaves, of depth i ; the right child has $n - 2^i \leq 2^i$ leaves and depth $\lfloor \log_2(n - 2^i) \rfloor \leq i$ by hypothesis. The depth of M_n is therefore $i + 1$, which equals $\lfloor \log_2 n \rfloor$ since $2^i < n \leq 2^{i+1}$. $\square$

Example 3.3 (Six and seven leaves). For $n = 6$, $i = \lfloor \log_2 5 \rfloor = 2$ and $6 = 4 + 2$. The left child is the perfect range P_1 over $x_1, \dots, x_4$; the right child has two leaves, is perfect, and has value $P_2 = H(x_5 \parallel x_6)$; hence $rt_6 = H(P_1 \parallel P_2)$. For $n = 7$, $i = \lfloor \log_2 6 \rfloor = 2$ and $7 = 4 + 3$; the three-leaf right child splits as $3 = 2 + 1$, so $7 = 4 + (2 + 1)$ and

$$rt_7 = H(P_1 \parallel H(H(x_5 \parallel x_6) \parallel x_7)).$$

3.2 Peaks and bagging

For $n \geq 1$, $\text{popcount}(n)$ is the number of ones in the binary expansion of n .

Definition 3.4 (Peaks). Let $n \geq 1$ and write $n = 2^{a_1} + \dots + 2^{a_d}$ with $a_1 > \dots > a_d \geq 0$, so that $d = \text{popcount}(n)$, and let $s_i = \sum_{l < i} 2^{a_l}$ for $1 \leq i \leq d$. The *peaks* of the sequence $(x_1, \dots, x_n)$ are $P_1, \dots, P_d$, where P_i is the perfect range over $(x_{s_i+1}, \dots, x_{s_i+2^{a_i}})$, of altitude $\text{alt}(P_i) = a_i$. There is one peak per set bit of n , of altitude that bit's index; the altitudes strictly decrease from the left, and the peaks cover consecutive runs of leaves. The symbol P_i denotes both the subtree and its root value.

The peaks are the mountains of ZIP 221, “Background”, whose leftmost peak is the highest. The same section numbers nodes in the order of their creation; that numbering enters no committed value and no check, and the root is a function of the leaf sequence and the bagging order of Definition 3.5 alone. No node numbering is used in this volume.

Definition 3.5 (Bagging orders). Let $P_1, \dots, P_d$ be the peaks of $(x_1, \dots, x_n)$. The *right fold* and the *left fold* of the peaks are

$$\begin{aligned}\text{Bag}_R(P_1, \dots, P_d) &= H(P_1 \parallel H(P_2 \parallel \dots H(P_{d-1} \parallel P_d) \dots)), \\ \text{Bag}_L(P_1, \dots, P_d) &= H(\dots H(H(P_1 \parallel P_2) \parallel P_3) \dots \parallel P_d);\end{aligned}$$

a single peak is its own bag under either fold. The left fold is the order of ZIP 221, “Background”, which repeatedly joins the two leftmost peaks. For $b \in \{R, L\}$ the tree M_n^b consists of the d perfect peak subtrees joined by the $d - 1$ internal *bagging nodes* of Bag_b , and rt_n^b is its root value. Both are determined by n, b and the leaf sequence.

Proposition 3.6 (Peaks of the recursive form). *For every $n \geq 1$ the peaks of $(x_1, \dots, x_n)$ are exactly the left children of the right-spine nodes of M_n , from the root down to the first perfect right-spine node, followed by that node. Hence $M_n = M_n^R$ and $\text{rt}_n = \text{Bag}_R(P_1, \dots, P_d)$.*

Proof. By Lemma 3.2(3) the nodes named are perfect subtrees whose sizes are the terms of the binary expansion of n , largest first, covering consecutive runs of leaves from the left: they are $P_1, \dots, P_d$ in order. Along the spine, the t -th node passed has value $H(P_t \parallel v_{t+1})$ with v_{t+1} the value of the next spine node, and the last spine node considered is P_d ; unwinding gives the right fold. $\square$

Lemma 3.7 (Path lengths under the two baggings). *Let $n \geq 1$ have peaks $P_1, \dots, P_d$ of altitudes $a_1 > \dots > a_d$, let $b \in \{R, L\}$, and let leaf j lie in P_i . The depth $\text{depth}_b(n, j)$ of leaf j in M_n^b is*

$$\text{depth}_R(n, j) = \begin{cases} a_i + i & i < d, \\ a_d + d - 1 & i = d, \end{cases} \quad \text{depth}_L(n, j) = \begin{cases} a_1 + d - 1 & i = 1, \\ a_i + d - i + 1 & i \geq 2. \end{cases}$$

Consequently the depth of M_n^R is $\lceil \log_2 n \rceil$; the depth of M_n^L is $\lceil \log_2 n \rceil + \text{popcount}(n) - 1 \leq 2\lceil \log_2 n \rceil$, attained by the leaves of P_1 ; and the two depths differ exactly when $d \geq 3$. For $n = 7$ they are 3 and 4; for $n = 127$ they are 7 and 12. Leaf depths are functions of (b, n, j) ; for $d \geq 2$ they are not all equal.

Proof. The leaf lies at depth a_i below the root of P_i ; it remains to find the depth of that root in the bag. Under Bag_R the t -th bagging node lies at depth $t - 1$ and has left child P_t for $t \leq d - 1$, and the last bagging node has right child P_d ; so P_i lies at depth i for $i < d$ and P_d at depth $d - 1$. Under Bag_L the root has right child P_d and left child $\text{Bag}_L(P_1, \dots, P_{d-1})$, and inductively P_i lies at depth $d - i + 1$ for $i \geq 2$ while P_1 shares depth $d - 1$ with P_2 ; for $d = 1$ the single peak lies at depth 0.

Maxima. Under R , $a_i \leq a_1 - (i - 1)$ gives $a_i + i \leq a_1 + 1$ and $a_d + d - 1 \leq a_1$; the maximum is $a_1 + 1$ for $d \geq 2$ and a_1 for $d = 1$, which is $\lceil \log_2 n \rceil$ in both cases, as n is a power of two exactly when $d = 1$. Under L , for $i \geq 2$, $a_i + d - i + 1 \leq a_1 + d - 2i + 2 \leq a_1 + d - 2$, so the maximum is $a_1 + d - 1$, attained by P_1 ; with $a_1 = \lceil \log_2 n \rceil$ and $d = \text{popcount}(n) \leq \lceil \log_2 n \rceil + 1$ this is the stated value and bound. The maxima $a_1 + 1$ and $a_1 + d - 1$ coincide for $d = 2$, both equal a_1 for $d = 1$, and differ for $d \geq 3$. For $d \geq 2$ a leaf of P_1 and a leaf of P_d differ in depth under R ($a_1 + 1$ against at most a_1), and a leaf of P_1 and a leaf of P_2 under L ($a_1 \neq a_2$). $\square$

A *cycle* of H is a value v and a nonempty sequence of evaluations of H that, starting from inputs that include v , recomputes v .

Theorem 3.8 (Fold order). *Let $n \geq 1$ have d peaks. The shapes M_n^R and M_n^L coincide if and only if $d \leq 2$. For $d \geq 3$:*

1. *the roots rt_n^R and rt_n^L are distinct hash expressions over the same peaks, and from values of the peaks for which $\text{rt}_n^R = \text{rt}_n^L$ a collision or a cycle of H is computed;*
2. *for every leaf j , the sequence of sides of the siblings on leaf j 's path above its peak differs between M_n^R and M_n^L ;*
3. *the peaks and the perfect subtrees beneath them coincide, and so do the asymptotic costs of append, of storage of the peaks and of a path, each logarithmic in n ; the leaf depths differ as in Lemma 3.7, so the bound $\lceil \log_2 n \rceil$ on the path length of ePrint 2019/226 does not hold for M_n^L .*

Proof. Definition 3.1 splits $n = 2^{\lceil \log_2 (n-1) \rceil} + (\text{rest})$ with the first peak on the left and recurses on the rest, which is the right fold (Proposition 3.6). For $d = 1$ both folds are P_1 and for $d = 2$ both are $H(P_1 \parallel P_2)$. For $d \geq 3$ the root's left child is P_1 under R and a bagging node under L , so the shapes differ.

(1) The root inputs are $P_1 \parallel \text{Bag}_R(P_2, \dots, P_d)$ and $\text{Bag}_L(P_1, \dots, P_{d-1}) \parallel P_d$. If the root values agree and these 2κ -bit strings differ, they are a collision. Otherwise $P_1 = \text{Bag}_L(P_1, \dots, P_{d-1})$ and $P_d = \text{Bag}_R(P_2, \dots, P_d)$, each a value recomputed from itself by $d - 2 \geq 1$ evaluations of H : a cycle.

(2) Above P_1 the path has under R the single sibling $\text{Bag}_R(P_2, \dots, P_d)$ and under L the $d - 1 \geq 2$ siblings $P_2, \dots, P_d$. Above P_d it has under R the $d - 1 \geq 2$ siblings $P_{d-1}, \dots, P_1$ and under L the single sibling $\text{Bag}_L(P_1, \dots, P_{d-1})$. Above P_i with $2 \leq i < d$ the first sibling is on the right under R , namely $\text{Bag}_R(P_{i+1}, \dots, P_d)$, and on the left under L , namely $\text{Bag}_L(P_1, \dots, P_{i-1})$.

(3) Both trees are built on the peaks of Definition 3.4, which depend on the leaf sequence alone; there are $d \leq \lceil \log_2 n \rceil + 1$ of them. Appending x_{n+1} joins the last peaks of equal altitude with x_{n+1} , at most $\lceil \log_2 n \rceil + 1$ evaluations of H , and re-bagging the at most $\lceil \log_2 (n + 1) \rceil + 1$ new peaks costs one evaluation fewer than their number, under either fold. A path has at most $2\lceil \log_2 n \rceil$ siblings under either fold by Lemma 3.7, whose depths are the stated ones. $\square$

Collision resistance alone does not exclude a cycle, so part (1) does not assert that the two roots differ numerically. No later statement of this volume uses such an inequality; the later sections use the shapes and path lengths of part (2) and Lemma 3.7.

Example 3.9 (Eleven leaves). The eleven-leaf tree of ZIP 221, “Background”, has $11 = 8 + 2 + 1$ and peaks P_1, P_2, P_3 of altitudes 3, 1 and 0, joined by two bagging nodes. The left fold that ZIP 221 commits and the right fold are

$$\text{rt}_{11}^L = H(H(P_1 \parallel P_2) \parallel P_3), \quad \text{rt}_{11}^R = H(P_1 \parallel H(P_2 \parallel P_3)).$$

A leaf of P_1, P_2, P_3 has path length 4, 3, 2 under Bag_R and 5, 3, 1 under Bag_L . Figure 2 shows both trees.

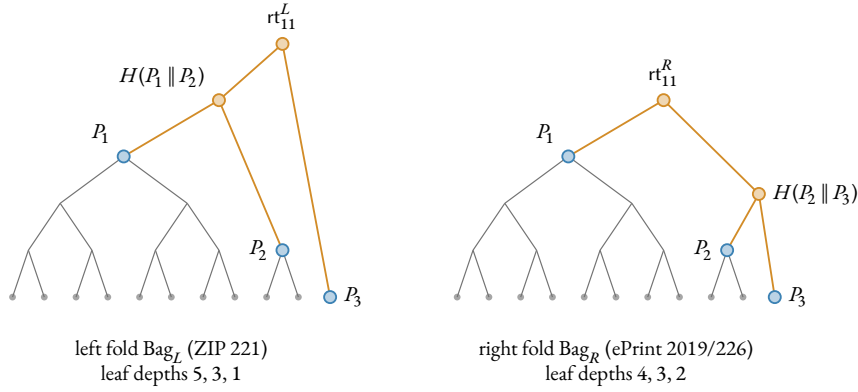


Figure 2. The eleven-leaf tree of Example 3.9, without node numbering. Both panels carry the same peaks P_1, P_2, P_3 (blue) of altitudes 3, 1, 0 over the same leaves. Left: the left fold $\text{rt}_{11}^L = H(H(P_1 \parallel P_2) \parallel P_3)$ that ZIP 221 commits. Right: the right fold $\text{rt}_{11}^R = H(P_1 \parallel H(P_2 \parallel P_3))$ of the recursion of Definition 3.1. Bagging nodes are amber. The leaves of P_1 lie one level deeper under the left fold; the depths below each panel are those of a leaf of P_1, P_2, P_3 .

3.3 Appending a leaf

Construction 3.10 (Append). Every node v of a range carries its leaf count m . Procedure $\text{Append}(v, x)$ takes the root v of a range over m leaves and a value $x \in \{0, 1\}^k$ and returns the root of a range over $m + 1$ leaves:

1. if m is a power of two (v is perfect; $m = 1$ included), return a new node with left child v , right child the leaf x , value $H(v \parallel x)$ and leaf count $m + 1$;
2. otherwise, with children v_ℓ and v_r , replace v_r by $\text{Append}(v_r, x)$, set the value of v to $H(v_\ell \parallel v_r)$ with the new v_r , set its leaf count to $m + 1$, and return v .

The construction is ePrint 2019/226, Algorithm 5. The order $H(v \parallel x)$ in step 1 is that of the algorithm and of Definition 3.1; the base case of the paper's proof of its Theorem 6 writes the reverse order.

Theorem 3.11 (Correctness of append). *For every leaf count $n \geq 1$, $\text{Append}(M_n, x_{n+1})$ is M_{n+1} over $(x_1, \dots, x_{n+1})$: the leaves of M_n followed by x_{n+1} as the rightmost leaf.*

Proof. By induction on n , following ePrint 2019/226, Theorem 6. If $n = 2^i$, Definition 3.1 for $n + 1$ has $\lfloor \log_2 n \rfloor = i$ and right part 1: left child M_n , right child the leaf x_{n+1} , which is step 1. Otherwise $n = 2^i + m$ with $1 \leq m < 2^i$, and $n + 1 = 2^i + (m + 1)$ with the same i , since $m + 1 \leq 2^i$. The left child M_{2^i} is unchanged, and the right child is Append applied to the range M_m over $(x_{2^i+1}, \dots, x_n)$, which is M_{m+1} over $(x_{2^i+1}, \dots, x_{n+1})$ by hypothesis; step 2 rehashes the root. $\square$

Proposition 3.12 (Cost of append). *Procedure $\text{Append}(M_n, x_{n+1})$ creates or rehashes only right-spine nodes of M_n and one new node, with at most $\lfloor \log_2 n \rfloor + 1$ evaluations of H . Every*

left child of M_n , and the perfect right-spine node at which the recursion stops, is a subtree of M_{n+1} with unchanged values.

Proof. The recursion descends the right spine until the first node whose leaf count is a power of two, at depth at most $\lceil \log_2 n \rceil$ (Lemma 3.2(4)), evaluates H once to create the new node, and once more at each spine node above it. Left children and the node where the descent stops are never modified. $\square$

Figure 1 shows the case $n = 6$: two evaluations of H create $H(P_2 \parallel x_7)$ and rt_7 .

3.4 Inclusion paths and prefix roots

Definition 3.13 (Inclusion path). Let $b \in \{R, L\}$, $n \geq 1$, $1 \leq j \leq n$, and $m = \text{depth}_b(n, j)$ (Lemma 3.7). Let $u_0, u_1, \dots, u_m$ be the nodes from leaf j to the root of M_n^b . The *inclusion path* of leaf j is $\pi = (\pi_1, \dots, \pi_m)$, where π_t is the value of the sibling of u_{t-1} , listed bottom-up. The *side* $\sigma_t \in \{\text{left}, \text{right}\}$ records whether that sibling is the left or the right child of u_t . The sides and m are functions of (b, n, j) through the shape of M_n^b .

The verifier $\text{Verify}_b(\text{rt}, n, j, x, \pi)$, on $\text{rt}, x \in \{0, 1\}^*$ and a sequence π of κ -bit values:

1. rejects unless $|\pi| = \text{depth}_b(n, j)$;
2. sets $y_0 = x$ and, for $t = 1, \dots, m$, $y_t = H(\pi_t \parallel y_{t-1})$ if $\sigma_t = \text{left}$ and $y_t = H(y_{t-1} \parallel \pi_t)$ otherwise;
3. accepts if and only if $y_m = \text{rt}$.

By construction Verify_b accepts (x_j, π) with π the inclusion path of leaf j against rt_n^b . Under Bag_R every path has at most $\lceil \log_2 n \rceil$ siblings (Lemma 3.7); ePrint 2019/226, Definition 10, counts at most $\log_2 n + 1$ hashes, the leaf pair included. For $d \geq 2$, path lengths differ between leaves (Lemma 3.7). The paper’s Algorithm 4 requires $|\pi| = \lceil \log_2(n - 1) \rceil$ and so rejects valid paths: 1,225 of the 8,384 leaf paths of the ranges M_n , $2 \leq n \leq 129$, among them both leaves of M_2 and leaf 5 of M_6 . The verifier above fixes the length from (b, n, j) instead. For n a power of two, M_n^b is a perfect tree, and the inclusion path and its verifier coincide with the *Crypto Guide*’s Definition “Authentication path” and Definition “Path verification” (§“Authentication paths and membership proofs”) at index $j - 1$, with the bits b_t fixed by j instead of carried in the path.

Theorem 3.14 (Prefix roots from one path). For $1 \leq j \leq n$, from x_j and the inclusion path of leaf j in M_n a verifier computes rt_j and, for $j \geq 2$, rt_{j-1} , the roots of the ranges over $(x_1, \dots, x_j)$ and $(x_1, \dots, x_{j-1})$.

Proof. For rt_j , by induction on n , following ePrint 2019/226, Theorem 7, with siblings bottom-up. If $j = n$, $\text{rt}_j = \text{rt}_n$ is the recomputed root y_m . Otherwise $n > 1$; let 2^i be the largest power of two strictly below n , so that the root’s children are M_{2^i} and a range over $n - 2^i$ values.

1. If $j = 2^i$, rt_j is the value of the root’s left child, which is y_{m-1} .

2. If $j < 2^i$, the path without its top sibling is the inclusion path of leaf j in the left child M_{2^i} , and the hypothesis applies.
3. If $2^i < j < n$, then $j - 1 \geq 2^i$ and $j < 2^{i+1}$, so 2^i is also the largest power of two strictly below j . The top sibling is P , the value of M_{2^i} , and the path without it is the inclusion path of leaf $j - 2^i$ in the right child; by hypothesis it yields the root r' over $(x_{2^i+1}, \dots, x_j)$, and $\text{rt}_j = H(P \parallel r')$ by Definition 3.1.

For rt_{j-1} , the bottom-up fold of Lemma 3.15, whose proof does not use this theorem. The paper's proof lists the path top-down with the path nodes included, while its Algorithm 3 outputs the siblings bottom-up; the bottom-up format of Definition 3.13 is the one fixed here. $\square$

Lemma 3.15 (Left siblings cover the prefix). *Let $b \in \{R, L\}$, $1 \leq j \leq n$, and consider the inclusion path of leaf j in M_n^b . Its left siblings are subtrees whose leaf runs partition $\{1, \dots, j-1\}$, higher siblings covering earlier leaves; its right siblings cover $\{j+1, \dots, n\}$ and enter none of the folds below. Let $j \geq 2$ and let leaf j lie in peak P_i .*

1. Under Bag_R the left siblings, from the top down, are the peaks of $(x_1, \dots, x_{j-1})$. The bottom-up fold (set r to the lowest left sibling, then $r \leftarrow H(\pi \parallel r)$ for each further left sibling π upwards) yields rt_{j-1} ; started instead from $r = x_j$ and applied to every left sibling, it yields rt_j .
2. Under Bag_L , for $i \geq 2$ the topmost left sibling is $\text{Bag}_L(P_1, \dots, P_{i-1})$ and the others lie in P_i , with sizes the terms of the binary expansion of $j-1-s_i$; for $i=1$ all left siblings lie in P_1 . The top-down fold (set r to the topmost left sibling, then $r \leftarrow H(r \parallel \pi)$ for each further left sibling π downwards) yields rt_{j-1}^L .

Proof. A leaf $l < j$ leaves the path of leaf j at their lowest common ancestor, in whose left child l lies; that left child is a left sibling on the path. Distinct ancestors give disjoint runs, higher ancestors earlier runs. The same argument for $l > j$ gives the right siblings. Inside a perfect subtree the left siblings of the leaf at offset o from its first leaf are perfect subtrees whose sizes are the terms of the binary expansion of o , largest highest.

Leaf j lies at offset $o = j-1-s_i$ in P_i , and $j-1 = s_i + o$, so the peaks of $(x_1, \dots, x_{j-1})$ are $P_1, \dots, P_{i-1}$ followed by the left siblings inside P_i , top-down. Under R the siblings $P_{i-1}, \dots, P_1$ lie above P_i as left children of bagging nodes (proof of Lemma 3.7); the bottom-up fold is then the right fold of the peaks of $(x_1, \dots, x_{j-1})$, which is rt_{j-1} by Proposition 3.6. Started from x_j , it is the right fold of those peaks followed by x_j . The largest power of two strictly below j is the highest term of $j-1$, the size of the first of these peaks, and the recursion of Definition 3.1 on j splits off the peaks of $j-1$ in turn and ends at the single leaf x_j ; this fold is therefore rt_j . Under L with $i \geq 2$ the root of P_i is the right child of the bagging node $H(\text{Bag}_L(P_1, \dots, P_{i-1}) \parallel P_i)$, and every higher node on the path is a left child; the top-down fold is the left fold of the peaks of $(x_1, \dots, x_{j-1})$. For $i=1$ the root of P_1 is a left child and every higher node on the path too. $\square$

Construction 3.16 (Prefix root). Procedure $\text{PrefixRoot}_b(n, j, \pi)$ takes $b \in \{R, L\}$, $2 \leq j \leq n$ and a sequence π of κ -bit values with $|\pi| = \text{depth}_b(n, j)$, with sides σ_t as in Definition 3.13. It selects the π_t with $\sigma_t = \text{left}$ and outputs their bottom-up fold for $b = R$ and their top-down fold for $b = L$ (Lemma 3.15).

Correctness of the construction is Lemma 3.15. The construction corrects the printed pseudocode of ePrint 2019/226, Algorithm 6, which uses an undefined index, ends in a vestigial boolean comparison, and has a call signature that does not match its call in Algorithm 2, and it fixes the siblings bottom-up.

Theorem 3.17 (Position binding). *Let $b \in \{R, L\}$, $n \geq 1$ and $1 \leq j \leq n$. From any $\text{rt} \in \{0, 1\}^\kappa$, chosen by the adversary, and two pairs $(x, \pi) \neq (x', \pi')$ both accepted by $\text{Verify}_b(\text{rt}, n, j, \cdot, \cdot)$, a collision of H is computed with at most $2 \text{depth}_b(n, j)$ evaluations of H . In particular $x \neq x'$ yields a collision, and the siblings of an accepted path and every value it recomputes are functions of (rt, b, n, j) , barring a collision.*

Proof. Both chains $y_0, \dots, y_m$ and $y'_0, \dots, y'_m$ of Definition 3.13 have the length $m = \text{depth}_b(n, j)$ and the sides fixed by (b, n, j) , and $y_m = y'_m = \text{rt}$. Since $(x, \pi) \neq (x', \pi')$, some step has differing inputs; let t be the highest step with $(y_{t-1}, \pi_t) \neq (y'_{t-1}, \pi'_t)$. Then $y_t = y'_t$: for $t = m$ both equal rt , and for $t < m$ the inputs of step $t + 1$ agree. The two 2κ -bit inputs of step t place their operands on the same sides and differ, and H maps both to y_t : a collision. For $n = 1$ the path is empty and Verify_b accepts only $x = \text{rt}$, so no two distinct pairs are accepted. The extractor is deterministic and the reduction tight in the sense of the *Crypto Guide*, §“Reductions”; it is the first-level-of-agreement argument of the *Crypto Guide*, Proposition “Binding of the root” (§“Binary Merkle trees”) and §“Soundness: forging a path implies a collision”. $\square$

The theorem extends the *Crypto Guide*’s Theorem “Merkle trees are succinct position-binding vector commitments” (§“Vector commitments”) from perfect trees to the shapes fixed by n and the bagging order. It is stated for a root chosen by the adversary, not for a fixed honest root as in the *Crypto Guide*’s Remark “Scope: a fixed honest root”: in FlyClient the root of the head comes from the prover.

Corollary 3.18 (Prefix binding and root sensitivity). *Let $b \in \{R, L\}$ and $1 \leq j \leq n$.*

1. *If $\text{rt} = \text{rt}_n^b$ over $(x_1, \dots, x_n)$ and Verify_b accepts (x, π) at position j , then $x = x_j$ and, for $j \geq 2$, $\text{PrefixRoot}_b(n, j, \pi) = \text{rt}_{j-1}^b$ over $(x_1, \dots, x_{j-1})$, barring a collision: one accepted path binds $x_1, \dots, x_j$ as a prefix of the committed sequence.*
2. *If $(x'_1, \dots, x'_m)$ differs from $(x_1, \dots, x_m)$ at some $i \leq m$, then $\text{rt}_m^b(x') \neq \text{rt}_m^b(x)$, barring a collision: changing x_i changes every root rt_m^b with $m \geq i$.*

When x_i is the hash of block i , binding the blocks themselves requires in addition collision resistance of that hash.

Proof. (1) The honest pair (x_j, π^*) is accepted. By Theorem 3.17 the pairs (x, π) and (x_j, π^*) are equal barring a collision, so $x = x_j$ and the fold of Construction 3.16 on π is the fold on π^* , which

is rt_{j-1}^b by Lemma 3.15. (2) The two trees have the same shape M_m^b . The honest paths of leaf i in both are accepted against the common root, with $x_i \neq x'_i$; Theorem 3.17 yields a collision. $\square$

Part (1) is ePrint 2019/226, Corollary 3, and part (2) its Corollary 4, stated for both bagging orders and with the collision named.

3.5 Aggregating ranges

A verifier that samples leaves by cumulative weight needs each path to bind the sampled leaf's weight interval, and a verifier that constrains unsampled leaves needs aggregates over their runs; both follow from metadata propagated up the tree (ePrint 2019/226, Section 2 and Section 6).

Definition 3.19 (Aggregating range). An *aggregating range* extends Definition 3.1, after ZIP 221, “Specification”, which propagates metadata “by either summing the metadata of both children, or inheriting the metadata of a specific child”. It is given by:

1. a set of *node tuples* $v = (v.h, v.f_1, \dots, v.f_r)$ with $v.h \in \{0, 1\}^k$;
2. a *serialisation* ser from tuples to bit strings that is injective and prefix-free, no value of ser being a proper prefix of another, so that $\text{ser}(\ell) \parallel \text{ser}(r)$ determines the pair (ℓ, r) ;
3. a *leaf map* from a leaf's data to its tuple;
4. for each field f_i one rule: inherit from the left child, inherit from the right child, or sum, in $\mathbb{Z}$ or in $\mathbb{Z}/2^{256}\mathbb{Z}$;
5. the *parent map* $\text{par}(\ell, r)$, whose hash field is $H(\text{ser}(\ell) \parallel \text{ser}(r))$, over the children's full serialisations, and whose other fields follow the rules.

A labelling of the shape M_n^b with leaf tuples at the leaves is *consistent* when every internal node is par of its children; its *root* is the root's tuple. Inclusion paths and Verify_b are those of Definition 3.13 with tuples in place of values and par in place of $H(\cdot \parallel \cdot)$; Verify_b accepts if and only if the recomputed root tuple equals the given one.

A serialisation of fixed width is prefix-free. Injectivity alone does not suffice: with $\text{ser}(x) = 0$, $\text{ser}(y) = 01$ and $\text{ser}(z) = 10$, the pairs (x, z) and (y, x) have the same concatenation 010 .

Lemma 3.20 (Aggregate binding). *Let ser be injective and prefix-free, $b \in \{R, L\}$ and $n \geq 1$.*

1. *From two consistent labellings of M_n^b with equal root tuples that differ at some node, a collision of H is computed; barring a collision, equal roots imply agreement on every field of every node.*
2. *From any root tuple, chosen by the adversary, and two distinct accepted paths (leaf tuple and sibling tuples) at one position j , a collision of H is computed: an accepted path binds the full tuple of every node it recomputes and of every sibling it carries.*

3. For $2 \leq j \leq n$ and the inclusion path of leaf j in a consistent labelling, the folds of Lemma 3.15 run with par in place of $H(\cdot \parallel \cdot)$ yield the root tuple of the consistent labelling of M_{j-1}^b over the first $j - 1$ leaf tuples.

Proof. Unique decoding: if $\text{ser}(\ell) \parallel \text{ser}(r) = \text{ser}(\ell') \parallel \text{ser}(r')$, one of $\text{ser}(\ell)$, $\text{ser}(\ell')$ is a prefix of the other, hence they are equal, and $\ell = \ell'$ by injectivity; then $\text{ser}(r) = \text{ser}(r')$ and $r = r'$.

(1) Take a node at which the labellings differ and whose parent they agree on; one exists because they agree at the root. The parent tuples are equal, so their hash fields are, while the child pairs differ; by unique decoding the two hash inputs differ: a collision.

(2) As in the proof of Theorem 3.17, with tuples: at the highest step whose inputs differ, the outputs are equal tuples, hence have equal hash fields, and the inputs are distinct pairs, hence by unique decoding distinct strings: a collision.

(3) In a consistent labelling the tuple of every subtree, a bagging node included, is determined by its leaf tuples through par . By Lemma 3.15 the left siblings are, under R , the peak tuples of the consistent labelling over the first $j - 1$ leaves, and so are they under L with $i \leq 2$. Under L with $i \geq 3$, the topmost is the tuple of the bagging node $\text{Bag}_L(P_1, \dots, P_{i-1})$ of that labelling, and the others are its peak tuples inside P_i . The folds of that lemma, run with par , therefore yield the root tuple of M_{j-1}^b . The engine of (1) and (2) is the *Crypto Guide*, Proposition “Binding of the root” (§“Binary Merkle trees”). $\square$

Lemma 3.21 (Cumulative-weight interval of a leaf). *Let ser be injective and prefix-free, let w be a summed field, w_j the w -field of the tuple of leaf j , and W the sum of the w -fields of the left siblings on an accepted path of leaf j .*

1. *Binding, for any root tuple chosen by the adversary: barring a collision, W and w_j are functions of the root, b , n and j . One accepted path therefore binds the interval $[W, W + w_j)$ of leaf j .*
2. *Meaning, for the path of a consistent labelling: $W = \sum_{i < j} w_i$, under either bagging order. For a sum taken modulo 2^{256} , this holds as an identity of integers when the sum of w over all n leaves is below 2^{256} .*

Part (2) is not claimed for a root chosen by the adversary, whose siblings off the path are never recomputed from their children.

Proof. (1) Lemma 3.20(2). (2) In a consistent labelling the summed field of a node is the sum over its leaves, by induction on height; with the total below 2^{256} no reduction occurs. The left siblings partition $\{1, \dots, j - 1\}$ by Lemma 3.15, a property of any binary tree. $\square$

Two instances follow. The difficulty Merkle mountain range of ePrint 2019/226, whose nodes aggregate difficulty and time, is Definition 5.6 in §5.3. The node vector of ZIP 221, whose nodes aggregate work, heights and pool data, is Definition 6.3 in §6.3.

4 The FlyClient security model

This section fixes the model in which the FlyClient theorem is stated: the variable-difficulty backbone with its unit of work, the adversary and persistence; the retarget rule, the attack it excludes and the limit of that exclusion for a sampling verifier; the (c, L) assumption; and the definitions of security and succinctness for chain proofs.

4.1 The variable-difficulty backbone model

The model of this section is the one ePrint 2019/226 adopts in Section 3.1 and Section 3.2, after Garay, Kiayias and Leonardos (the variable-difficulty backbone, CRYPTO 2017), and it is stated as the paper adopts it. Whether a Zcash chain satisfies its hypotheses is classified in §7, not here.

Definition 4.1 (Variable-difficulty backbone model). Fix the security parameter λ (*Math Guide*, §“The security parameter”) and a hash length κ . The execution proceeds in rounds $r = 1, 2, \dots$. The proof-of-work function is a random oracle $H: \{0, 1\}^* \rightarrow \{0, 1\}^\kappa$ (*Crypto Guide*, §“The random oracle model”), its outputs read as integers in $[0, 2^\kappa)$. In round r there are ν_r honest and t_r adversarial *active players*, and each active player makes q sequential queries to H per round. A block B declares a *target* $T \in [1, 2^\kappa]$ in its header, and B carries a *valid proof of work* at T if and only if $H(B) < T$. A message broadcast in round r is delivered to every player in round $r + 1$, in an order the adversary chooses. The honest participation sequence $(\nu_r)_r$ is (γ, s) -*respecting*: for every set S of at most s consecutive rounds,

$$\max_{r \in S} \nu_r \leq \gamma \min_{r \in S} \nu_r.$$

Each block also records the round in which it was mined, which Definition 4.5 reads.

The paper’s paraphrase in Section 3.1 omits “consecutive”; read literally, for $s \geq 3$ it bounds participation over the whole execution.

The paper writes λ for the oracle output length and n_r for the honest player count; here the length is κ , λ being the security parameter, and the count is ν_r , n being a leaf count (§1.2).

Definition 4.2 (Difficulty level and total work). A single query to H meets target T with probability $T/2^\kappa$. The *difficulty level* of T is

$$D(T) = 2^\kappa / T,$$

the expected number of queries per block valid at T . A harder target is a smaller T and a larger $D(T)$: the level is a work-like quantity inversely proportional to the target and is never the target itself. The *total work* of a finite sequence of blocks $B_1, \dots, B_m$ with declared targets $T_1, \dots, T_m$ is

$$\omega = \sum_{i=1}^m D(T_i),$$

the paper’s *cumulative difficulty*. A chain is *heavier* than another when its total work is larger.

The paper also calls a level a “target” in its Definition 7; in this volume D denotes only a level and T only a target. The Zcash analogue, the work of a block computed from its compact target, is defined in the *Consensus Guide*, §“Work, and the best chain”, from protocol specification, §“Definition of Work”. It is a different normalisation of the same ordering and is not used by the model.

Definition 4.3 (Adversary). Fix μ with $0 < \mu < 1$. The *adversary* is a PPT algorithm (*Math Guide*, §“Algorithms, running time, and PPT”; *Crypto Guide*, §“Adversaries and the security parameter”) that controls the t_r adversarial players of each round r , with $t_r < \mu v_r$, so that it makes at most $t_r q$ queries to H in round r . It is *rushing*: in each round it receives every honest message of the round before choosing its own, and it orders the delivery of all messages. Its goal is to make a light client accept a chain whose total work is at least that of the honest chain.

The prior knowledge of the verifier is fixed once, in Definition 4.16.

Assumption 4.4 (Persistence and liveness). The backbone parameters $(\mu, \gamma, s, \tau, E, f)$, with τ , E and f the constants of Definition 4.5, are such that there is a *persistence depth* ℓ with, except with probability negligible in λ :

1. (persistence) for all honest players P_1, P_2 and rounds $r_1 \leq r_2$, the chain of P_1 at round r_1 with its last ℓ blocks removed is a prefix of the chain of P_2 at round r_2 ;
2. (liveness) a transaction given to every honest player for a number of consecutive rounds fixed by the parameters lies in that stable prefix of every honest chain.

ePrint 2019/226, Section 3.1 (p. 7), adopts this as a parameter choice, citing Theorems 26 and 27 of its reference [22], Garay, Kiayias and Leonardos, “The Bitcoin backbone protocol: analysis and applications” (EUROCRYPT 2015), a fixed-difficulty analysis; it names no theorem of its reference [23] (CRYPTO 2017), the source of the variable-difficulty model. No cited theorem establishes the assumption in the model of Definition 4.1.

The tips of honest chains need not coincide: persistence gives agreement only on all but the last ℓ blocks. The paper’s sentence that persistence and liveness give “a single chain adopted by all honest full nodes” overstates this; the assumption states the corrected form. The persistence depth ℓ is distinct from the catch parameter k of §5.

4.2 Retargeting and the difficulty-raising attack

Definition 4.5 (Target recalculation with dampening). Fix constants $E \geq 1$ (the *retarget-epoch length*, in blocks), $\tau > 1$ (the *dampening factor*), an initial participation estimate $v_0 > 0$ and an initial target T_0 . Targets are constant within each *retarget epoch* of E consecutive blocks. At the end of an epoch whose E blocks were mined at target T in Δ rounds, read from the rounds the blocks record, the *participation estimate* is

$$\hat{v}(T, \Delta) = \frac{2^* E}{q T \Delta},$$

and the target of the next epoch is

$$\text{Retarget}(T, \Delta) = \begin{cases} T/\tau & \text{if } (\nu_0/\hat{\nu}(T, \Delta)) T_0 < T/\tau, \\ \tau T & \text{if } (\nu_0/\hat{\nu}(T, \Delta)) T_0 > \tau T, \\ (\nu_0/\hat{\nu}(T, \Delta)) T_0 & \text{otherwise,} \end{cases}$$

that is, the value $(\nu_0/\hat{\nu}) T_0$ clamped to $[T/\tau, \tau T]$. Let $f = q\nu_0 T_0/2^\kappa$, the expected number of blocks per round of ν_0 players at T_0 . A chain *respects* the rule when every epoch's target is the image under Retarget of the previous epoch's target and round span.

The unclamped value is the *raw retarget*

$$\frac{\nu_0}{\hat{\nu}(T, \Delta)} T_0 = \frac{\nu_0 T_0 q T \Delta}{2^\kappa E} = \frac{T \Delta f}{E},$$

which restores f blocks per round at the estimated participation. The current target T is an input of the rule; the paper's list of constants omits it. The definition is ePrint 2019/226, Definition 1 (Section 3.1, p. 7), after Garay, Kiayias and Leonardos, Definition 2. The function is written Retarget rather than the paper's $D(n_0, T_0)$, because D is a difficulty level, and ν_0 and T_0 are the initial constants of the cited definition, not the previous-epoch quantities of the gloss in the paper's Definition 1. The retarget epoch is distinct from the upgrade epoch of §6.

Under variable difficulty the total work of a chain, not its length, decides between chains. An adversary may therefore mine fewer blocks at higher levels and use the larger variance of a sum of few heavy terms to exceed a more powerful honest chain: the *difficulty-raising attack* of Bahack (arXiv:1312.7013), recalled in ePrint 2019/226, Section 3.1.

Proposition 4.6 (Success of the difficulty-raising attack). *Model block discovery by exponential clocks (Consensus Guide, §“Memoryless clocks and the attribution sequence”): honest queries arrive at rate Λ per unit time and adversarial queries at rate $\Lambda/3$, one third of the honest power; each query succeeds independently with probability $1/D$ at level D , so that the successes of a party querying at rate Λ' at level D form an exponential clock of rate Λ'/D . Fix a time $t_0 > 0$ and let $W = \Lambda t_0$.*

- (i) *The expected total work of the blocks the honest players find in $[0, t_0]$ is W , whatever their targets, so the expected total work of the honest chain at t_0 is at most W .*
- (ii) *Suppose the adversary may mine one block at an arbitrary level, in violation of Definition 4.5, and sets its level to $D = W$. Then it finds that block, whose work alone equals W , before t_0 with probability*

$$1 - e^{-1/3} \approx 0.2835,$$

about 28.3 per cent, a constant independent of λ .

Proof. (i) An honest query at target T yields a block of work $D(T)$ with probability $T/2^\kappa = 1/D(T)$, hence has expected work 1 whatever the target. By linearity of expectation the expected work found in $[0, t_0]$ equals the expected number of honest queries in $[0, t_0]$, which is $\Lambda t_0 = W$

(*Math Guide*, §“Exponential clocks, Poisson counts, and the Chernoff method”, Theorem “Poisson arrival counts”). The honest chain carries at most the work found.

(ii) At level $D = W$ the adversary’s successes form an exponential clock of rate $(\Lambda/3)/W = 1/(3t_0)$. Its first arrival exceeds t_0 with probability $e^{-t_0/(3t_0)} = e^{-1/3}$ (*Math Guide*, same section, Lemma “Tail, mean, and memorylessness”), so it occurs before t_0 with probability $1 - e^{-1/3}$. $\square$

In the discrete query model the same statement holds as an inequality: if $W/3$ is an integer, the adversary’s $W/3$ queries at level W all fail with probability $(1 - 1/W)^{W/3} \leq e^{-1/3}$, since $1 - x \leq e^{-x}$, so it succeeds with probability at least $1 - e^{-1/3}$. Since $\log(1 - x) \geq -x - x^2$ for $0 \leq x \leq 1/2$, $(1 - 1/W)^{W/3} \geq e^{-1/3 - 1/(3W)} \geq e^{-1/3}(1 - 1/(3W))$, so for every such $W \geq 3$ the two probabilities differ by at most $e^{-1/3}/(3W) < 1/W$. The paper states “roughly 28%” without derivation (ePrint 2019/226, Section 3.1, p. 7); the comparison, as in the paper, is with the expected honest work.

Remark 4.7 (The clamp as a defence). Definition 4.5 excludes the attack of Proposition 4.6: a target changes only at epoch ends and by at most the factor τ , so no block may declare an arbitrary level. The paper, citing Garay, Kiayias and Leonardos, states that this rule suffices against difficulty-raising attacks for full nodes, which see every transition and reject a chain that violates it (ePrint 2019/226, Section 3.1, p. 7); that sufficiency is the cited authors’ result and is not proved here. A verifier that samples few blocks sees few transitions. The clamp constrains the transitions it does not see only through data that the chain commits to, which §5.3 constructs; no conclusion about that construction is drawn here. The Zcash retarget rule is a different function, a per-block rule over an averaging window, defined in protocol specification, §“Difficulty adjustment”, and described in the *Consensus Guide*, §“Difficulty in motion”; it is not an instance of Definition 4.5.

4.3 The (c, L) -adversary

Fix an honest player and let C be its chain at round r , the *honest chain*. A *fork* of a chain C' against C has *fork point* a , the last block common to C and C' ; its blocks are those of C' after a . A block of the fork is *validly mined* if it carries a valid proof of work at its declared target (Definition 4.1), and *invalid* otherwise; its *validly mined work* is the total work of its validly mined blocks.

Assumption 4.8 ((c, L) -adversary). The printed form (ePrint 2019/226, Assumption 2, Section 3.2, p. 8) reads: “There exists no adversary in the variable-difficulty model that respects the target recalculation function from Definition 1 and can, with non-negligible probability, produce a fork that contains more than L blocks such that a c fraction of the difficulty weight in these blocks is honest.”

The operative form, for $0 < c < 1$ and $L \geq 1$: for every PPT adversary respecting Definition 4.5 in the model of Definition 4.1, except with probability negligible in λ , every fork it produces that has more than L blocks after its fork point a , and total work after a at least the total work of C after a at the same round, has validly mined work less than c times its total work after a . Such a fork is therefore at least a $(1 - c)$ fraction invalid by weight.

The operative form departs from the printed one in three stated respects. The word “honest” in the printed form means validly mined. The length condition is “more than L blocks”,

the printed assumption’s own wording, where the paper’s preceding sentence (Section 3.2, p. 8) says “of length L or longer”; the choice is consistent with checking the last L blocks in full (Remark 4.10). The operative form quantifies only over forks whose work after the fork point reaches that of the honest chain, following the same sentence (“a c fraction of the honest forks weight”): the unrestricted reading fails for every $\mu > 0$, since the adversary’s own fully valid private fork of more than L blocks has valid fraction 1, and only forks that reach the honest weight can win under Definition 4.16.

Remark 4.9 (Two readings of c). In Assumption 4.8 the parameter c bounds the validly mined fraction of a competitive long fork’s weight. The paper’s evaluation (ePrint 2019/226, Section 7.1, p. 20, and Section 7.2) reads it instead as a power ratio ρ , adversarial to honest mining power, so that the adversary’s share of total power is $\rho/(1 + \rho)$: one third at $\rho = 1/2$, and $9/19 \approx 0.4737$, about 47.4 per cent, at $\rho = 0.9$, where the paper prints 47.3 per cent by truncation. The identification of c with ρ is not part of the assumption; it needs a bridge from mining power to fork weight, which Remark 4.13 classifies.

Remark 4.10 (Short forks and the checked suffix). Assumption 4.8 constrains only forks of more than L blocks, by design: over a short window the variance of mining lets even an adversary with small power produce a heavier short fork with non-negligible probability (ePrint 2019/226, Section 3.2, p. 8). A verifier relying on the assumption must therefore check in full a suffix of the claimed chain containing at least its last L blocks. At constant difficulty the paper takes $L = \delta n$; under variable difficulty it states the checked suffix as a weight fraction δ (Section 6, p. 17), leaving its relation to the block count L implicit. The size of the suffix under variable difficulty is fixed in §5.4 (Remark 5.24); no conclusion about it is drawn here.

Remark 4.11 (Lemma 1 of the paper). The paper’s only derivation of Assumption 4.8 from mining power is its Lemma 1 with Corollary 1 (ePrint 2019/226, pp. 8–9), stated here to register their defect. Lemma 1 as printed: in the constant-difficulty backbone, if X is the number of blocks any adversary mines while the honest chain adopts L blocks, and the adversary’s block rate is at most μ times the honest adoption rate, then for $c > \mu$,

$$\Pr[X \geq cL] \leq e^{L(c-\mu)} (c/\mu)^{-cL}.$$

Corollary 1: for $L = \Theta(\lambda)$ and every μ there is $c < 1$ for which the assumption holds at constant difficulty.

The printed bound is the Chernoff bound of a Poisson variable of mean μL (*Math Guide*, §“Exponential clocks, Poisson counts, and the Chernoff method”, Corollary “Chernoff bound for a Poisson variable”). The proof asserts without proof that such a variable models X , and it switches to the undefined symbols $1 - \delta$ and n , a footnote setting $\mu = 1 - \delta$. Under independent exponential block clocks the count X is negative binomial (same section, Theorem “Negative-binomial arrival count”), and the printed expression is not an upper bound for it in general: the *Math Guide*’s Example “The Poisson expression is not a bound for the arrival count”, in the same section, exhibits $\mu = 1/2$, $c = 3/4$, $L = 200$ and $L = 400$ with the exact tail above the printed expression. Lemma 1 and Corollary 1 are therefore unproved as printed; Proposition 4.12 replaces them in the clock model.

The clock model identifies, at constant difficulty, honest adoption and adversarial block discovery after a fork point a with independent exponential clocks started at a , of rates 1 and μ , the rates normalised to the honest adoption rate (*Consensus Guide*, §“Memoryless clocks and the attribution sequence”). For $M \geq 1$ let $X_{a,M}$ be the number of adversarial arrivals strictly before the M -th honest arrival after a . The count $X_{a,M}$ is nondecreasing in M .

Proposition 4.12 (Constant-difficulty bound). *In the clock model, count as validly mined in a fork from a only blocks the adversary finds after a . Let $0 < \mu < c < 1$ and*

$$\beta = \left(\frac{1+c}{1+\mu} \right)^{1+c} \left(\frac{\mu}{c} \right)^c, \quad \theta = \left(\frac{c(1+\mu)}{\mu(1+c)} \right)^c.$$

Then:

- (i) *for every fork point a and $M \geq 1$, $\Pr[X_{a,M} \geq cM] \leq \beta^M$, and $\beta < 1$;*
- (ii) *for any set A of candidate fork points, the probability that for some $a \in A$ a fork from a exists at some time with more than L blocks, total work after a at least that of the honest chain, and validly mined fraction at least c , is at most*

$$|A| \frac{(1+\theta) \beta^{L+1}}{1-\beta}.$$

For $|A|$ polynomial in λ and $L = \Theta(\lambda)$ this is negligible in λ , so Assumption 4.8 holds in the clock model, which recovers Corollary 1 there. The proposition says nothing about the variable-difficulty backbone.

Proof. (i) By memorylessness the clocks restart at a , and the bound is the *Math Guide*’s Theorem “Chernoff bound for the arrival count”, with the law of Theorem “Negative-binomial arrival count” (§“Exponential clocks, Poisson counts, and the Chernoff method”), which also shows $\beta < 1$. That proof attains β at $e^{t^*} = c(1+\mu)/(\mu(1+c)) > 1$ as $e^{-t^*c} \Phi_1(t^*) = \beta$, where $\Phi_M(t) = (1+\mu-\mu e^t)^{-M}$ is the moment generating function of $X_{a,M}$; note that $\theta = e^{t^*c}$.

(ii) Fix $a \in A$. At constant difficulty the work of a block is constant, so work is block count. Consider a time at which the honest chain has m blocks after a , and a fork from a at that time with L'' blocks, total work at least that of the honest chain and validly mined fraction at least c . Then $L'' \geq \max(m, L+1)$. The validly mined blocks were found by the adversary after a and before the $(m+1)$ -th honest arrival, so there are at most $X_{a,m+1}$ of them, and the fraction condition gives

$$X_{a,m+1} \geq cL'' \geq c \max(m, L+1).$$

If $m \leq L$, then $X_{a,L+1} \geq X_{a,m+1} \geq c(L+1)$, an event of probability at most β^{L+1} by (i). If $m = M-1$ with $M \geq L+2$, the event is $X_{a,M} \geq cM - c$; the Chernoff method (*Math Guide*, same section, Theorem “The Chernoff method”) at t^* bounds its probability by

$$e^{-t^*(cM-c)} \Phi_M(t^*) = e^{t^*c} (e^{-t^*c} \Phi_1(t^*))^M = \theta \beta^M.$$

The union bound (*Math Guide*, §“The union bound and a birthday calculation”) over these events gives, for the fork point a ,

$$\beta^{L+1} + \theta \sum_{M \geq L+2} \beta^M \leq \beta^{L+1} + \frac{\theta \beta^{L+2}}{1 - \beta} \leq \frac{(1 + \theta) \beta^{L+1}}{1 - \beta},$$

and the union over A contributes the factor $|A|$. For negligibility, β and θ are constants with $\beta < 1$, so for $L = \Theta(\lambda)$ the bound is a polynomial in λ times a function exponentially small in λ , which is negligible (*Math Guide*, §“Polynomial, exponential, and negligible functions”). $\square$

The source omits step (ii). The identification of the fork experiment with the clock model is the paper’s (proof of Corollary 1) and is not derived from Definition 4.1; the counting hypothesis on validly mined blocks is returned to in §5.2.

Remark 4.13 (Status of the (c, L) assumption). The paper conjectures that (c, L) can be derived from the backbone parameters $(\mu, \gamma, s, \tau, E, f)$ and leaves the derivation out of scope (“not trivial and out of the scope of this paper”, ePrint 2019/226, Section 3.2, p. 8). No such derivation exists: in the classes of Definition 1.3 it is an open problem. Proposition 4.12 covers only the constant-difficulty clock model and does not connect the variable-difficulty backbone to Assumption 4.8. Every result of this volume that uses the assumption is conditional on it.

4.4 Security and succinctness of chain proofs

Definition 4.14 (SPV predicate). Let $C = (B_0, \dots, B_\eta)$ be a chain with genesis block B_0 and head B_η . The *SPV predicate* of C is 1 if and only if every block $B_{\eta'}$ carries a valid proof of work at the target that Definition 4.5 assigns to its position, and for $1 \leq \eta' \leq \eta$ the hash of $B_{\eta'-1}$ is contained in the header of $B_{\eta'}$. The statement a prover proves at round r is that it knows a chain ending in B_η , with total work ω (Definition 4.2), whose SPV predicate is 1.

The definition is ePrint 2019/226, Definition 2 (Section 3.3, p. 9). The paper writes B_n and D for the head and the cumulative difficulty; here η is a height and ω the total work, n and D being reserved.

Definition 4.15 (Equivalence up to the last blocks). Two chains are *equivalent up to the last ℓ blocks*, with ℓ the persistence depth of Assumption 4.4, if removing the last ℓ blocks of each leaves one a prefix of the other. The verifier treats valid proofs of equivalent chains as proofs of one chain and takes the one of greatest total work as its representative; no predicate is evaluated on the last ℓ blocks.

The paper writes k for this depth (Section 3.3, p. 9); here ℓ, k being the catch parameter.

Definition 4.16 (Verifier setting). The verifier is a light client (Definition 2.1) whose only prior knowledge of the chain is the genesis block B_0 , and which has oracle access to H to check proofs of work. It receives a set $\mathcal{P}$ of proofs from several provers, at least one of which is honest, and accepts the SPV predicate of the proof whose head carries the greatest total work, up to Definition 4.15.

The honest-prover hypothesis places eclipse attacks, in which every prover is adversarial, outside the model (ePrint 2019/226, Section 3.3, p. 9).

Definition 4.17 (Security of a chain proof protocol). A *chain proof protocol* is a pair (P, V) of algorithms with oracle access to H : the prover P maps a chain to a proof, and the verifier V maps a set of proofs to an output (B, ω) , a head and a total work, or to rejection. The *security game*, for an environment, a PPT adversary $\mathcal{A}$ as in Definition 4.3 and a round r , is as follows.

1. The backbone execution of Definition 4.1 runs to round r , with $\mathcal{A}$ controlling the adversarial players.
2. At the start of round r the verifier receives a set $\mathcal{P}$ containing at least one proof that P computes on the chain of an honest full node; $\mathcal{A}$ chooses the other proofs.
3. The adversary wins if either of the following clauses fails.
 - (a) At the end of round r the verifier outputs the SPV predicate of greatest total work for some head B : a pair (B, ω) such that the SPV predicate holds for a chain ending in B of total work ω , and no proof in $\mathcal{P}$ of a chain whose SPV predicate holds carries more total work, up to Definition 4.15.
 - (b) Every honest full node at round r holds a chain of which the chain committed by B , with its last ℓ blocks removed, is a prefix.

The protocol is *secure* if for every environment, every PPT $\mathcal{A}$ and every round r the probability that $\mathcal{A}$ wins is negligible in λ .

The definition states both clauses of ePrint 2019/226, Definition 3 (Section 3.3, p. 9). The game idiom is that of the *Crypto Guide*, §“Security as a game”; the multi-prover game is defined here.

Definition 4.18 (Succinctness). A chain proof protocol (P, V) is *succinct* if for every PPT prover and every round r , every proof produced at round r has size $O(\text{polylog}(N))$ bits, where N is the number of blocks of the honest chain, $\text{polylog}(N) = (\log N)^{O(1)}$ as fixed in §1.2, and the asymptotics are in N with λ , c and L fixed.

The definition is ePrint 2019/226, Definition 4 (p. 9), after Kiayias, Miller and Zindros, “Non-interactive proofs of proof-of-work”, Definition 4.

Definition 4.19 (Non-interactive proof of proof of work). A *non-interactive proof of proof of work* is a chain proof protocol for the SPV predicate in which each prover sends a single message and no message passes from verifier to prover, and which is secure (Definition 4.17) and succinct (Definition 4.18).

The term names this class of protocols. Kiayias, Miller and Zindros use the same name for their superblock construction, a different member of the class.

5 The FlyClient protocol

This section constructs the protocol of ePrint 2019/226 in the model of §4, each object before its first use: the header chain commitment and the check applied to one sampled block, two lemmas on forks, the difficulty Merkle mountain range with its node checks, the sampling distribution, the interactive and the non-interactive protocol, the security theorem with its hypotheses and status, and verification after acceptance. Throughout, $H: \{0, 1\}^* \rightarrow \{0, 1\}^k$ is the hash function of §3, which is also the proof-of-work oracle of Definition 4.1; $\log$ is the natural logarithm and $\text{Adv}_H^{\text{col}}(\mathcal{B})$ the collision advantage of an algorithm $\mathcal{B}$ (*Crypto Guide*, Definition “Collision resistance”, §“Security notions: preimage, second-preimage, and collision resistance”), the security parameter left implicit.

5.1 Header chain commitments and per-sample checks

Blocks are written B_0 (the genesis block), $B_1, B_2, \dots$, with B_η the block at height η and $\text{hdr}(B)$ the header of B .

Construction 5.1 (Header chain commitment). Fix an aggregating range (Definition 3.19) under the bagging Bag_R (Definition 3.5) whose leaf map sends a block B to a node tuple $\text{leaf}(B)$ with hash field $\text{leaf}(B).h = H(\text{hdr}(B))$. Leaf j of every range is $\text{leaf}(B_{j-1})$, so leaf j is the block at height $j - 1$. For every height $\eta \geq 1$ let M_η be the range over the η leaves $\text{leaf}(B_0), \dots, \text{leaf}(B_{\eta-1})$ and rt_η its root tuple. The producer of B_η obtains M_η from $M_{\eta-1}$ by Construction 3.10 applied to $\text{leaf}(B_{\eta-1})$, with the parent map par in place of $H(\cdot \parallel \cdot)$ (M_1 is the single leaf $\text{leaf}(B_0)$), and records the hash field $\text{rt}_\eta.h$ in the header of B_η . The genesis header records no root. A full node accepts B_η only if its header records $\text{rt}_\eta.h$.

The construction has the following invariant: the header of B_η commits, through one root, to exactly the blocks strictly before it, and a head B_η commits to $n = \eta$ leaves, n denoting this leaf count below. The root’s fields other than its hash field are not recorded; a verifier recomputes the root tuple from its two children, which the hash field binds (Lemma 3.20). The range of ePrint 2019/226, Section 2, is the instance whose tuples carry the hash field alone; the paper’s difficulty Merkle mountain range is the instance of Definition 5.6. The one further header field is the paper’s only consensus change (Section 4.2). The paper indexes the committing block and its range inconsistently; the invariant fixes one indexing, and the paper’s is not carried. In the classes of Definition 1.3, this commitment from the genesis block is the paper’s design and no Zcash rule specifies it; the specified commitment is the epoch-scoped instance of “The chain-history commitment” (§6), from which nothing is concluded here.

One sample at index $j \in \{1, \dots, n\}$ against a head B_η must establish three facts: that the sampled block is leaf j under the head’s root, that the leaf’s tuple is the one its header determines, and that the root in the sampled header commits to exactly the first $j - 1$ of the same leaves.

Construction 5.2 (Per-sample check). The inputs are a root tuple rt with leaf count n and an index $1 \leq j \leq n$. The prover sends the header hdr_j of the block at height $j - 1$, the tuple v_j of leaf j , and its inclusion path π in M_n (Definition 3.13) with every sibling as a full tuple. The verifier performs the following checks.

- (a) *Leaf*: $v_j.h = H(\text{hdr}_j)$, and every other field of v_j that the leaf map determines from the header alone equals its value computed from hdr_j .
- (b) *Inclusion*: $\text{Verify}_R(\text{rt}, n, j, v_j, \pi)$ of Definition 3.13, in the tuple form of Definition 3.19, accepts; the side of each sibling and the path length $\text{depth}_R(n, j)$ are fixed by (j, n) .
- (c) *Prefix*: for $j \geq 2$, the hash field of $\text{PrefixRoot}_R(n, j, \pi)$ (Construction 3.16), run with par in place of $H(\cdot \parallel \cdot)$, equals the root recorded in hdr_j ; for $j = 1$, hdr_1 equals the genesis header, which the verifier knows.
- (d) *Proof of work*: hdr_j carries a valid proof of work at the target T it declares, $H(\text{hdr}_j) < T$ (Definition 4.1).

Any failure rejects.

ePrint 2019/226, Algorithm 2, lists checks (b) and (c); checks (a) and (d) come from its Definition 5 and Section 6.1. Check (b) uses the length rule of Definition 3.13, not that of the paper's Algorithm 4.

Proposition 5.3 (Soundness of the per-sample check). *Let rt be any root tuple, possibly chosen by the adversary, with leaf count n , and let $1 \leq j \leq n$. There is an explicit reduction $\mathcal{B}$ such that, for every adversary $\mathcal{A}$ that outputs samples at position j against (rt, n) , the probability that every sample passes Construction 5.2 while one of the following fails is at most $\text{Adv}_H^{\text{col}}(\mathcal{B})$:*

- (i) *any two passing samples at position j carry the same leaf tuple and the same header, so the header is bound to position j of the sequence that rt commits;*
- (ii) *every header-determined field of that leaf tuple is its value computed from that header;*
- (iii) *for $j \geq 2$, the root recorded in that header is the hash field of a prefix-root tuple that is the same for every passing sample at j , hence a function of (rt, n, j) ; if rt is the root of a consistent labelling over $v_1, \dots, v_n$, it is the root of the consistent labelling over $v_1, \dots, v_{j-1}$.*

Proof. The reduction $\mathcal{B}$ runs $\mathcal{A}$ and inspects two passing samples (hdr, v, π) and (hdr', v', π') at position j . (i) If $(v, \pi) \neq (v', \pi')$, Lemma 3.20(2), the tuple form of Theorem 3.17 under the prefix-free serialisation of Definition 3.19, yields a collision at the highest step whose inputs differ. If $v = v'$ but $\text{hdr} \neq \text{hdr}'$, check (a) gives $H(\text{hdr}) = v.h = H(\text{hdr}')$, a collision. (ii) is check (a). (iii) Barring the collisions of (i), $\pi = \pi'$, so the fold of check (c) over the left siblings of π is the same tuple for every passing sample, and check (c) equates its hash field with the recorded root. For a consistent labelling, Lemma 3.15 with Lemma 3.20(3) identifies the fold with the root over $v_1, \dots, v_{j-1}$, as Theorem 3.14 does for plain values. $\square$

5.2 Forks against a committed chain

A claimed chain C *forks* from the honest chain at its fork point, the last common block (§4.3). Its head records the root of a range over the leaf tuples $v'_1, \dots, v'_n$ of its blocks, and C *places* a block B

at leaf position j when $v'_j.h = H(\text{hdr}(B))$.

Lemma 5.4 (Honest blocks after the fork point). *Let the head of C record the hash field of the root of a consistent labelling over $v'_1, \dots, v'_n$. Let B be a block of the honest chain at height η above the fork point, whose header records the hash field of the honest root over the honest leaf tuples $v_1, \dots, v_\eta$, and let C place B at leaf position $j \geq 2$. Suppose that*

1. *the sequences $(v'_1.h, \dots, v'_{j-1}.h)$ and $(v_1.h, \dots, v_\eta.h)$ differ, possibly in length; and*
2. *no block header equals $\text{ser}(\ell) \parallel \text{ser}(r)$ for node tuples ℓ, r (leaf-internal separation).*

Then from a sample of B that passes Construction 5.2 against the head of C an explicit reduction computes a collision of H , or finds a leaf position $i < j$ of C whose tuple v'_i has hash field $H(\text{ser}(\ell) \parallel \text{ser}(r))$ for node tuples ℓ, r . A sample at i passes check (a) only on a header hashing to that value, which by hypothesis 2 is a collision. Consequently, barring a collision, either a sample on B rejects or leaf i does, and that block or leaf belongs, in the analysis of §5.4, to the invalid set of the fork.

Proof. By Proposition 5.3(iii) a passing sample makes the root recorded in $\text{hdr}(B)$ equal to the hash field of the root of the consistent labelling of M_{j-1} over $v'_1, \dots, v'_{j-1}$. That value is also the hash field of the honest root of M_η . Descend both labellings from their roots, keeping a pair of nodes with equal hash fields whose leaf subsequences differ in their hash fields. At a pair of internal nodes, equal hash fields $H(\text{ser}(\ell) \parallel \text{ser}(r)) = H(\text{ser}(\ell') \parallel \text{ser}(r'))$ with $(\ell, r) \neq (\ell', r')$ are a collision by unique decoding (proof of Lemma 3.20); otherwise the children agree as tuples, the leaf subsequences of the left children or of the right children differ, and the descent continues there. Two leaves with equal hash fields do not have different hash-field sequences, so the descent ends at a leaf opposite an internal node. If the leaf is honest, its hash field is $H(\text{hdr})$ for the header hdr of an honest block, and $H(\text{hdr}) = H(\text{ser}(\ell) \parallel \text{ser}(r))$ with distinct inputs by hypothesis 2 is a collision. If the leaf is a leaf v'_i of C , $i \leq j - 1$, opposite an honest internal node with children ℓ, r , then $v'_i.h = H(\text{ser}(\ell) \parallel \text{ser}(r))$, which is the second outcome. The descent takes at most $\max(\lceil \log_2(j-1) \rceil, \lceil \log_2 \eta \rceil)$ steps. $\square$

The lemma is ePrint 2019/226, Section 5.1 with Corollary 4 (here Corollary 3.18), extended to a block placed at a height other than its own, where the two roots have different leaf counts and hypothesis 2 is needed. It concerns honest blocks placed after the fork point and states nothing about blocks the adversary mines; those are the subject of the next lemma.

Lemma 5.5 (No reuse of work across the fork point). *Model H as a random oracle with κ -bit outputs (Crypto Guide, §“The random oracle model”), and let at most q_H queries to H be made in the experiment, by $\mathcal{A}$, by the verification of its output, and by the computation of the prefix root at every position of the output fork after its fork point. Call a query $H(x)$ reused if x is the header of a block B of the output fork after its fork point, x records the hash field of the prefix root of the fork at B ’s position, and the query precedes the first query whose answer is that recorded value. Then*

$$\Pr[\mathcal{A} \text{ outputs a fork with a block mined by a reused query}] \leq q_H^2 2^{-\kappa} + \text{Adv}_H^{\text{col}}(\mathcal{B})$$

for an explicit reduction $\mathcal{B}$. Consequently, except with that probability, only adversarial work done after the fork's prefix is fixed counts against the honest chain.

Proof. By Construction 5.1 the root recorded in a header after the fork point is the hash field of a tuple over the fork's prefix; by Proposition 5.3(iii) that tuple is a function of the fork's root, position and leaf count, barring a collision, which accounts for the term $\text{Adv}_H^{\text{col}}(\mathcal{B})$. Its hash field is the answer of a query, made at the latest during that computation. A reused query fixes a recorded value r before any query has answer r . The first query with answer r has an input not queried before, since a repeated input returns an earlier answer; its answer is uniform on $\{0, 1\}^\kappa$ and independent of all earlier queries, so it equals one of the at most q_H values recorded in earlier query inputs with probability at most $q_H 2^{-\kappa}$. The union bound over the at most q_H queries (*Math Guide*, §“The union bound and a birthday calculation”) gives $q_H^2 2^{-\kappa}$. $\square$

The paper asserts the statement in one sentence (ePrint 2019/226, Section 3.2); the proof above is this volume's. The bound is in the number of oracle queries, not in the mining rate.

5.3 The difficulty Merkle mountain range

A sampling verifier sees few difficulty transitions, so the transitions it does not see must be constrained by committed aggregates.

Definition 5.6 (Difficulty Merkle mountain range). The *difficulty Merkle mountain range* (difficulty MMR) is the instance of Definition 3.19 whose node v is the tuple

$$v = (v.h, v.w, v.t, v.D_{\text{start}}, v.D_{\text{next}}, v.n),$$

with a serialisation that concatenates injective fixed-width encodings of the six fields, hence prefix-free. For a block B whose header declares target T , the leaf map sets

- $v.h = H(\text{hdr}(B))$;
- $v.w = v.D_{\text{start}} = D(T)$, the difficulty level of T (Definition 4.2);
- $v.t$ the number of rounds between the round recorded by the predecessor of B and the round recorded by B (0 for the genesis block);
- $v.D_{\text{next}}$ the level of the target that Definition 4.5 assigns to the block after B ;
- $v.n = 1$.

The parent of ℓ and r has hash field $H(\text{ser}(\ell) \parallel \text{ser}(r))$, over both children's complete tuples; its fields w , t and n are the sums of the children's; it inherits D_{start} from ℓ and D_{next} from r .

The fields h , w , D_{start} and n of a leaf are determined by its header alone and are checked by item (a) of Construction 5.2; the fields t and D_{next} depend on neighbouring blocks and are constrained only by Construction 5.8. The definition makes three precisions against ePrint 2019/226, Definition 7. The paper writes $H(lc, rc)$ without fixing its input; hashing the children's hash fields

alone would leave their other fields unbound. The paper’s “timestamp” t is an elapsed time, and its “targets” w , D_{start} and D_{next} are difficulty levels. The paper’s checks name fields t_{start} , t_{end} and D_{end} that its Definition 7 never declares; the declared aggregate t suffices for every timing check below, so the field set is the declared one. The paper’s data field is omitted, since no check uses it. No encoding of the fields is fixed by the paper.

Lemma 5.7 (Weight binding for the difficulty MMR). *Let a sample at leaf j pass Construction 5.2 against a root rt with leaf count n , the shape being fixed by n under Bag_R . Except with probability $\text{Adv}_H^{\text{col}}(\mathcal{B})$ for an explicit reduction $\mathcal{B}$:*

1. *the leaf weight $v_j.w$ is the difficulty level $D(T)$ of the target T at which the header’s proof of work is valid;*
2. *the interval $[W, W + v_j.w)$, with W the sum of the fields w of the left siblings on the path, is a function of (rt, n, j) .*

Proof. (1) Check (a) sets $v_j.w = D(T)$ for the declared target T , check (d) validates the proof of work at that T , and Proposition 5.3(i) and (ii) bind the header and its fields to position j . (2) Lemma 3.21(1). Whether W equals the true prefix weight is part (2) of that lemma, which holds for a consistent labelling only and is not claimed here. $\square$

Construction 5.8 (Node checks). Assume, with ePrint 2019/226, Section 6.1, that the retarget-epoch length E of Definition 4.5 and the leaf count n are powers of two, so that the retarget epochs are the perfect subtrees of altitude $\log_2 E$. For every parent p on a sample’s path, with children ℓ and r (one of them a carried sibling), the verifier checks:

1. $p = \text{par}(\ell, r)$ under Definition 5.6;
2. $\ell.D_{\text{next}} = r.D_{\text{start}}$;
3. for each of ℓ and r , internal consistency:
 - (a) a node v of altitude below $\log_2 E$ lies inside one retarget epoch and has $v.w = v.n \cdot v.D_{\text{start}}$, and $v.D_{\text{next}} = v.D_{\text{start}}$ unless v contains the last block of its epoch;
 - (b) a node v of altitude $\log_2 E$ is one retarget epoch, has $v.w = E \cdot v.D_{\text{start}}$, and $v.D_{\text{next}} = D(\text{Retarget}(2^x / v.D_{\text{start}}, v.t))$, the level that Definition 4.5 assigns after an epoch of $v.t$ rounds at level $v.D_{\text{start}}$;
 - (c) a node v above altitude $\log_2 E$ is *feasible*: its $(v.t, v.w, v.D_{\text{start}}, v.D_{\text{next}}, v.n)$ is attained by some sequence of clamped retargets (Propositions 5.9 and 5.10).

At the sampled leaf the checks (a) and (d) of Construction 5.2 apply; their effect on w and D_{start} is Lemma 5.7. *Weight-position check*: when a leaf is requested at a point x of cumulative weight, $0 \leq x < rt.w$, the returned leaf j satisfies $W \leq x < W + v_j.w$, with W the sum of the fields w of the left siblings on its path; this interval is bound to (rt, n, j) by Lemma 3.21(1). Any failure rejects.

The construction is ePrint 2019/226, Section 6.1 (“Difficulty MMR Verification”); the paper states the weight-position check only informally (Section 2).

Proposition 5.9 (Feasibility envelope). *Let consecutive retarget epochs $0, \dots, z-1$ have levels $D_0 = D_{\text{start}}, D_1, \dots, D_{z-1}$, and let $D_z = D_{\text{next}} = \tau^y D_{\text{start}}$, where every retarget obeys the clamp of Definition 4.5, so that $D_{e+1}/D_e \in [1/\tau, \tau]$. Then:*

1. $|y| \leq z$ and, for every e ,

$$D_{\text{start}} \max(\tau^{-e}, \tau^{y-z+e}) \leq D_e \leq D_{\text{start}} \min(\tau^e, \tau^{y+z-e});$$

2. for integer y with $z-y$ even both envelopes are attained by clamped sequences, so the total weight $E \sum_{e < z} D_e$ is maximal when the difficulty is raised by τ per epoch and then lowered by τ per epoch to D_{next} , and minimal when it is lowered first and then raised;
3. an epoch at target T that lowers the difficulty maximally (new target τT) lasts $\Delta \geq \tau E/f$ rounds, and one that raises it maximally (new target T/τ) lasts $\Delta \leq E/(f\tau)$ rounds.

Proof. (1) Since $|\log_{\tau} D_{e+1} - \log_{\tau} D_e| \leq 1$, induction on e from epoch 0 gives $|\log_{\tau}(D_e/D_{\text{start}})| \leq e$, and from epoch z gives $|\log_{\tau}(D_e/D_{\text{start}}) - y| \leq z - e$; at $e = 0$ the second bound is $|y| \leq z$. (2) For $z-y$ even the pointwise upper envelope has exponent steps $+1$ up to $e = (y+z)/2$ and -1 after, and the lower envelope steps -1 up to $e = (z-y)/2$ and $+1$ after; both are clamped, and the weight is increasing in each D_e . (3) The raw retarget is $T\Delta f/E$ (§4.2); the clamp is reached from above when $T\Delta f/E \geq \tau T$ and from below when $T\Delta f/E \leq T/\tau$. A script checks the sums against brute force over exponent sequences, and the two thresholds. $\square$

Proposition 5.10 (Feasibility bounds). *In the setting of Proposition 5.9, let y be an integer with $y \geq 0$ and $z-y$ even. If a node spanning the z epochs is attained by a clamped sequence, then*

1. $y \leq z$ (Proposition 5.9(1));
2. $w \leq E D_{\text{start}} \frac{(\tau+1)\tau^{(y+z)/2} - \tau^{1+y} - 1}{\tau-1}$;
3. $w \geq E D_{\text{start}} \frac{\tau^y + \tau - (\tau+1)\tau^{-(z-y)/2}}{\tau-1}$;
4. if $y = z$, then $t \leq zE/(f\tau)$.

Proof. Bounds (2) and (3) are the geometric sums of the envelopes of Proposition 5.9(2); at $y = z$ both reduce to $(\tau^z - 1)/(\tau - 1)$. For (4), $y = z$ forces a maximal raise in each of the z epochs, each of at most $E/(f\tau)$ rounds by Proposition 5.9(3). A script confirms that the closed forms equal the brute-force extremes in all 123 cases with $\tau \in \{2, 4, 3/2\}$, $1 \leq z \leq 11$, $0 \leq y \leq z$ and $z-y$ even; at $\tau = 4$, $y = 0$, $z = 2$ they give 5 and $5/4$ per unit of E . $\square$

The paper states these checks with k and n in place of y and z , which here are the catch parameter and the leaf count (ePrint 2019/226, Section 6.1, pp. 18–19). Three departures are made.

The printed lower bound has exponent $(y + z)/2$ in place of $-(z - y)/2$ and is then negative, -5 against the corrected $5/4$ at $\tau = 4, y = 0, z = 2$. The printed bounds count one level per epoch, whereas the leaf rule $w = D_{\text{start}}$ sums per block, so both scale by E . The printed upper summation overcounts by τ^z at $y = z$, where its closed form is right. The paper further requires that t cover the epochs in which the difficulty is lowered, without a formula, and calls the cases of non-integer and negative y “similar” without stating them. The conditions above are necessary, not sufficient: the general feasibility predicate that item 3(c) of Construction 5.8 requires is not given by the paper.

Assumption 5.11 (Transition rewriting). In the model of Definition 4.1, for every adversary $\mathcal{A}$ that with non-negligible probability p outputs a chain C accepted by the node checks on every sample, some of whose difficulty transitions violate Definition 4.5, there is an adversary $\mathcal{A}'$ that makes the same number of oracle queries, respects the retarget rule, and with probability at least $p - \text{negl}(\kappa)$ outputs a chain C' whose validly mined blocks occupy the same positions as those of C .

The assumption is the statement of ePrint 2019/226, Lemma 4, taken as a hypothesis because its proof is a sketch. It handles subtrees whose aggregates are inconsistent with their children, which Lemma 3.21(2) does not cover. The paper’s proof strategy is as follows. Let x' be the highest node on a claimed path at which the node checks fail. Every path through x' , or carrying it as a sibling, rejects, so barring a collision every leaf of the subtree M' under the parent of x' is unsampleable without rejection. The adversary $\mathcal{A}'$ replaces the transitions inside M' by clamped ones consistent with the aggregates $(w, t, D_{\text{start}}, D_{\text{next}}, n)$ of the root of M' , which is possible because that node passes the feasibility check and the leaves of M' , being invalid, need no proof of work; this is repeated until no invalid transition remains. The paper writes “consistency with x_0 ” where the parent of x' is meant.

Remark 5.12 (Gaps in the rewriting argument). (1) The feasibility step presumes a complete feasibility predicate, which the paper gives only in the simplified setting of Proposition 5.10. (2) Rewriting the leaves of M' changes every later root and hence every later header, so $\mathcal{A}'$ must fix the rewrite before it mines the later valid blocks, and must mine them anew; the paper asserts without a simulation argument that this succeeds with probability at least $p - \text{negl}(\kappa)$. (3) “The same valid blocks” holds for positions, not for header contents. Assumption 5.11 is therefore the paper’s claim, not a result of this volume.

5.4 The sampling distribution

Uniform sampling fails on forks near the tip: almost every uniform sample lands before a late fork point, where the two chains agree. If the fork point were known, each sample after it would be invalid with probability at least $1 - c$ under Assumption 4.8.

Construction 5.13 (Dyadic-suffix sampling). At constant difficulty, over a claimed chain of n blocks and with u samples per interval, the verifier draws, for every integer i with $0 \leq i < \log_2 n$, u samples uniformly and independently from the last $n/2^i$ blocks, and checks every one of the last $\min(L, u)$ blocks.

The samples do not depend on the prover's answers, so the protocol is one-shot and public-coin (*Crypto Guide*, §“Interactive protocols, proofs, and arguments”). The construction is ePrint 2019/226, Section 5.3, with u in place of the paper's k .

Lemma 5.14 (Dyadic miss probability). *Under Assumption 4.8, at constant difficulty, for every fork point a and every fork from a of more than L blocks whose total work after a is at least that of the honest chain, the probability that the $u \log_2 n$ samples of Construction 5.13 include no invalid block of the fork is at most $((1+c)/2)^u$, except with probability negligible in λ .*

Proof. Let the fork point be a and choose i with $(2^i - 1)n/2^i \leq a < (2^{i+1} - 1)n/2^{i+1}$. The interval of the last $n' = n/2^i$ blocks then contains a in its first half, so the fork length $l = n - a$ satisfies $l > n'/2$, and by Assumption 4.8, at constant difficulty, the interval holds at least $(1-c)l > (1-c)n'/2$ invalid blocks. Each of its u independent uniform samples misses them with probability at most $1 - (1-c)/2 = (1+c)/2$, and the samples of the other intervals are discarded. A script checks the bound for $c \in \{0.25, 0.5, 0.9\}$, $u \in \{1, 5, 20\}$ and intervals of 8, 64 and 1,024 blocks. $\square$

The lemma is ePrint 2019/226, Lemma 2. A fork of at most $\min(L, u)$ blocks lies in the checked suffix. When $u < L$, a fork of l blocks with $u < l \leq L$ is covered by neither clause, since Assumption 4.8 does not constrain it; the construction is sound for $u \geq L$, or with the last L blocks checked in full.

Definition 5.15 (Continuum model and catch probability). The claimed chain is the interval $[0, 1]$ in relative cumulative work: a block whose cumulative-work interval is $[W, W + w)$ within total committed work ω occupies $[W/\omega, (W + w)/\omega)$; genesis is at 0 and the head at 1. A *fork point* is a point $a \in [0, 1]$. By Assumption 4.8 in its operative form, the adversary designates an *invalid set* $S \subseteq [a, 1]$ of Lebesgue measure at least $(1-c)(1-a)$; every such S is *admissible*. A *sampling density* is a probability density φ on $[0, 1]$ (*Math Guide*, §“Beyond finite spaces: countable additivity, limits, and densities”). When a suffix $[1 - \delta, 1]$ is checked in full, a set S that meets it in positive measure is caught with probability 1, and any other S is caught by one draw with probability $\int_S \varphi$. The *single-draw catch probability* at a is the infimum $p_\varphi(a)$ of these probabilities over admissible S . For Q independent draws (*Math Guide*, §“Conditional probability and independence”) against a set S fixed before the draws, all miss with probability $(1 - \int_S \varphi)^Q \leq (1 - p_\varphi(a))^Q$. The *design problem* is to choose φ to maximise $\min_a p_\varphi(a)$.

Lemma 5.16 (Best response). *Let φ be non-decreasing on an interval $[a, b]$ with $b \leq 1$, and let $0 \leq m \leq b - a$. Among measurable $S \subseteq [a, b]$ of measure m , the integral $\int_S \varphi$ is minimal for $S = [a, a + m]$. Hence against a density non-decreasing on $[a, 1 - \delta]$, with $[1 - \delta, 1]$ checked in full, an adversary forking at $a \leq 1 - \delta/c$ places its valid weight last, and its invalid set is $[a, 1 - (1 - a)c]$.*

Proof. With $I = [a, a + m]$, $\int_S \varphi - \int_I \varphi = \int_{S \setminus I} \varphi - \int_{I \setminus S} \varphi$; the two sets have equal measure, $S \setminus I$ lies right of $a + m$ and $I \setminus S$ left of it, and φ is non-decreasing, so the difference is non-negative. A set meeting the suffix in positive measure is caught with certainty, so the adversary's best set lies

in $[a, 1 - \delta]$ and has the least admissible measure $(1 - c)(1 - a)$; it ends at $a + (1 - c)(1 - a) = 1 - (1 - a)c \leq 1 - \delta$. $\square$

Construction 5.17 (Indifference density). Equal catch probability at every fork point, that is $\Phi(1 - c(1 - a)) - \Phi(a)$ constant in a for an antiderivative Φ of φ , gives on differentiation

$$\varphi(a) = c\varphi(1 - c(1 - a)).$$

The function $\varphi(\xi) = K/(1 - \xi)$ solves it. With $K = (1 - c)/c$ every invalid segment $[a, 1 - c(1 - a)]$ has mass $(c - 1) \log(c)/c$, which is $\log 2 \approx 0.693147$ at $c = 1/2$. The equation fixes φ only up to a factor periodic in $\log(1 - \xi)$ with period $\log(1/c)$. The function is not integrable on $[0, 1]$, its mass on $[0, 1 - \epsilon]$ being $K \log(1/\epsilon)$, so it is not a density there.

The segment mass is $K(\log(1 - a) - \log(c(1 - a))) = -K \log c$; a script checks it for $c \in \{0.3, 0.5, 0.9\}$ and $a \in \{0, 0.2, 0.7\}$, and the divergence. The construction is ePrint 2019/226, Section 5.4, p. 16.

Construction 5.18 (FlyClient sampling density). Fix $0 < c < 1$ and $\delta \in (0, c]$; the suffix $[1 - \delta, 1]$ is checked in full. The *FlyClient sampling density* is

$$g(\xi) = \begin{cases} \frac{1}{(\xi - 1) \log \delta} & 0 \leq \xi \leq 1 - \delta, \\ 0 & \text{otherwise,} \end{cases} \quad G(\xi) = \frac{\log(1 - \xi)}{\log \delta} \quad (0 \leq \xi \leq 1 - \delta).$$

Both factors of the denominator are negative, so $g > 0$ on its support, and g is increasing there; G is its distribution function, with $G(1 - \delta) = 1$. For $a \leq (c - \delta)/c$ the invalid segment $[a, 1 - (1 - a)c]$ lies in $[0, 1 - \delta]$ and has catch probability

$$G(1 - (1 - a)c) - G(a) = \frac{\log c}{\log \delta},$$

which is $1/k$ at $\delta = c^k$. For $a > (c - \delta)/c$ every admissible invalid set meets the checked suffix in positive measure, and the fork is caught with probability 1.

Normalisation, G and the catch probability are direct integrations. For the last clause, $[a, 1 - \delta]$ has measure $1 - \delta - a < (1 - c)(1 - a)$ exactly when $a > 1 - \delta/c = (c - \delta)/c$. A script checks normalisation, G and the catch probability $1/k$ for $c \in \{0.25, 0.3, 0.5, 0.75, 0.9\}$ and $2 \leq k \leq 9$. The density is ePrint 2019/226, Section 5.4, p. 16; Figure 3 shows it.

Definition 5.19 (Sampling map). Let the head's root tuple rt carry committed work $rt.w$, with leaf count n . The *sampling map* sends a uniform $U \in [0, 1]$ to the point $\xi = 1 - \delta^U$, the inverse of G at U , so that ξ has density g ; it sends ξ to the point $x = \xi \cdot rt.w$ of cumulative work, and x to the leaf j whose interval $[W, W + v_j.w)$ contains x . That interval is bound to (rt, n, j) by Lemma 3.21(1) and is checked by the weight-position check of Construction 5.8. The *checked suffix* is the set of blocks whose interval meets $[(1 - \delta) rt.w, rt.w)$, together with the last L blocks and the head.

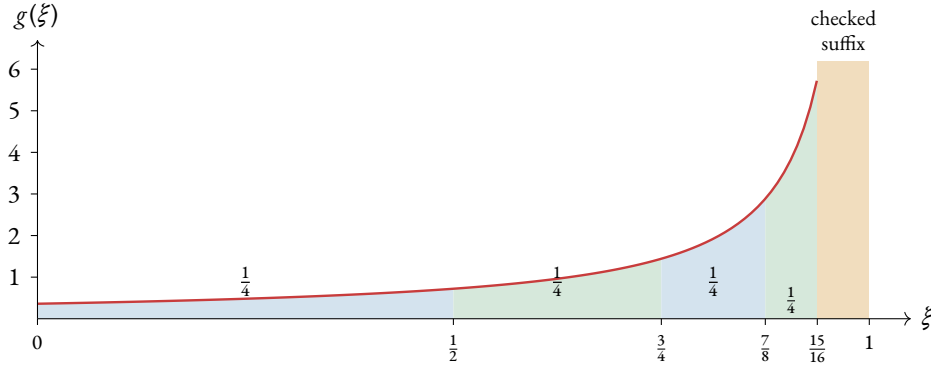


Figure 3. The FlyClient sampling density g of Construction 5.18 for $c = 1/2$ and $k = 4$, so $\delta = 1/16$ and $g(\xi) = 1/((1 - \xi) \log 16)$ on $[0, 15/16]$. The amber strip is the suffix $[15/16, 1]$, checked in full, on which g vanishes. The four shaded segments $[1 - c^i, 1 - c^{i+1}]$, $i = 0, \dots, 3$, are the invalid sets of an adversary forking at $1 - c^i$ with its valid weight last; each has mass $1/4$ under g , and together they tile the support, which is the proof of Theorem 5.21(1).

Assumption 5.20 (Discretisation). For every chain C output by an adversary respecting Definition 4.5, and for the fork point a of C against the honest chain, if C has more than L blocks after a and total work after a at least that of the honest chain, then one draw of Definition 5.19 returns an invalid block, or a block on which the per-sample check fails, with probability at least $p_g(a)$ of Definition 5.15 minus a function negligible in κ ; the Q draws are independent.

The paper asserts that the continuum analysis “still produces a good distribution for the discrete case” (ePrint 2019/226, Section 5.4) and gives no proof; the hypothesis is named here.

Theorem 5.21 (Optimal sampling distribution). *Let $0 < c < 1$, let $k \geq 1$ be real and $\delta = c^k$. Suppose that the suffix $[1 - \delta, 1]$ is checked in full and that the adversary’s valid weight after any fork point a is at most a c fraction. Then:*

1. *if k is an integer, for every sampling density φ , $\min_a p_\varphi(a) \leq 1/k$;*
2. *the density g of Construction 5.18 attains $\min_a p_g(a) = 1/k$, against every placement of the invalid weight.*

Proof. (1) Tiling. At the fork points $a_i = 1 - c^i$, $i = 0, \dots, k - 1$, the placement with valid weight last is available to the adversary against every φ , and leaves the invalid segment $[1 - c^i, 1 - c^{i+1}]$, of measure $c^i(1 - c) = (1 - c)(1 - a_i)$, inside $[0, 1 - \delta]$. These k segments tile $[0, 1 - c^k] = [0, 1 - \delta]$, so $1 \geq \sum_i \int_{[1 - c^i, 1 - c^{i+1}]} \varphi \geq k \min_i p_\varphi(a_i)$. No monotonicity of φ is used. (2) The density g is increasing on $[0, 1 - \delta]$, so for $a \leq (c - \delta)/c$ Lemma 5.16 makes $[a, 1 - (1 - a)c]$ the adversary’s best set, whose catch probability is $1/k$ by Construction 5.18; later fork points meet the suffix and are caught with probability 1. A script checks the k tiling masses. $\square$

The theorem is ePrint 2019/226, Theorem 2, pp. 16–17. The paper prints $c \in (0, 1]$, where $\log \delta = 0$ at $c = 1$ and g is undefined; the domain here is $0 < c < 1$. The paper’s proof sums over

$k + 1$ segments, $i = 0, \dots, k$, the last of which lies in the checked suffix; the range is $i = 0, \dots, k - 1$. The paper’s Lemma 3, a monotonicity lemma, is not needed.

Proposition 5.22 (Sample count). *Let $k > 1$. Then Q independent draws from g all miss an optimally placed invalid set with probability at most $(1 - 1/k)^Q$, and*

$$(1 - 1/k)^Q \leq 2^{-\sigma} \iff Q \geq \frac{\sigma}{\log_2(1/(1 - 1/k))}.$$

At constant difficulty with the last $L = c^k n$ blocks checked, $k = \log_{1/c}(n/L)$ and the least such Q is $\Theta(\sigma k)$, which is $O(\sigma \log_{1/c} n)$; for constant L it satisfies

$$\lim_{n \rightarrow \infty} \frac{Q}{\sigma \log_c(1/2) \log n} = 1.$$

Here σ is the statistical exponent, not the security parameter.

Proof. The miss bound is Definition 5.15 with Theorem 5.21(2); the equivalence follows on taking logarithms. Since $\log_2(1/(1 - 1/k)) = 1/(k \log 2) + O(1/k^2)$, the least Q satisfies $Q \sim \sigma k \log 2 = \sigma \log_c(1/2) \log(n/L)$, and $\log(n/L)/\log n \rightarrow 1$ for constant L . A script cross-checks the least Q against exact rational arithmetic for $2 \leq k \leq 9$ at $\sigma = 50$. $\square$

The proposition is ePrint 2019/226, Section 5.4 (“Optimizing the Proof Size”), p. 17.

Remark 5.23 (Domain of the catch parameter). At $k = 1$, $\delta = c$: one draw suffices in the continuum model, and $\log_2(1/(1 - 1/k))$ is undefined, so Proposition 5.22 is stated for $k > 1$. For $k < 1$, $\delta > c$ and the checked suffix catches every adversary; the paper assumes $k \geq 1$ without loss of generality. For non-integer $k > 1$, Construction 5.18 still gives catch probability $\log c / \log \delta = 1/k$ and hence the sample bound. The tiling of part (1) of Theorem 5.21 needs integral k .

Remark 5.24 (Length of the checked suffix). Under variable difficulty the suffix checked in full is the weight fraction δ , and a sound instantiation makes it contain at least the last L blocks, as Definition 5.19 does (Remark 4.10). The length L is bounded below by the L of the (c, L) assumption in force and trades against Q : a larger L means a larger δ , a smaller k and fewer samples. The asymptotic size bound holds for constant L ; within it, L may be chosen numerically. Which (c, L) holds for Zcash is classified in “The deployment boundary” (§7).

Example 5.25 (Sample counts). At $c = 1/2$ and $\sigma = 50$, with $k = \log_2(n/L)$, the least Q with $(1 - 1/k)^Q \leq 2^{-50}$ is as follows.

n	L	k	bound	Q
2^{20}	100	13.36	445.3	446
2^{22}	100	15.36	514.7	515
2^{22}	1,000	12.03	399.5	400
2^{24}	100	17.36	584.0	585
2^{26}	100	19.36	653.4	654

Every k here is non-integer, so Remark 5.23 applies: the counts are sufficient, and their optimality is not claimed.

5.5 The interactive protocol

Construction 5.26 (Interactive FlyClient). The parties are a verifier $\mathcal{V}$ in the setting of Definition 4.16, which knows the genesis block B_0 , and provers $\mathcal{P}_1, \dots, \mathcal{P}_m$, at least one of them honest. The parameters are c, L , a real $k > 1$, $\delta = c^k$ and Q . The ranges are difficulty MMRs (Definition 5.6) committed by Construction 5.1.

1. Each prover sends its head B_η , whose header records $\text{rt}_n.\text{h}$ over $B_0, \dots, B_{\eta-1}$, together with the two children of the root tuple (for $n = 1$, the single leaf tuple). The verifier recomputes the root tuple rt with par , rejects unless $\text{rt}.\text{h}$ equals the recorded value, and reads $n = \text{rt}.n$ and the claimed total work $\omega = \text{rt}.w + D(T_{\text{head}})$, where T_{head} is the target the head declares.
2. If all heads are identical, $\mathcal{V}$ accepts that head.
3. Otherwise $\mathcal{V}$ orders the claims by claimed total work, heaviest first, and for each prover draws Q independent uniforms, maps each to a point of cumulative work by Definition 5.19, and requests the leaf at each point.
4. For each requested point the prover returns the header, the leaf tuple and its inclusion path with full node tuples. The sample must pass Construction 5.2, whose checks (a) and (d) require a proof of work valid at a target that the difficulty-MMR fields declare, and Construction 5.8 on every node of its path, including the weight-position check that the drawn point lies in the returned leaf's interval.
5. The prover sends the checked suffix of Definition 5.19, and $\mathcal{V}$ checks it in full: every block passes the per-sample and node checks, and consecutive blocks, the head included, satisfy the linkage and target conditions of Definition 4.14, the target of each block being the one that the field D_{next} of its predecessor's leaf declares.
6. Any failure rejects that prover. Among the surviving claims $\mathcal{V}$ accepts the heaviest, outputting its head and claimed total work; claims equivalent under Definition 4.15 count as one claim.

The construction is ePrint 2019/226, Algorithms 1 and 2 with Definition 5 and Sections 5.3, 5.4 and 6.1. Step 5 belongs to the protocol by Sections 5.3 and 5.4 of the paper, although its Algorithm 1 omits it. For claims of different weight the paper delegates the comparison to the generic verifier of Kiayias, Miller and Zindros (“Non-interactive proofs of proof-of-work”), which checks the longest claim first; under variable difficulty the order is by claimed total work, as in step 3.

5.6 The non-interactive protocol

Construction 5.27 (Non-interactive FlyClient). Construction 5.26 is public-coin and one-round: the only message of $\mathcal{V}$ is its sampling coins, which do not depend on the provers' answers. The non-interactive protocol applies the Fiat–Shamir transform (*Crypto Guide*, §“The Fiat–Shamir transform: from interactive to non-interactive”), the challenge hashing the instance. Let $\mathcal{O}$ be a random oracle (*Crypto Guide*, §“The random oracle model”); the uniforms are $U_i = \mathcal{O}(H(\text{hdr}(B_\gamma)), i)$, $i = 1, \dots, Q$, read as points of $[0, 1)$, seeded by the proof-of-work output of the head. Through rt_n the head commits to the whole claimed chain.

1. *Prover*. It computes $U_1, \dots, U_Q$ from its head and the sampled leaves by Definition 5.19, and sends one message: the head with the root's children, for each i the sampled header, leaf tuple and inclusion path with full node tuples, and the checked suffix.
2. *Verifier*. For each proof received it recomputes $U_1, \dots, U_Q$ from the head and runs steps 1 and 4 to 6 of Construction 5.26 on the points they determine.

The paper suggests a standard hash (ePrint 2019/226, Section 6.2) and fixes no domain separation, no map from hash outputs to points and no proof encoding. In the classes of Definition 1.3 the construction is designed but unspecified for Zcash.

Assumption 5.28 (Head-recomputation budget). During one execution the adversary produces at most R heads that carry a valid proof of work.

Assumption 4.8 bounds the validly mined weight of a fork, not the number of head recomputations, and does not imply Assumption 5.28. The paper's budget $R = cn$ (“our security assumption gives a concrete bound on the number of PoW puzzles the adversary can solve, which is cn ”, ePrint 2019/226, Section 6.2) is asserted, not derived.

Proposition 5.29 (Grinding bound). Assume Assumptions 5.28, 5.20, 5.11 and 4.8, let $\mathcal{O}$ be a random oracle, let H be the random oracle of Definition 4.1, with κ -bit outputs, and let $k > 1$. Let at most $q_{\mathcal{O}}$ queries to $\mathcal{O}$ and q_H queries to H be made. The probability that an adversary makes Construction 5.27 accept a claimed chain whose fork is not caught is at most

$$R(1 - 1/k)^Q + q_{\mathcal{O}} q_H 2^{-\kappa} + \text{Adv}_H^{\text{col}}(\mathcal{B}) + \text{negl}(\kappa)$$

for an explicit reduction $\mathcal{B}$. Hence

$$Q > \frac{\sigma + \log_2 R}{\log_2(1/(1 - 1/k))}$$

suffices for failure probability below $2^{-\sigma}$ plus negligible terms. This Q is $\Theta((\sigma + \log R)k)$ and, at constant difficulty with constant L , $\Theta((\sigma + \log R) \log_{1/c} n)$; it exceeds the interactive count of Proposition 5.22 by the $\log_2 R$ bits of grinding.

Proof. Each head with a valid proof of work fixes, through rt_n , the claimed chain and its invalid set before its challenge $\mathcal{O}(H(\text{hdr}(B_\gamma)), \cdot)$ is known, barring a collision of H (Proposition 5.3). The challenge of a header cannot be computed before $H(\text{hdr})$ is queried, and that same query decides

validity. Since $\mathcal{O}$ is independent of H , the challenges of the at most R valid heads are uniform and independent of which heads are valid, except when $\mathcal{O}$ was queried at a value before it arose as an H -output, which has probability at most $q_{\mathcal{O}} q_H 2^{-\kappa}$. So by Assumption 5.20 and Theorem 5.21 the Q samples of each valid head all miss with probability at most $(1 - 1/k)^Q$ plus a negligible term. A head without a valid proof of work fails step 5 of Construction 5.26. By Assumption 5.28 at most R heads qualify, and the union bound (*Math Guide*, §“The union bound and a birthday calculation”) gives the bound. The threshold follows on taking logarithms, and its order as in the proof of Proposition 5.22. $\square$

The bound is proved here because the *Crypto Guide* proves Fiat–Shamir security only for Sigma protocols (§“The Fiat–Shamir transform: from interactive to non-interactive”); the argument above uses only that the challenge is a random-oracle image of the proof-of-work output of a header committing to the whole instance. Seeded by the header itself, the challenge could be evaluated before the proof of work is attempted, and an adversary could mine only heads whose samples miss.

Example 5.30 (Grinding cost). At $c = 1/2$, $\sigma = 50$, $L = 100$ and $n = 2^{22}$, $k = \log_2(n/L) \approx 15.36$. Proposition 5.22 gives $Q \geq 514.7$, so 515 samples. With the paper’s $R = cn = 2^{21}$ the budget adds 21 bits, and Proposition 5.29 gives $Q > 730.8$, so 731 samples. At the other (n, L) of Example 5.25, namely $(2^{20}, 100)$, $(2^{22}, 1,000)$, $(2^{24}, 100)$ and $(2^{26}, 100)$, the non-interactive counts are 615, 568, 853 and 981.

Proposition 5.31 (Transferability). *A proof of Construction 5.27 is publicly verifiable: its verification uses only the proof, the genesis block and the public oracles, so any party can relay it unchanged, and it convinces every verifier that the original convinces. For a given head the sampled points are deterministic; under a canonical proof encoding there is therefore one proof per head, which one party can compute for every verifier.*

Proof. The verifier’s decision is a deterministic function of the proof and public data, and the points are a function of the head. $\square$

The paper’s statement that “there only exists a single valid non-interactive proof” (ePrint 2019/226, Section 8.1) holds only under a canonical encoding, which no ZIP specifies. ZIP 221, “Security and Privacy Considerations” records that, because FlyClient proofs are non-interactive and publicly verifiable, “they could be shared among many light clients after the initial server interaction”.

5.7 The security theorem

Theorem 5.32 (FlyClient security and succinctness). *Assume:*

1. $0 < c < 1$, a real $k > 1$ and $\delta = c^k$;
2. the variable-difficulty backbone model (Definition 4.1), in which every adversary is a (c, L) -adversary (Assumption 4.8);

3. H collision resistant and, as in Definition 4.1, a random oracle, and the sampling oracle $\mathcal{O}$ a random oracle;
4. Assumptions 5.28, 5.20, 5.11 and 4.4;
5. a verifier that knows only the genesis block, and at least one honest prover (Definition 4.16);
6. no block header equals $\text{ser}(\ell) \parallel \text{ser}(r)$ for node tuples ℓ, r (leaf-internal separation, hypothesis 2 of Lemma 5.4).

Then an adversary wins the game of Definition 4.17 against Construction 5.27 with probability at most $R(1 - 1/k)^Q$ plus negligible terms, which is at most $2^{-\sigma}$ plus negligible terms when Q satisfies Proposition 5.29. Proofs have

$$O(L_\delta + (\sigma + \log R) k \log n)$$

headers and hashes, where L_δ is the number of blocks of the checked suffix. The construction is succinct (Definition 4.18) whenever L_δ, k and $\log R$ are $O(\text{polylog}(n))$, in particular at constant difficulty with constant L and $\delta = L/n$, where $L_\delta = L$ and $k = \log_{1/c}(n/L)$; there, with the paper's $R = cn$, the size is $O(L + (\sigma + \log n) \log_{1/c}(n) \log n)$.

Proof structure. The proof composes the obligations O1 to O6 of Remark 5.33, following the five steps of ePrint 2019/226, Section 6.3. (0) The honest prover's proof passes every check: its labelling is consistent, its transitions are clamped, and Lemma 3.21(2) makes each weight interval its true prefix weight. (1) Assumption 5.11 replaces an adversary with invalid transitions by one that respects the retarget rule, to which Assumption 4.8 applies (O3). (2) O1 pins every sampled answer to one committed chain; O2 makes every block after the fork point adversarial, or places a leaf of the fork in its invalid set, and makes its validly mined weight post-fork; O3 binds each sampled leaf's weight interval to the root, while the agreement of that interval with the chain's true prefix weight rests on the node checks and Assumption 5.11. (3) O4: a fork heavier than the honest chain and longer than L blocks is caught by one draw with probability at least $1/k$, and a fork within the checked suffix with certainty. (4) O5: the Fiat-Shamir step with the grinding bound. (5) O6: the size count. Agreement of honest full nodes on the prefix up to the last ℓ blocks, the second clause of Definition 4.17, is Assumption 4.4. $\square$

The theorem is ePrint 2019/226, Theorem 1 (p. 10, restated p. 20); ZIP 221, "Security and Privacy Considerations" restates its honest-connection and genesis hypotheses. The paper states the size as $O(L + \lambda \log_{1/c}(n) \log n)$. That equals the constant-difficulty bound above with $R = cn$ only when $\log n = O(\lambda)$, for instance for n polynomial in λ ; under the convention that n grows with σ fixed, the theorem states the $\log R$ form and no stronger bound.

Remark 5.33 (Proof obligations). Each obligation is listed with the results that discharge it.

O1 Binding: Theorem 3.17, Corollary 3.18, Lemma 3.20 and Proposition 5.3.

O2 Fork structure: Lemmas 5.4 and 5.5.

- O3 Weight binding and transitions: Lemma 5.7, Construction 5.8 and, for subtrees whose aggregates are inconsistent, Assumption 5.11.
- O4 Catch probability and sample count: Theorem 5.21, Definition 5.19, Assumption 5.20 and Proposition 5.22.
- O5 Grinding: Proposition 5.29, which fixes the non-interactive Q .
- O6 Size: Corollary 5.34.

Corollary 5.34 (Proof size). *For the interactive protocol with Q from Proposition 5.22, constant L and constant difficulty, a proof has size $O(\sigma \log n \log_{1/c} n + L)$: $Q = \Theta(\sigma \log_{1/c} n)$ samples, each a header and a path of at most $\lceil \log_2 n \rceil$ nodes under Bag_R (Lemma 3.7), and the L suffix blocks. For the non-interactive protocol, with Q from Proposition 5.29, σ is replaced by $\sigma + \log_2 R$.*

Proof. Multiply Q by the cost of one sample. The suffix blocks form a contiguous run of leaves ending at leaf n , so the union of their paths has $O(L + \log n)$ nodes and is sent once. $\square$

The corollary is ePrint 2019/226, Corollary 2 (Section 5.4), which states Θ ; the matching lower bound is not proved here.

Remark 5.35 (Status of the security theorem). Theorem 5.32 is the paper’s conditional claim, not an independently completed proof. Deriving the (c, L) assumption from the backbone parameters is open (Remark 4.13); the paper’s Lemma 1 is invalid as printed (Remark 4.11); its Lemma 4 is a sketch with recorded gaps and enters as Assumption 5.11; the step from the continuum to blocks is Assumption 5.20; and the budget $R = cn$ is asserted. Proved in this volume are binding (O1), fork structure (O2) under leaf–internal separation, the binding of sampled weight intervals to the root, optimality, the sample count and the grinding bound.

5.8 Verification after acceptance

Proposition 5.36 (Inclusion after acceptance). *Let a head B_η with root tuple rt_η be accepted. Membership of the block at height $j - 1$ in the accepted chain is established by one inclusion path of leaf j against rt_η , and membership of a transaction in that block by one further Merkle path against the transaction root its header carries; no intermediate header is needed. Both are binding barring a collision.*

Proof. For the block, Proposition 5.3(i); for the transaction path, *Crypto Guide*, §“Soundness: forging a path implies a collision”. $\square$

The proposition is ePrint 2019/226, Sections 1 and 4.2.

Proposition 5.37 (Verification against a fresh head). *A client may run the protocol against the freshest heads and store one recent block B^* of its previously accepted chain lying more than 2ℓ blocks below that chain’s head. Let the height of B^* be less than that of the newly accepted head minus ℓ . Under the hypotheses of Theorem 5.32, except with its soundness error, the accepted head’s*

committed range contains B^ at its height, which the client checks by one inclusion path (Proposition 5.36); a head is rejected if a sample exposes an invalid block.*

Proof. By Theorem 5.32 both accepted chains are equivalent up to their last ℓ blocks to chains of honest full nodes. A head B_η commits to heights 0 to $\eta - 1$, so clause (b) of Definition 4.17 at the earlier round places B^* in every honest chain of that round with its last ℓ blocks removed, and Assumption 4.4 places it in every later honest chain. The height of B^* is below that of the new head minus ℓ , so B^* lies in the part of the new committed chain that clause (b) makes a prefix of an honest chain. $\square$

The paper states without qualification that the range of a fresh head “cannot contain invalid blocks and it must contain all stable blocks” (ePrint 2019/226, Section 4.2, “Unstable Blocks”); that sentence is not carried.

Proposition 5.38 (Subchain proofs, the paper’s claim). *Assume the hypotheses of Theorem 5.32. Suppose a verifier accepted a valid proof for a chain with head B_η when the honest chain had that head, and later, when the honest chain has head $B_{\eta'}$, $\eta' > \eta$, receives a subproof for the blocks from B_η to $B_{\eta'}$, sampled as an instance of the protocol with B_η in place of genesis, together with one inclusion path showing that B_η lies in the range committed by $B_{\eta'}$. Then, except with the soundness error of Theorem 5.32 for the instance from B_η , it accepts no chain that a verifier given a full proof for height η' would reject. The instance from B_η , which samples only leaves after B_η of a range committed from B_0 , is the paper’s design (ePrint 2019/226, Section 8.2) and is not constructed here.*

Proof idea. This is ePrint 2019/226, Theorem 3 (Section 8.2), with the paper’s proof idea. A fork after B_η faces a fresh instance of the protocol with B_η as genesis. A fork before B_η fails the inclusion path, since the verifier’s B_η is not in the adversary’s range, barring a collision (Proposition 5.36). $\square$

The proposition concerns one commitment range from the genesis block.

6 The chain-history commitment

This section states the commitment that Zcash block headers carry, each rule cited to the ZIP or the protocol specification section that fixes it, and each object before its first use: the upgrade epochs over which it ranges, the node vector of its Merkle mountain range, the hashing of the tree and its root commitment, the authorising-data root and the block commitment hash of ZIP 244 that wraps the root from NU5, the header consensus rules, and the range of blocks to which each header commits.

6.1 The deployed chain commitment

ZIP 221 (Final), “FlyClient - Consensus-Layer Changes”, changes the semantics of the block header and the consensus rules from Heartwood. The 32-byte header field that held the root of the

Sapling note commitment tree instead commits to the root of a Merkle mountain range (Definition 3.1; ZIP 221, “Terminology”: a binary hash tree that admits appends of new leaves “without changing the value of existing nodes”) whose nodes carry aggregated metadata. Its leaves are the blocks from the last activation height below the committing block up to the parent of that block: the *epoch tree* of Definition 6.2. ZIP 221 thus deploys the invariant of Construction 5.1 restricted to one *upgrade epoch*, the heights from one activation height up to the next (Definition 6.1), and not from the genesis block. The tree and its node fields 1 to 14 are ZIP 221’s; fields 15 to 17 are ZIP 258’s (Draft, in force from NU6.3 by Definition 1.2); and from NU5 the root is committed as the first input of the *block commitment hash*, a personalised hash of the root together with the block’s authorising-data root (ZIP 244; “The block commitment hash”, §6.6). Seven node fields serve reasoning about difficulty without matching the node of Definition 5.6 (ePrint 2019/226, Definition 7) field for field (Remark 6.4). Each term of this paragraph carries here only its defining clause; its definition is in the subsection named.

The purpose of the commitment, after ZIP 221, “Motivation”, is to let a light client verify, with a number of headers logarithmic rather than linear in the chain length and correctly with high probability, the validity of a chain received from a full node, the inclusion of a block in it, and metadata of any block or range of blocks; the commitment also admits subtree proofs, which check only blocks later than the most recently verified one. Transaction inclusion within a block is left to ZIP 307 (Draft). No claim of this paragraph is proved in this section; the protocol that would realise it is that of “The FlyClient protocol” (§5), and its relation to the deployed commitment is classified in “The deployment boundary” (§7).

6.2 Upgrade epochs and epoch trees

Definition 6.1 (Upgrade epoch). Let $0 = A_0 < A_1 < A_2 < \dots$ be the distinct activation heights of the network upgrades of a chain (ZIP 200, “Activation mechanism”), height 0 counted as an activation height (ZIP 221, “Specification”). The named upgrades and their order are those of Definition 1.1; the list of activation heights includes those of upgrades that definition does not name. The *upgrade epoch* beginning at A_i is the interval of heights $[A_i, A_{i+1})$, unbounded for the last activation height. For $i \geq 1$ its blocks are validated under one consensus rule set and one *consensus branch ID*, the 32-bit identifier of the last upgrade that activates at A_i (ZIP 200, “Specification”; *Ironwood Guide*, §“The transaction format”). The upgrade epoch is distinct, by name and by object, from the retarget epoch of Definition 4.5; the word “epoch” is not used alone. The Heartwood activation height is written A_H , and B_{A_H} is the Heartwood activation block.

Definition 6.2 (Epoch tree). For a block height $\eta > 0$, the *preceding activation height* $\alpha(\eta)$ is the largest activation height $A_i < \eta$, the “preceding network upgrade height” of ZIP 221, “Specification”; $\alpha(0)$ is undefined. For $\eta > A_H$ the *epoch tree* $\mathcal{T}_\eta$ is the Merkle mountain range $\mathcal{M}_n^L$ under the left fold Bag_L (Definition 3.5), bagging nodes included as internal nodes, with the

$$n = \eta - \alpha(\eta) \geq 1$$

leaves $B_{\alpha(\eta)}, \dots, B_{\eta-1}$ in height order: leaf j is $B_{\alpha(\eta)+j-1}$. Since $\alpha(\eta)$ is the last activation height

below η , all its leaves lie in the one upgrade epoch beginning at $\alpha(\eta)$. Its node contents are fixed in “The node vector” (§6.3) and its hashing in “The epoch tree and its root commitment” (§6.4). No epoch tree is defined for $\eta \leq A_H$.

Which header commits to $\mathcal{T}_\eta$ is not part of Definition 6.2: it is Definition 6.22 in “The header consensus rules” (§6.7), and its consequences for the committed ranges are stated in “The committed range” (§6.8).

6.3 The node vector

Definition 6.3 (Node vector). The epoch tree $\mathcal{T}_\eta$ is an instance of Definition 3.19 under Bag_L . Every node v of it, leaf, internal node, bagging node or root, carries one *node vector* of fields, serialised canonically by the map ser of Definition 6.5; that serialisation is used whenever a parent commitment is formed. The fields are numbered as in ZIP 221, “Tree Node specification”. For a block $B_{\eta'}$ with header hdr' , the *leaf map* sets both members of each earliest/latest pair from $B_{\eta'}$ itself:

- (1) field `hashSubtreeCommitment` is $\text{BlockHash}(\text{hdr}')$ (Definition 2.3), in internal byte order and unpersonalised;
- (2), (3) fields `nEarliestTimestamp` and `nLatestTimestamp` are the field `nTime` of hdr' ;
- (4), (5) fields `nEarliestTargetBits` and `nLatestTargetBits` are the field `nBits` of hdr' (the semantics of `nTime` and `nBits`: *Consensus Guide*, §“Difficulty in motion”);
- (6), (7) fields `hashEarliestSaplingRoot` and `hashLatestSaplingRoot` are $\text{LEBS2OSP}_{256}(\text{rt})$ for rt the root of the Sapling note commitment tree of the final Sapling treestate of $B_{\eta'}$ (protocol specification, §“Block Header Encoding and Consensus”), an opaque 32-byte value in this volume;

- (8) field `nSubTreeTotalWork` is the work

$$\lfloor 2^{256} / (\text{ToTarget}(\text{nBits}) + 1) \rfloor$$

of $B_{\eta'}$ (*Consensus Guide*, §“Work, and the best chain”, Definition “Work of a block”);

- (9), (10) fields `nEarliestHeight` and `nLatestHeight` are the height η' ;
- (11) field `nSaplingTxCount` is the number of transactions of $B_{\eta'}$ in which `vSpendsSapling` or `vOutputsSapling` is non-empty;
- (12), (13) fields `hashEarliestOrchardRoot` and `hashLatestOrchardRoot` are the Orchard-pool anchor of $B_{\eta'}$, the root of the Orchard pool’s note commitment tree in the final treestate of $B_{\eta'}$ (*Ironwood Guide*, §“Anchors”, Definition “Treestate and anchor”);
- (14) field `nOrchardTxCount` is the number of transactions of $B_{\eta'}$ in which `vActionsOrchard` is non-empty (*Ironwood Guide*, §“The transaction format”);

Table 2. The fields of the node vector of Definition 6.3: serialised type, value at the leaf for the block $B_{\eta'}$ with header hdr' , rule at an internal node (“hash” is Construction 6.10), and source: ZIP 221, “Tree Node specification”, for fields 1 to 14, and ZIP 258, “Changes to ZIP 221”, for fields 15 to 17.

	Field	Type	Leaf value for $B_{\eta'}$	Internal node	ZIP
1	hashSubtreeCommitment	char[32]	BlockHash(hdr')	hash	221
2	nEarliestTimestamp	uint32	nTime	inherit left	221
3	nLatestTimestamp	uint32	nTime	inherit right	221
4	nEarliestTargetBits	uint32	nBits	inherit left	221
5	nLatestTargetBits	uint32	nBits	inherit right	221
6	hashEarliestSaplingRoot	char[32]	final Sapling root	inherit left	221
7	hashLatestSaplingRoot	char[32]	final Sapling root	inherit right	221
8	nSubTreeTotalWork	uint256	work of the block	sum mod 2^{256}	221
9	nEarliestHeight	CompactSize	height η'	inherit left	221
10	nLatestHeight	CompactSize	height η'	inherit right	221
11	nSaplingTxCount	CompactSize	transactions with Sapling spends or outputs	sum	221
12	hashEarliestOrchardRoot	char[32]	Orchard-pool anchor	inherit left	221
13	hashLatestOrchardRoot	char[32]	Orchard-pool anchor	inherit right	221
14	nOrchardTxCount	CompactSize	transactions with Orchard-pool Actions	sum	221
15	hashEarliestIronwoodRoot	char[32]	Ironwood-pool anchor	inherit left	258
16	hashLatestIronwoodRoot	char[32]	Ironwood-pool anchor	inherit right	258
17	nIronwoodTxCount	CompactSize	transactions with Ironwood-pool Actions	sum	258

(15)–(17) fields hashEarliestIronwoodRoot, hashLatestIronwoodRoot and nIronwoodTxCount are the Ironwood-pool anchor of $B_{\eta'}$ and the number of its transactions in which vActionsIronwood is non-empty, computed as fields 12 to 14.

The *parent rule*: an internal node with left child ℓ and right child r takes each earliest field (2, 4, 6, 9, 12, 15) from ℓ and each latest field (3, 5, 7, 10, 13, 16) from r ; it sums the additive fields, field 8 modulo 2^{256} and the per-pool counts 11, 14 and 17 in $\mathbb{Z}$; and it sets field 1 by Construction 6.10, the personalised hash of $\text{ser}(\ell) \parallel \text{ser}(r)$. Fields 12 to 14 are ZIP 221’s own, added at NU5 (ZIP 252). Fields 15 to 17 are ZIP 258’s, “Changes to ZIP 221” (Draft), whose rules apply from NU6.3 activation (Definition 1.2).

For field 8, ZIP 221, “Tree Node specification”, states that computations modulo 2^{256} are fine because cumulative chain work is assumed to be at most 2^{256} , an aggregate effort of 2^{256} or more being infeasible; under that assumption, with the total below 2^{256} , the field of a node is the integer sum of the work of its leaves (Lemma 3.21(2)). Table 2 summarises the definition.

Remark 6.4 (The FlyClient fields). ZIP 221, “FlyClient Requirements and Recommendations”, designates seven fields, 2 to 5 and 8 to 10, which “enable FlyClient in the variable-difficulty model” by letting a light client reason about the application of the difficulty adjustment over a range of blocks. They serve the purpose of Definition 5.6 without matching it field for field. The leaf count of a node, the field n of the difficulty MMR, is $n\text{LatestHeight} - n\text{EarliestHeight} + 1$ (Lemma 6.6). The field w of the difficulty MMR sums difficulty levels (Definition 4.2), whereas $n\text{SubTreeTotalWork}$ sums work, a different normalisation. The field $n\text{EarliestTargetBits}$ is a

compact target, ordered inversely to D_{start} . No field carries the elapsed time t ; the timestamps of the first and last block stand in its place. No field carries D_{next} : the next target of a Zcash block is a function of an averaging window of preceding blocks (*Consensus Guide*, §“Difficulty in motion”), which the aggregates of a node do not determine.

Fields 6, 7 and 11 to 14 are ZIP 221’s “Non-FlyClient Commitments”, chosen “primarily to improve light client security guarantees”, with no counterpart in ePrint 2019/226. Field 1 “forms the structure of the commitment tree” (ZIP 221, “Tree nodes”), and fields 15 to 17 are the ZIP 258 counterparts for the Ironwood pool. The earliest roots let a client check the treestate transitions over a range of blocks without recomputing the roots from genesis. ZIP 221, “Non-FlyClient Commitments”, states that `hashLatestSaplingRoot` serves the purpose of the former header field `hashFinalSaplingRoot`, one block delayed (Proposition 6.23). The per-pool counts follow the rule of the same section that a new shielded pool adds a count of its transactions, applied at NU5 (fields 12 to 14) and at NU6.3 (fields 15 to 17, ZIP 258).

Definition 6.5 (Field vector by upgrade epoch). The fields are encoded as follows: a `char[32]` field as its 32 bytes; a `uint32` or `uint256` field little-endian, as $I2LEOSP_{32}$ or $I2LEOSP_{256}$ (protocol specification, §“Integers, Bit Sequences, and Endianness”); a height or count as its `CompactSize` encoding in minimal form (Definition 2.2); and each pool-root field as $LEBS2OSP_{256}$ of the root, as the specification encodes the Orchard anchor (protocol specification, §“Transaction Encoding and Consensus”). For a node v , $\text{ser}(v)$ is the concatenation of the encodings of its fields in field order. The *field vector* of an upgrade epoch is:

1. fields 1 to 11 in the Heartwood and Canopy upgrade epochs;
2. fields 1 to 14 in the NU5, NU6, NU6.1 and NU6.2 upgrade epochs;
3. fields 1 to 17 in the NU6.3 and NU7 upgrade epochs.

The vectors are named by their fields. ZIP 221, “Tree Node specification”, omits fields 12 to 14 “before NU5 activation”, and ZIP 258, “Changes to ZIP 221”, omits fields 15 to 17 “before NU6.3 activation”; neither says whether the height of a leaf’s block or that of the committing block governs, and the two differ only for the tree committed by an activation block. *Adopted reading*, resolving that ambiguity: every node of $\mathcal{T}_\eta$ uses the field vector of the upgrade epoch beginning at $\alpha(\eta)$, the upgrade epoch of its leaves.

The reading is adopted for the following reason. Under the other reading the tree $\mathcal{T}_A$ committed by B_A , at an activation height A that adds fields, would serialise every node of the previous upgrade epoch with the added fields; every internal node would change value, and $\mathcal{T}_A$ would not be an append of $\mathcal{T}_{A-1}$, contrary to the property by which ZIP 221, “Terminology”, defines a Merkle mountain range. The ZIP’s pseudocode, which takes a leaf’s Orchard fields from the leaf’s own block and serialises them only when present, is consistent with the reading. Two consequences rest on it: $\mathcal{T}_A$ uses the previous upgrade epoch’s vector, and $\mathcal{T}_{A+1}$ is the first tree with the new one; under the other reading $\mathcal{T}_A$ would carry the new vector. For NU7, no ZIP among the sources of NU7 consensus changes listed in ZIP 259 (Draft), “NU7 consensus changes”, amends ZIP 221 or ZIP 244, so the nodes of the trees of the NU7 upgrade epoch carry fields 1 to 17 under either

reading.

The fixed-width fields occupy 144, 208 and 272 bytes in the three vectors, and the three, four and five CompactSize fields 1 to 9 bytes each, so a serialised node has 147 to 171, 212 to 244 and 277 to 317 bytes, the ranges that ZIP 221 and ZIP 258 state. A toy chain computed by script illustrates the reading, with an activation block that commits a tree of the previous vector and the next block the first to commit a tree of the new one; it establishes nothing.

Lemma 6.6 (Leaf count and shape). *Every node v of $\mathcal{T}_\eta$, bagging nodes included, spans a run of consecutive leaves $B_{\eta_1}, \dots, B_{\eta_2}$ with $v.\text{nEarliestHeight} = \eta_1$ and $v.\text{nLatestHeight} = \eta_2$, so its leaf count is $\text{nLatestHeight} - \text{nEarliestHeight} + 1$. The node v roots a perfect subtree of $\mathcal{T}_\eta$ if and only if that count is a power of two. In particular a leaf is the only kind of node with count 1.*

Proof. By induction on the parent rule of Definition 6.3. A leaf has both heights equal to its block's. An internal node inherits nEarliestHeight from its left child and nLatestHeight from its right child, and in a range under Bag_L the run of the left child immediately precedes that of the right child, the heights of the leaves of $\mathcal{T}_\eta$ being consecutive by Definition 6.2. A perfect subtree of altitude a spans 2^a leaves (Definition 3.1), and every node inside a peak roots a perfect subtree. A bagging node of the left fold spans the peaks $P_1, \dots, P_i$ for some $i \geq 2$ (Definition 3.5), whose sizes are distinct powers of two, set bits of n (Definition 3.4); a sum of two or more distinct powers of two has at least two set bits and is not a power of two. An internal node spans at least two leaves. A script checks the statement for $1 \leq n \leq 1,024$. $\square$

Lemma 6.7 (Leaves and header linkage). *For $\alpha(\eta) \leq \eta' \leq \eta - 1$, field 1 of the leaf of $\mathcal{T}_\eta$ for $B_{\eta'}$ equals the field `hashPrevBlock` of the header of $B_{\eta'+1}$, which for $\eta' = \eta - 1$ is the committing block B_η . Consequently a received header is matched to its leaf by computing its block hash and comparing it with field 1, and the leaves add no hashing beyond the block hashes.*

Proof. By Definition 6.3, field 1 of the leaf is `BlockHash` of the header of $B_{\eta'}$ in internal byte order; by Definition 2.3, the field `hashPrevBlock` of $B_{\eta'+1}$ is that same hash. ZIP 221, “Tree Node specification”, states the identity. $\square$

Remark 6.8 (Non-monotone timestamps). Consensus bounds the field `nTime` of a block below, strictly above the median of the timestamps of the preceding eleven blocks, and above, by that median plus 90 minutes (protocol specification, §“Block Header Encoding and Consensus”; *Consensus Guide*, §“Difficulty in motion”). It orders two timestamps only at a distance: the median-time-past never decreases and rises by at least one second every eleven blocks, so a block at least $11 \cdot 5,400 = 59,400$ heights after another has the strictly later timestamp. By the same count, $\text{nLatestTimestamp} \geq \text{nEarliestTimestamp}$ holds for every node with $\text{nLatestHeight} - \text{nEarliestHeight} \geq 11 \cdot 5,399 = 59,389$, and for a leaf, whose two timestamps are equal; for smaller spans this argument gives no order. ZIP 221, “Tree Node specification”, says that the reverse order “may occur within subtrees smaller than `PoWMedianBlockSpan` blocks”; this is an example, not a bound. A block whose timestamp lies up to 90 minutes ahead of the median exceeds the timestamps of the later blocks whose timestamps track real time, until real time passes it, and 90 minutes spans more than eleven target spacings (protocol specification, §“Constants”; from NU7, ZIP 218).

6.4 The epoch tree and its root commitment

The tree is defined from the prose of ZIP 221, “Background”, “Specification”, “Tree Node specification” and “Block header semantics and consensus rules”, with ZIP 258, “Changes to ZIP 221”. That prose is ambiguous at two points, both at an activation block: the field vector, settled by the adopted reading of Definition 6.5, and the branch ID of the personalisation, settled by Definition 6.9. The pseudocode of ZIP 221 is cited only as corroboration of the two readings; it carries editorial defects, among them in the serialisation of the Orchard fields, none of which bears on either reading.

Definition 6.9 (Branch ID of a tree). ZIP 221, “Tree Node specification”, states that the personalisation encodes “the consensus branch ID for the epoch of the block containing the commitment”, and continues: “Which is to say, each node in the tree commits to the consensus branch that produced it.” The two sentences agree at every $\eta > A_H$ that is not an activation height, since B_η and the leaves of $\mathcal{T}_\eta$ then lie in one upgrade epoch. At an activation height $A > A_H$ the first names the upgrade epoch beginning at A and the second the upgrade epoch of the leaves $B_{\alpha(A)}, \dots, B_{A-1}$. At A_H no epoch tree is defined (Definition 6.2). *Adopted reading*, resolving an ambiguity of specified text: the branch ID $b(\mathcal{T})$ of an epoch tree $\mathcal{T}$ is the consensus branch ID of the upgrade epoch whose blocks are the leaves of $\mathcal{T}$, and every internal node and the root commitment of $\mathcal{T}$ are hashed under it.

The reading is adopted for three reasons. (1) “Which is to say” introduces the second sentence as a restatement of the first, and it is the sentence that assigns a branch ID to each node. (2) Under the first reading the tree committed by B_A would not be an append of the tree committed by B_{A-1} , every internal node changing value, contrary to the property by which ZIP 221, “Terminology”, defines a Merkle mountain range (the reason for which Definition 6.5 adopts the leaves’ vector); and the whole previous upgrade epoch would be rehashed in one block, the cost that ZIP 221, “Header Format Change”, calls impractical and avoids by a special case at the Heartwood activation block and at no later one. (3) The pseudocode of ZIP 221 hashes each parent under the branch ID of its left child and the root commitment under that of the root node, each leaf carrying the branch ID of its own block. Every later statement about the tree committed by an activation block rests on this reading and cites it; under the first reading each holds with the branch ID of the new upgrade epoch in place of $b(\mathcal{T}_A)$.

Construction 6.10 (Hashing of the epoch tree). Let $h(P, x)$ be BLAKE2b-256 with the 16-byte personalisation P on input x (*Ironwood Guide*, §“Transaction digests and signatures”; *Crypto Guide*, §“Domain separation and personalisation”), and let β be the 4-byte little-endian encoding of $b(\mathcal{T}_\eta)$. The personalisation is the 12 ASCII bytes `ZcashHistory` followed by β , 16 bytes in all. The tree $\mathcal{T}_\eta$ is the range M_n^L whose nodes are node vectors (Definition 6.3) under the field vector of Definition 6.5. Its bagging nodes, which join the two leftmost remaining peaks until one remains,

$$(\dots ((P_1 \parallel P_2) \parallel P_3) \dots) \parallel P_d,$$

are ordinary internal nodes, and a single peak is the root node. For an internal node with chil-

Table 3. The branch IDs entering the personalisation $\text{ZcashHistory} \parallel \beta$ of Construction 6.10: one row per upgrade epoch that bears an epoch tree, each tree hashed under its own personalisation, with the ZIP that fixes the branch ID (ZIP 258, “NU6.3 deployment”; ZIP 259, “NU7 network constants”), the little-endian bytes β and the field vector of Definition 6.5.

Upgrade epoch	Branch ID	Fixed by	Bytes 13 to 16 of the personalisation	Fields
Heartwood	0xF5B9230B	ZIP 250	0B 23 B9 F5	1 to 11
Canopy	0xE9FF75A6	ZIP 251	A6 75 FF E9	1 to 11
NU5	0xC2D6D0B4	ZIP 252	B4 D0 D6 C2	1 to 14
NU6	0xC8E71055	ZIP 253	55 10 E7 C8	1 to 14
NU6.1	0x4DEC4DF0	ZIP 255	F0 4D EC 4D	1 to 14
NU6.2	0x5437F330	ZIP 257	30 F3 37 54	1 to 14
NU6.3	0x37A5165B	ZIP 258 (Draft)	5B 16 A5 37	1 to 17
NU7	0x77190AD9	ZIP 259 (Draft)	D9 0A 19 77	1 to 17

dren ℓ and r , field 1 is

$$h(\text{ZcashHistory} \parallel \beta, \text{ser}(\ell) \parallel \text{ser}(r)),$$

over the children’s full serialisations; its other fields follow the parent rule of Definition 6.3. The branch ID is not a node field: it enters only through the personalisation, and whoever recomputes a node hash supplies $b(\mathcal{T}_\eta)$ by Definition 6.9, the upgrade epoch being fixed by the node’s heights. For the Heartwood branch ID 0xF5B9230B the last four bytes of the personalisation are 0B 23 B9 F5.

Definition 6.11 (Chain-history root). For $\eta > A_H$ the *chain-history root* of $\mathcal{T}_\eta$ is

$$\text{hashChainHistoryRoot}(\mathcal{T}_\eta) := h(\text{ZcashHistory} \parallel \beta, \text{ser}(\text{root node of } \mathcal{T}_\eta)),$$

with β encoding $b(\mathcal{T}_\eta)$ (ZIP 221, “Tree Node specification”). It is not field 1 of the root node: the committed value covers the root’s aggregates, namely the total work modulo 2^{256} , the first and last timestamps, target bits and heights, the boundary pool roots and the per-pool counts. When $\alpha(\eta) = \eta - 1$ the tree has one leaf, which is its root node, and the value is defined there too. ZIP 258, “Changes to ZIP 221”, leaves the definition unchanged apart from the added fields.

Definition 6.12 (History input of a block). For $\eta \geq A_H$ the *history input* of B_η is

$$\text{hashLightClientRoot}(\eta) := \begin{cases} \text{the string of 32 zero bytes} & \text{if } \eta = A_H, \\ \text{hashChainHistoryRoot}(\mathcal{T}_\eta) & \text{if } \eta > A_H. \end{cases}$$

The value at A_H is the sentinel that ZIP 221, “Block header semantics and consensus rules”, requires in the Heartwood activation block and states “MUST NOT be interpreted as a root hash”. The history input is a total function of the height from A_H on, and it is the value that ZIP 244, “Block Header Changes”, names “hashLightClientRoot (as described in ZIP 221)”.

Proposition 6.13 (Binding of the chain-history root). *Let $\mathcal{T}$ and $\mathcal{T}'$ be epoch trees hashed under one branch ID b . If $\text{hashChainHistoryRoot}(\mathcal{T}) = \text{hashChainHistoryRoot}(\mathcal{T}')$, then either $\mathcal{T}$ and $\mathcal{T}'$ agree on every field of every node, or a collision of $h(\text{zcashHistory} \parallel \beta, \cdot)$ is computed from them efficiently. If in addition SHA-256d is collision resistant, the leaves' block hashes bind the headers of the leaf blocks.*

Proof. The proposition is the ZIP 221 instance of Lemma 3.20(1), whose hypotheses are checked as follows. *Serialisation.* The branch ID b fixes one upgrade epoch and hence one field vector for both trees (Table 3; Definitions 6.9 and 6.5). Its field order is fixed, and each field is of fixed width or a minimal CompactSize, whose first byte fixes its length (Definition 2.2). A serialised node therefore parses field by field: *ser* is injective, and no serialised node is a proper prefix of another, since the two parse to the same fields up to the end of the shorter and the number of fields is fixed; minimality makes *ser* a function of the node. *Root and shape.* If the root commitments are equal, then the two root serialisations are equal, or they are distinct inputs with equal images, a collision. Equal root nodes have equal *nEarliestHeight* and *nLatestHeight*, hence equal leaf count n by Lemma 6.6, hence the same shape $\mathcal{M}_n^L$. The shape fixes which positions are leaves, so no leaf of one tree is compared with an internal node of the other, although the tree carries no layer tags (ZIP 221, “Tree nodes”, identifies leaves by their block range of 1). *Extraction.* Lemma 3.20(1) then yields a collision at the first level of disagreement. *Headers.* Field 1 of each leaf is the SHA-256d block hash of its header (Definition 2.3; *Crypto Guide*, §“The transparent layer’s hashes”); equal leaves with distinct headers are a collision of SHA-256d. $\square$

Proposition 6.14 (Binding of range transaction counts). *Let a node v of $\mathcal{T}_\eta$ be authenticated by an inclusion path (Definition 3.13, in the tuple form of Definition 3.19, siblings as full node vectors) accepted against the chain-history root of the epoch tree of a chain the client trusts. Barring a collision as in Proposition 6.13, $v.\text{nSaplingTxCount}$, and where present $v.\text{nOrchardTxCount}$ and $v.\text{nIronwoodTxCount}$, equal the numbers of transactions in the blocks of the height range of v in which $v.\text{SpendsSapling}$ or $v.\text{OutputsSapling}$, $v.\text{ActionsOrchard}$ and $v.\text{ActionsIronwood}$, respectively, are non-empty. Hence a client that obtains the transactions of those blocks, each authenticated against its block’s transaction root, detects the withholding of any such transaction by a count mismatch, and a range whose count is zero contains none and may be skipped; the count does not identify the missing transaction.*

Proof. By Lemma 3.20(2), an accepted path binds the full vector of every node it recomputes and of every sibling it carries, so v is the node of the trusted tree at its position, barring a collision; the serialisation is prefix-free by the proof of Proposition 6.13. In that tree the count fields are consistent, and by the parent rule of Definition 6.3 and Lemma 6.6 the value of each count field of v is its sum over the leaves of v ’s height range. The reading as detection of withholding is ZIP 221, “Non-FlyClient Commitments”. $\square$

Proposition 6.15 (Cross-branch separation). *For branch IDs $b \neq b'$ with encodings $\beta \neq \beta'$ the personalisations $\text{zcashHistory} \parallel \beta$ and $\text{zcashHistory} \parallel \beta'$ differ. Model $h(P, \cdot)$, for each personalisation P , as an independent random oracle with 256-bit output. An adversary making q oracle*

queries, the queries of the verification of its output included, outputs x, x' with

$$h(\text{ZcashHistory} \parallel \beta, x) = h(\text{ZcashHistory} \parallel \beta', x')$$

with probability at most $q^2 / 2^{256}$. Hence no internal-node hash or root commitment computed under b equals one computed under b' , except with that probability. Leaves are unpersonalised block hashes, identical in every branch, and are not separated.

Proof. The modelling hypothesis is that of the *Crypto Guide*, §“Domain separation and personalisation”, Proposition “Domain separation yields independent oracles”: distinct tags give independent random oracles. Both values must be answers to queries, one to each oracle; each of the at most q^2 pairs of such queries agrees with probability 2^{-256} , and the union bound (*Math Guide*, §“The union bound and a birthday calculation”) gives the bound. ZIP 221, “Tree nodes”, states the design intent: “valid nodes from one consensus branch cannot be used to make false statements about parallel consensus branches”. $\square$

6.5 The authorising-data root

The header field `hashMerkleRoot` (Definition 2.3) is the root of Bitcoin’s SHA-256d transaction Merkle tree over the transaction identifiers of the block (protocol specification, §“Block Header Encoding and Consensus”). The identifier of a version-5 or version-6 transaction excludes its authorising data (ZIP 244; *Ironwood Guide*, §“Transaction digests and signatures”, Definition “Authorising-data digest; effecting and authorising data”), so for such transactions `hashMerkleRoot` does not commit to the authorising data; the identifier of a version-4 transaction covers its whole encoding. The version-4 format is valid until NU7, from which ZIP 2003 (Draft), deployed by ZIP 259 (Draft), makes it invalid.

Construction 6.16 (Authorising-data root). Let a block have the transactions $t_1, \dots, t_m$, where $m \geq 1$ because of the coinbase transaction. Leaf i is the *authorising-data commitment* of t_i : for version 5, the digest `auth_digest` of ZIP 244, “Authorizing Data Commitment”; for version 6, the digest of the *Ironwood Guide*’s Definition “Authorising-data digest; effecting and authorising data” (ZIP 229 (Draft), “Block Commitments”: `hashAuthDataRoot` takes the version-6 digest “with no further structural change”); and for a transaction of a version before 5, the 32 bytes `0xFF`, a placeholder used only in this tree and occurring only in blocks before NU7. The leaves are padded with leaves of 32 zero bytes to $2^{\lceil \log_2 m \rceil}$, and `hashAuthDataRoot` is the root of the binary Merkle tree of height $\lceil \log_2 m \rceil$ over them, each non-leaf node being $h(\text{ZcashAuthDatHash}, \text{left} \parallel \text{right})$, under a 16-byte personalisation without branch ID. The insertion order MUST equal that of the transaction identifiers in the tree under `hashMerkleRoot`, so that one path identifies the same transaction in both trees (ZIP 244, “Block Header Changes”).

Proposition 6.17 (Joint binding of the two transaction roots). Assume, with ZIP 244, “Authorizing Data Commitment”, that the pair of a transaction’s identifier and its authorising-data commitment is binding: distinct transactions with equal pairs form a collision of that pair map. Let two lists of m transactions have equal `hashMerkleRoot` and equal `hashAuthDataRoot`. Then

the lists are equal, or a collision of SHA-256d, of $h(\text{ZcashAuthDataHash}, \cdot)$ or of the pair map is computed from them. The binding is claimed for lists of the given length only: the identifier tree is Bitcoin's transaction Merkle tree and inherits its duplicate-last-node caveat, and the header commits to no transaction count.

Proof. For fixed m , Bitcoin's duplication rule and the zero padding of Construction 6.16 are fixed maps of the list, so the path argument of the *Crypto Guide*, §“Binary Merkle trees”, Proposition “Binding of the root”, applies to each tree: the two lists of identifiers agree, else a collision of SHA-256d is extracted, and the two lists of authorising-data commitments agree, else a collision of $h(\text{ZcashAuthDataHash}, \cdot)$. The common insertion order aligns the two leaf sequences, so each position carries equal pairs, and distinct transactions there form a collision of the pair map. ZIP 244, “Authorizing Data Commitment”, states that the pair “constitutes a commitment to all the data of a serialized transaction”; for a transaction of a version before 5 the identifier covers all data and the second member is the constant placeholder. The caveat is the *Crypto Guide*, §“Layer and domain separation”, Remark “Unbalanced trees, padding, and a deployed failure”. $\square$

6.6 The block commitment hash

In this subsection 0^{32} denotes the string of 32 zero bytes.

Definition 6.18 (Block commitment hash). For a block B_η at a height in or after the NU5 upgrade epoch, the *block commitment hash* is

$$\text{hashBlockCommitments}(B_\eta) := h(\text{ZcashBlockCommit}, \text{hashLightClientRoot}(\eta) \parallel \text{hashAuthDataRoot}(B_\eta) \parallel 0^{32}),$$

with $\text{hashLightClientRoot}(\eta)$ the history input of Definition 6.12, $\text{hashAuthDataRoot}(B_\eta)$ the root of Construction 6.16 for B_η , and 0^{32} the *terminator* (ZIP 244, “Block Header Changes”). The input has $32 + 32 + 32 = 96$ bytes, and the 16-byte personalisation carries no branch ID. The history input is a function of the blocks before B_η ; the authorising-data root is a function of B_η itself.

Proposition 6.19 (Reopening the block commitment). *From NU5 the chain-history root appears in no header. Let c be the value $\text{hashBlockCommitments}(B_\eta)$ of a block in or after the NU5 upgrade epoch. A verifier given a claimed history input r , or a claimed root node vector from which it computes $r = h(\text{ZcashHistory} \parallel \beta, \text{ser}(\text{root}))$ with β encoding the branch ID of Definition 6.9, and a 32-byte value a , accepts r if and only if*

$$c = h(\text{ZcashBlockCommit}, r \parallel a \parallel 0^{32});$$

only the 32-byte value of hashAuthDataRoot is needed, not its tree. Two distinct accepted pairs $(r, a) \neq (r', a')$ exhibit a collision of $h(\text{ZcashBlockCommit}, \cdot)$. Hence an accepted r is the block's history input except with that collision probability, and an inclusion path in the epoch tree is checked against a header through it.

Proof. The pair $(\text{hashLightClientRoot}(\eta), \text{hashAuthDataRoot}(B_\eta))$ is accepted by Definition 6.18. The input $r \parallel a \parallel 0^{32}$ has fixed length, so distinct pairs give distinct inputs, and two accepted pairs with equal image c are a collision. $\square$

Remark 6.20 (Extension of the block commitment hash). ZIP 244, “Block Header Changes”, treats the value as “a linked list of hashes terminated by arbitrary data”: a later upgrade may replace the terminator by the hash of a new structure that ends in its own terminator. The same section states two interoperability rules: fully validating nodes **MUST** always use the entire structure defined by the latest activated protocol version they support, and a third-party server that supports a later protocol version **SHOULD** provide a value that can be used in place of the all-zero terminator, so that a light client can validate the parts it needs. No upgrade through NU7 replaces the terminator: ZIP 258, “Changes to ZIP 221”, leaves `hashBlockCommitments` “unaffected” beyond the added node fields; ZIP 229, “Block Commitments”, changes no structure; and no source of NU7 consensus changes listed in ZIP 259, “NU7 consensus changes”, amends ZIP 244.

6.7 The header consensus rules

Definition 6.21 (The commitment field by upgrade epoch). The 32-byte header field that follows `hashMerkleRoot` (Definition 2.3) is named `hashReserved` before Sapling, `hashFinalSaplingRoot` in the Sapling and Blossom upgrade epochs, `hashLightClientRoot` in the Heartwood and Canopy upgrade epochs, and `hashBlockCommitments` from NU5 (protocol specification, §“Block Header Encoding and Consensus”; ZIP 221, “Block header semantics and consensus rules”; ZIP 244, “Block Header Changes”). The header byte format and version are unchanged by each renaming.

Definition 6.22 (Header commitment rule). For a block B_η the commitment field satisfies the following rules.

- (i) For $\eta < A_H$ it follows the earlier rules of the protocol specification, §“Block Header Encoding and Consensus”, not restated here: in the Sapling and Blossom upgrade epochs it is `LEBS2OSP256` of the root of the Sapling note commitment tree of the block’s final Sapling treestate.
- (ii) For $A_H \leq \eta$ in the Heartwood or Canopy upgrade epoch it **MUST** equal the history input `hashLightClientRoot`(η) of Definition 6.12, which supplies the sentinel at A_H .
- (iii) For η in or after the NU5 upgrade epoch, the NU5 activation block included and without sentinel, it **MUST** equal `hashBlockCommitments`(B_η) of Definition 6.18.

(ZIP 221, “Block header semantics and consensus rules”; ZIP 244, “Block Header Changes”; protocol specification, §“Block Header Encoding and Consensus”.) The value is undefined for the genesis block, which no case covers.

In the NU5 activation block $B_{A_{\text{NU5}}}$, A_{NU5} the NU5 activation height, the history input is `hashChainHistoryRoot`($\mathcal{T}_{A_{\text{NU5}}}$), the root of the completed tree of the Canopy upgrade epoch.

It is hashed under the Canopy branch ID 0xE9FF75A6 (ZIP 251) by the adopted reading of Definition 6.9, the first sentence of ZIP 221 giving the NU5 branch ID instead, and it is built from nodes of fields 1 to 11 by the adopted reading of Definition 6.5. By Definition 6.22 every valid block after the Heartwood activation block commits to the root of its epoch tree: directly in the Heartwood and Canopy upgrade epochs, and from NU5 as the first input of the block commitment hash.

6.8 The committed range

Proposition 6.23 (One-block delay). *For $\eta > A_H$ the tree to which B_η commits (Definition 6.22) has the leaves $B_{\alpha(\eta)}, \dots, B_{\eta-1}$. No header commits to a tree that contains its own block, and $B_{\eta-1}$ first enters a committed tree at B_η . Consequently the final Sapling root of a block at a height of at least A_H is committed one block later, as the field `hashLatestSaplingRoot` of the root node committed by the next block. ZIP 221, “Non-FlyClient Commitments”, accepts the delay because “using the most recent treestate as a transaction anchor is already unsafe in reorgs”.*

Proof. By Definition 6.2, η lies outside $[\alpha(\eta), \eta - 1]$. Since $\eta - 1 \geq A_H$ and $\alpha(\eta)$ is the largest activation height below η , $\alpha(\eta) \leq \eta - 1$, so $B_{\eta-1}$ is the last leaf of $\mathcal{T}_\eta$; it is no leaf of $\mathcal{T}_{\eta'}$ for $\eta' \leq \eta - 1$, whose leaves lie below η' . Then Definition 6.22. The latest-root field of the root node is that of its last leaf, by the parent rule of Definition 6.3. $\square$

Theorem 6.24 (Reset at each activation height). *Let $A > A_H$ be an activation height and $A' = \alpha(A)$ the preceding one. Then:*

1. B_A commits to the complete tree $\mathcal{T}_A$ over $B_{A'}, \dots, B_{A-1}$, the whole upgrade epoch beginning at A' , hashed under that upgrade epoch’s branch ID $b(\mathcal{T}_A)$ by the adopted reading of Definition 6.9 and built with that upgrade epoch’s field vector by the adopted reading of Definition 6.5; the commitment field equals `hashChainHistoryRoot`($\mathcal{T}_A$) before NU5, and `hashBlockCommitments`(B_A) with it as first input from NU5;
2. B_{A+1} commits to the single-leaf tree over B_A ;
3. for $A' < \eta < A$, the leaves of $\mathcal{T}_\eta$ are a proper prefix of those of $\mathcal{T}_A$, so every ended upgrade epoch from Heartwood on is committed completely by exactly one header, that of the next activation block;
4. the reset occurs at every upgrade, including those that leave the field vector unchanged (Canopy, NU6, NU6.1, NU6.2 and NU7).

Proof. By the definition of α , $\alpha(A) = A'$ and $\alpha(A + 1) = A$. For $A' < \eta < A$, $\alpha(\eta) = A'$, so $\mathcal{T}_\eta$ has the leaves $B_{A'}, \dots, B_{\eta-1}$, a proper prefix of those of $\mathcal{T}_A$; only $\mathcal{T}_A$ has the full leaf set, and for $\eta > A$ no tree has the leaf $B_{A'}$, since $\alpha(\eta) \geq A$. The commitment fields follow from Definition 6.22. Part (4) holds because α depends on the activation heights alone. A script illustrates (1) to (3) on a toy chain, a one-block upgrade epoch included; it establishes nothing. $\square$

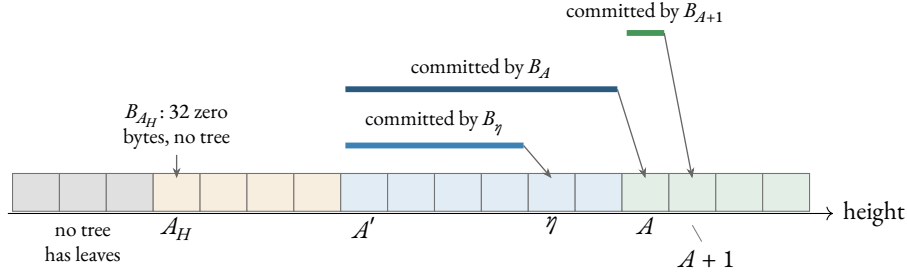


Figure 4. Committed ranges on a height axis, one cell per block, with the Heartwood activation height A_H and two later activation heights $A' < A$, consecutive, and $A' < \eta < A$. Grey: heights below A_H , a leaf of no epoch tree (Corollary 6.25). The Heartwood activation block carries the sentinel of Definition 6.12 and commits no tree. Each bar is the leaf range of the tree to which the header at the arrow's head commits: B_η commits $B_{A'}, \dots, B_{\eta-1}$; the activation block B_A commits the whole upgrade epoch $B_{A'}, \dots, B_{A-1}$ (Theorem 6.24(1)); and B_{A+1} commits the single leaf B_A (Theorem 6.24(2)). No bar reaches below A_H .

Figure 4 shows the ranges of the theorem.

Corollary 6.25 (No commitment below the Heartwood activation height). *The Heartwood activation block B_{A_H} commits to no tree, its history input being the sentinel of Definition 6.12. For $\eta > A_H$, $\alpha(\eta) \geq A_H$, so no committed tree has a leaf below A_H . Below A_H the commitment field holds a Sapling root or is reserved (Definition 6.21).*

Proof. The height A_H is an activation height below every $\eta > A_H$, and $\alpha(\eta)$ is the largest such; then Definition 6.22, item (ii). $\square$

Corollary 6.26 (The NU7 upgrade epoch). *Let A_{NU7} be the NU7 activation height. The block $B_{A_{\text{NU7}}}$ commits, as the first input of its block commitment hash, to the completed tree of the NU6.3 upgrade epoch, hashed under the NU6.3 branch ID $0x37A5165B$ by the adopted reading of Definition 6.9. The block $B_{A_{\text{NU7}}+1}$ commits to a new tree over $B_{A_{\text{NU7}}}$, hashed under the NU7 branch ID $0x77198AD9$ (ZIP 259, Draft) with the field vector of Definition 6.5 for the NU7 upgrade epoch.*

Proof. The instance $A = A_{\text{NU7}}$ of Theorem 6.24, NU6.3 being the upgrade that precedes NU7 (Definition 1.1), with the constants of ZIP 259, “NU7 network constants”, and ZIP 258, “NU6.3 deployment”. $\square$

7 The deployment boundary

This section classifies every component of a Zcash FlyClient in the classes of Definition 1.3. It opens with one table that places each hypothesis of Theorem 5.32 and each obligation of Remark 5.33 in its class, with the cited fact that places it there. The subsections that follow elaborate the rows: the specified commitment and the absence of a specified consumer, the differences between the paper’s commitment and the deployed one, the conditions that the ZIPs impose on any sample check of a Zcash verifier, and the open problems.

7.1 Classification of the security hypotheses

Remark 7.1 (Classification of the hypotheses). The classes of Definition 1.3 apply to components, not to properties. A property proved in this volume of a specified object is recorded as “the object is specified, and the property is proved here”, never as specified. Table 4 classifies every hypothesis of Theorem 5.32 and every obligation O1 to O6 of Remark 5.33, together with two components of the deployed commitment, the chain commitment (row (1)) and its composition across upgrade epochs (row (4)); O1 and O2 share row (3). Each row gives its class and the cited fact that places it there or, for an assumption on the client’s environment, the source that records it. Hypothesis 1 of the theorem fixes the parameters c , k and δ , whose values an instantiation chooses; rows (7) and (9) classify what those values rest on. The rows read as follows.

- (1) *Chain commitment*: specified. It is the epoch-scoped, metadata-bearing, branch-separated epoch tree $\mathcal{T}_\eta$ (Definition 6.2, Construction 6.10), to whose chain-history root every header after the Heartwood activation block commits under Definition 6.22, enforced by consensus (ZIPs 221 and 244; ZIP 258, Draft, whose rules are in force from NU6.3 by Definition 1.2).
- (2) *Collision resistance of the hash functions*: the functions are specified, namely personalised BLAKE2b-256 h for tree nodes, the chain-history root, the authorising-data root and the block commitment hash (ZIPs 221 and 244; protocol specification, §“BLAKE2 Hash Functions”), and SHA-256d for block hashes, hence for leaves (protocol specification, §“Block Header Encoding and Consensus”). Their collision resistance is a standard assumption (*Crypto Guide*, §“Security notions: preimage, second-preimage, and collision resistance”).
- (3) *O1, binding within one upgrade epoch*: the commitment is specified (Definition 6.22), and the property is proved here of the specified object (Proposition 6.13); no normative text fixes the property. *O2, fork structure for ZIP 221 trees*: open, since it is conditional on a sufficient sample check for those trees, which no specification or result supplies (Proposition 7.4); Lemmas 5.4 and 5.5 are stated for the paper’s construction only. Hypothesis 6 of Theorem 5.32, leaf–internal separation, enters only through Lemma 5.4 and shares the class of O2: it is stated for the paper’s construction, and for ZIP 221 trees no specification or result supplies its counterpart.
- (4) *Composition across upgrade epochs*: open. Each upgrade epoch has its own tree (Theorem 6.24), and ZIP 221 states no rule that chains them.
- (5) *The verifier’s anchor*, the genesis block in the paper: open (Corollary 6.25).
- (6) *O3, weight binding*: the node vector is specified (Definition 6.3), and the binding of the field-8 interval of a leaf to the root and its position within one upgrade epoch is proved here of the specified object (Lemma 3.21(1) with Proposition 6.13). The link from a sampled header to its leaf’s weight is conditional on the same sufficient sample check, and so open. Sampling by the chain’s total work is open, field 8 of a root summing one upgrade epoch (Definition 6.3; Proposition 7.8).

- (7) *Assumption 4.8, the transition checks of O3 (Construction 5.8) and Assumption 5.11 under the Zcash retarget rule*, in both of its regimes: open (protocol specification, §“Difficulty adjustment”; ZIP 218, Draft).
- (8) *Assumption 5.20*: open for Zcash; no analysis relates the work of individual blocks to the invalid segments of a Zcash fork.
- (9) *O4, catch probability and sample count*: designed but unspecified, and proved in the paper’s model under its hypotheses (Theorem 5.21; Proposition 5.22).
- (10) *The random-oracle hypothesis on the sampling hash*: open. No ZIP or specification section fixes a sampling hash for Zcash, so the hypothesis has no object; the model is that of the *Crypto Guide*, §“The random oracle model”.
- (11) *O5, grinding*: the transform of Construction 5.27 is designed but unspecified. Its Zcash transcript (header bytes, hash, domain separation, and map from output to weight points across upgrade epochs) and a derivation of the budget R of Assumption 5.28 are open.
- (12) *O6, size*: the bound of Corollary 5.34 is designed but unspecified; a Zcash proof format is open.
- (13) *At least one honest prover*: an assumption on the client’s connectivity, not a component, so no class applies. ZIP 221, “Security and Privacy Considerations”, summarises the analysis of the paper, not the setting of ZIP 221 itself, as follows: “The security analysis assumes adversary mining power is bounded by a known fraction of combined mining power of honest nodes, and cannot drop or tamper with messages between client and full nodes. It also assumes the client is connected to at least one full node and knows the genesis block.” The bound on mining power is row (7); the absence of dropping or tampering and the connection to at least one full node are this row; knowledge of the genesis block is row (5).

Open problems (§7.5) derives rows (4), (5), (7), (11) and (12), and the total work of row (6), from cited rules; O2 of row (3), with the header-to-weight link of row (6), and rows (8) and (10) are open for the reasons stated above.

The verifier as a whole is designed but unspecified. The protocol of “The FlyClient protocol” (§5) exists with the paper’s theorem in the paper’s model; no ZIP specifies a Zcash instantiation; and the paper’s model does not cover Zcash’s retarget rule. The paper proves security only for its model of retargeting once per retarget epoch (Definition 4.5), places rules that retarget from a moving average outside it, and credits them with heuristic guarantees only (ePrint 2019/226, Section 3.1, pages 7–8); it names Ethereum there and does not name Zcash. ZIP 221, “Security and Privacy Considerations”, names Zcash among the chains “with rapidly adjusting difficulty” that the paper leaves open.

7.2 The specified commitment and its consumers

Row (1) of Remark 7.1 is the following. ZIP 221 (Final) specifies the tree, with FlyClient as its stated motivation (ZIP 221, “Motivation”). ZIP 221, “FlyClient Requirements and

Table 4. The components of a Zcash FlyClient, covering every hypothesis of Theorem 5.32 and every obligation of Remark 5.33, with its class under Definition 1.3 and the result or normative text that places it there. Rows are numbered as in Remark 7.1, which states each in full.

	Hypothesis or obligation	Class	Placed by
(1)	chain commitment $\mathcal{T}_\eta$	specified	Definition 6.22; ZIPs 221, 244, 258
(2)	collision resistance of h and SHA-256d	functions specified; resistance assumed	ZIPs 221, 244; <i>Crypto Guide</i>
(3)	O1: binding within one upgrade epoch	commitment specified; property proved here	Proposition 6.13
	O2: fork structure for ZIP 221 trees; hypothesis 6 (leaf–internal separation)	open problem	Proposition 7.4; sufficiency not claimed
(4)	composition across upgrade epochs	open problem	Theorem 6.24; ZIP 221 silent
(5)	anchor of the verifier	open problem	Corollary 6.25
(6)	O3: weight binding	node vector specified; interval binding proved here; header-to-weight link and total work open	Lemma 3.21(1); Proposition 7.8
(7)	(c, L) , transition checks, transition rewriting	open problem	retarget rule of the specification; ZIP 218
(8)	discretisation	open problem	no analysis for Zcash
(9)	O4: catch probability, sample count	designed but unspecified	Theorem 5.21; Proposition 5.22
(10)	random oracle for sampling	open problem	no sampling hash specified
(11)	O5: grinding	transform designed but unspecified; transcript and R open	Construction 5.27; Assumption 5.28
(12)	O6: size	designed but unspecified; format open	Corollary 5.34
(13)	at least one honest prover	environment; no class	ZIP 221, “Security and Privacy Considerations”

Recommendations”, designates seven fields for reasoning about the difficulty adjustment over a range of blocks: `nEarliestTimestamp`, `nLatestTimestamp`, `nEarliestTargetBits`, `nLatestTargetBits`, `nEarliestHeight`, `nLatestHeight` and `nSubTreeTotalWork` (Remark 6.4). The pool roots and transaction counts are its “Non-FlyClient Commitments”. No ZIP or specification section specifies a consumer of the tree, a proof object or a verifier. The specified class depends on no implementation: Definition 1.2 places the fields of ZIP 258 (Draft) in force from NU6.3.

Remark 7.2 (Recomputation against proof verification). The consensus rule of Definition 6.22 is an equality between the commitment field of a header and a value that the validator recomputes from the chain it holds: $\text{hashChainHistoryRoot}(\mathcal{T}_\eta)$ in the Heartwood and Canopy upgrade epochs, and the block commitment hash from NU5 (protocol specification, §“Block Header Encoding and Consensus”; ZIP 221, “Block header semantics and consensus rules”; ZIP 244, “Block Header Changes”). No ZIP or specification section defines an inclusion proof for the tree, a proof object or a verification algorithm. The operation that the paper’s verifier performs, Construction 5.2, is therefore unspecified for Zcash.

No specified light-client protocol consumes the commitment. ZIP 307 (Draft) does not reference ZIP 221, while ZIP 221, “Motivation”, defers the verification of transaction inclusion to ZIP 307: it “follows the usual reference protocol”.

7.3 The paper’s commitment and the deployed commitment

Remark 7.3 (Deltas from the paper’s commitment). Against Construction 5.1 and Definition 5.6, the deployed commitment differs in five respects.

- (1) *Scope*. It is epoch-scoped, not rooted at the genesis block (Theorem 6.24). The one-block delay of Proposition 6.23 is shared with the paper, whose header also commits to exactly the blocks strictly before it.
- (2) *Metadata*. The seven FlyClient fields serve the purpose of Definition 5.6 (ePrint 2019/226, Definition 7) without matching it field for field (Remark 6.4); the pool roots and counts have no counterpart.
- (3) *Hashing*. Every parent is hashed by the personalised, branch-separated BLAKE2b-256 $h(\text{ZcashHistory} \parallel \beta, \text{ser}(\ell) \parallel \text{ser}(r))$ over full serialisations, the root commitment $\text{hashChainHistoryRoot}$ covering the root’s aggregates, and every leaf carries the SHA-256d block hash; the paper hashes with one generic collision-resistant H (Propositions 6.13 and 6.15).
- (4) *Placement*. The root enters through the repurposed commitment field (Definition 6.21), and from NU5 only as the first input of the block commitment hash (Proposition 6.19), whereas the paper adds a header field that can replace the previous block hash (ePrint 2019/226, Section 4.2, page 12).
- (5) *Bagging*. The deployed tree bags its peaks by the left fold Bag_L , the paper by the right fold Bag_R (ePrint 2019/226, Definition 11). Path tops, prefix folds and the maximal path

length therefore differ whenever the leaf count has three or more set bits (Theorem 3.8; Lemma 3.7).

ZIP 221, “Tree Node specification”, states: “Other than the metadata commitments, the MMR tree’s construction is standard.” Items (1), (3), (4) and (5) record the respects in which the deployed construction also differs from the paper’s.

7.4 A Zcash verifier

The verifier is designed but unspecified (Remark 7.1). The conditions below are obligations that ZIPs 221 and 244 impose on any sample check for ZIP 221 trees; they are not a construction of this volume, and no Zcash verifier is specified here. A *sample check for ZIP 221 trees* is a deterministic algorithm that takes the header of a head, an index j and a prover’s message, and accepts or rejects; its purpose is to establish, for the head’s epoch tree, the conclusions of Proposition 5.3 and Lemma 5.7. A condition is *necessary* when some message that meets every other condition and violates it also violates such a conclusion, so that a check that does not reject that message fails the conclusion.

Proposition 7.4 (Necessary conditions of a Zcash sample check). *Let the head B_η , $\eta > A_H$, commit to T_η with $n = \eta - \alpha(\eta)$ leaves, let β encode $b(T_\eta)$, and let a sample be leaf j of T_η , $1 \leq j \leq n$, the block at height $\alpha(\eta) + j - 1$, with received header hdr_j . Each of the following conditions, the height clause of (ii) and the parsing clause of (iv) excepted, is necessary for a sample check for ZIP 221 trees. Sufficiency is not claimed. The height clause of (ii) is required for the leaf’s stated heights to match its position (Lemma 6.6); parsing under the field vector is the hypothesis under which Proposition 6.13 and Lemma 3.20(2) apply; the necessity of neither is claimed.*

- (i) *Field 1 of the leaf equals $\text{BlockHash}(\text{hdr}_j)$, the SHA-256d hash of the full received header, Equihash solution included (Lemma 6.7).*
- (ii) *Fields 2 and 3 of the leaf equal the field $n\text{Time}$ of hdr_j , fields 4 and 5 its field $n\text{Bits}$; field 8 equals the work $\lfloor 2^{256} / (\text{ToTarget}(n\text{Bits}) + 1) \rfloor$ of that $n\text{Bits}$; and fields 9 and 10 equal $\alpha(\eta) + j - 1$, the height that the position fixes, the header carrying no height (Definition 2.3).*
- (iii) *The header hdr_j satisfies proof of work in the sense of Definition 2.3: a valid Equihash solution, and the difficulty filter at the target $\text{ToTarget}(n\text{Bits})$ it declares.*
- (iv) *The sampled leaf of (i) and (ii) and the path siblings are received as full node serialisations, parsed under the field vector of Definition 6.5 for the upgrade epoch beginning at $\alpha(\eta)$. Starting from that leaf, every parent is recomputed: field 1 as $h(\text{ZcashHistory} \parallel \beta, \text{ser}(\ell) \parallel \text{ser}(r))$ (Construction 6.10), the other fields by the parent rule of Definition 6.3. The recomputed root node yields $\text{hashChainHistoryRoot}(T_\eta)$, which is compared with the head’s commitment field under Definition 6.22: directly in the Heartwood and Canopy upgrade epochs, and from NU5 through Proposition 6.19 with the head’s hashAuthDataRoot .*

- (v) The sides and the length of the path are those that j and n fix under Bag_L , the length being $\text{depth}_L(n, j)$ (Lemma 3.7).
- (vi) For $j \geq 2$: the root node ρ output by $\text{PrefixRoot}_L(n, j, \pi)$ (Construction 3.16), run on the path π with the parent rule in place of $H(\cdot \parallel \cdot)$, is the claimed root node of $\mathcal{T}_{a(\eta)+j-1}$, and the value

$$r = h(\text{ZcashHistory} \parallel \beta, \text{ser}(\rho))$$

is accepted against the commitment field of hdr_j under Definition 6.22: r equals that field in the Heartwood and Canopy upgrade epochs, and from NUS r is accepted by Proposition 6.19 with the hashAuthDataRoot of the sampled block.

Proof. For each condition, a message that meets the other five and violates it also violates the conclusion named.

- (i) An honest header with valid proof of work, paired with a leaf whose field 1 is the hash of a different, unmined block of the claimed tree. The proof of work checked is not that of the committed block, contrary to Proposition 5.3(i) and Lemma 5.7(1).
- (ii) An honest header paired with a leaf of inflated field 8, in a tree that the adversary builds over such leaves under a head that it mines, so that every path verifies. The header-determined fields are not those of the header, contrary to Proposition 5.3(ii), and the interval that Lemma 3.21(1) binds belongs to a forged weight.
- (iii) A tree built over fabricated headers without valid solutions. Every other condition is met by construction, and unmined headers pass, contrary to Lemma 5.7(1).
- (iv) If a node is taken in any other form, the fields that do not enter the recomputation are unbound. With the siblings supplied as field-1 digests and the path checked on field 1 alone, the siblings' field 8, which fixes the interval of Lemma 3.21, and the leaf's pool fields (6, 7 and, where the vector carries them, 11 to 17), which its header does not determine, are chosen freely; two passing samples at j then carry distinct leaf tuples, contrary to Proposition 5.3(i). If the recomputation starts from a node other than the leaf matched in (i) and (ii), the matched header is bound to nothing. Proposition 6.13 binds full serialisations only.
- (v) The single-block tuple of a leaf placed at a position where the shape $\mathcal{M}_n^L$ has an internal node: a tree with fewer than n leaves then passes, and a tuple committed at another depth is accepted as leaf j . Theorem 3.17, which assumes the shape fixed by n and the bagging order, does not apply.
- (vi) Honest blocks, mined on the honest chain after the fork point, appended as leaves of the fork's tree. They pass (i) to (v), while their own commitments bind the honest prefix, not the fork's first $j - 1$ leaves, contrary to Proposition 5.3(iii). This is the case that Lemma 5.4 excludes by the prefix check.

No condition is shown sufficient. □

The conditions are the ZIP 221 counterparts of the checks of Construction 5.2: (i) and (ii) of check (a), (iii) of check (d), (iv) and (v) of check (b), and (vi) of check (c). Three further statements bound their reach. For $j = 1$ the sample is the block $B_{\alpha(\eta)}$. For $\alpha(\eta) > A_H$ it is an activation block, which commits to the completed tree of the previous upgrade epoch (Theorem 6.24); for $\alpha(\eta) = A_H$ it carries the sentinel of Definition 6.12 (Corollary 6.25). In both cases its prefix check belongs to epoch composition, which is open. The consensus condition $n\text{Bits} = \text{ThresholdBits}(\alpha(\eta) + j - 1)$ of Definition 2.3 depends on preceding headers and is not decided by one sample; it is the object of the transition checks of row (7), which are open. And condition (iv) makes Proposition 6.13 apply, but the fork structure of row (3) additionally needs a sufficient check, which is open.

Example 7.5 (Per-sample size). The size is evaluated at the scale of one upgrade epoch, since n is the leaf count of one epoch tree and never the length of the chain. The message of Proposition 7.4 for one sample consists of the 1,487-byte header (§2.1), the sampled leaf’s own serialisation of at most 317 bytes (Definition 6.5), at most $\lfloor \log_2 n \rfloor + \text{popcount}(n) - 1$ siblings of at most 317 bytes each (Lemma 3.7), and the 32 bytes of the sampled block’s `hashAuthDataRoot`. For $n = 2^{20}$ a path has at most 20 siblings, and a sample at most

$$1,487 + 317 + 20 \cdot 317 + 32 = 8,176 \text{ bytes,}$$

of which the header is 18.2 per cent. For $n = 2^{20} - 1$, with 20 peaks, a path has at most 38 siblings, and a sample at most

$$1,487 + 317 + 38 \cdot 317 + 32 = 13,882 \text{ bytes,}$$

of which the header is 10.7 per cent. The leaf’s fields fixed by its header and its position, 1 to 5 and 8 to 10, could be omitted by a proof format, which is open. The figures are computed by script.

7.5 Open problems

The following open items, each a row of Remark 7.1 except the serving interface, are derived below from cited rules: epoch composition, the anchor below the Heartwood activation height, total work, the proof format, the serving interface, the Fiat–Shamir transcript, and the security analysis under Zcash’s retarget rule with its NU7 regime. Composition with compact-block scanning follows. Availability of prover data is not a protocol question.

Remark 7.6 (Epoch composition). Epoch trees are hashed under distinct branch IDs, and each complete root is committed only by the next activation header (Theorem 6.24). No specified rule chains them, and ZIP 221, “Specification”, is silent. Composition is needed for prefix proofs and for difficulty checks: the averaging window and the median-time span of the first blocks of an upgrade epoch reach into the previous upgrade epoch (protocol specification, §“Difficulty adjustment”), so a transition check at an upgrade boundary needs two epoch trees. Proposition 5.38 needs an inclusion path of the accepted head against the later head’s root, and so applies within one epoch tree. Across epoch trees it does not apply: no tree of a later upgrade epoch contains a leaf of an earlier one (Definition 6.2), and only the activation header commits to the completed earlier tree. No design exists: the item is open.

Remark 7.7 (Anchor). The anchor below the Heartwood activation height is open, by Corollary 6.25.

Proposition 7.8 (No committed node aggregates total work). *Let $\eta > A_H$.*

- (a) *If SHA-256d is collision resistant, the header of B_η commits through hashPrevBlock to the headers of $B_0, \dots, B_{\eta-1}$, hence to their fields nBits and to the total work ω of that chain (§2.1; the model's counterpart is Definition 4.2).*
- (b) *For every node v of $\mathcal{T}_\eta$, field 8 is the sum modulo 2^{256} of the work of the blocks at heights $v.\text{nEarliestHeight}$ to $v.\text{nLatestHeight}$, all in $[\alpha(\eta), \eta - 1]$; field 8 of no node of $\mathcal{T}_\eta$ depends on $B_0, \dots, B_{\alpha(\eta)-1}$.*
- (c) *Hence no inclusion path against the root to which B_η commits opens ω . Sampling by the chain's cumulative work (Definition 5.19) and the acceptance rule need field 8 of the root of every completed epoch tree $\mathcal{T}_A$, $A_H < A \leq \alpha(\eta)$ an activation height, each committed only by B_A (Theorem 6.24), together with the work of the blocks below A_H , which no tree commits (Corollary 6.25).*

No ZIP addresses the item; it is open.

Proof. (a) The block hash covers the whole header, nBits and hashPrevBlock included (Definition 2.3); induction down the chain, a divergence yielding a collision of SHA-256d. (b) Field 8 of a leaf is its block's work and that of an internal node the sum of its children's (Definition 6.3); the leaves of $\mathcal{T}_\eta$ are $B_{\alpha(\eta)}, \dots, B_{\eta-1}$ (Definition 6.2); the height range of v is that of Lemma 6.6. (c) A path opens only fields of nodes of $\mathcal{T}_\eta$, and by (b) the only work-valued field, field 8, sums work over leaves of $\mathcal{T}_\eta$; the pool-root fields depend on earlier blocks, but through treestates, from which work is not computed. The work of each block is at least 1, since $\text{ToTarget}(\text{nBits}) + 1 \leq 2^{256}$, so ω exceeds the work of the leaves of $\mathcal{T}_\eta$ whenever $\alpha(\eta) > 0$. The remaining work is committed only as stated, by Theorem 6.24 and Corollary 6.25. ZIP 221, "Tree Node specification", assumes that cumulative chain work stays below 2^{256} , which makes each modular sum exact (Lemma 3.21(2)). $\square$

Part (b) does not contradict the statement of (a) that every header commits to the total work: that commitment is through the linkage of every header, which a sampling verifier does not receive.

Remark 7.9 (Proof format). The proof format is open; no ZIP or specification section defines one. From NU5 any format carries, for every header whose commitment it opens, the 32-byte hashAuthDataRoot needed to reopen the block commitment hash (Proposition 6.19).

Remark 7.10 (Serving interface). The serving interface is open: no ZIP or specification section defines a message for requesting or serving tree nodes, inclusion paths or assembled proofs. Availability of prover data is not a protocol question. Every node, and hence every inclusion path, of $\mathcal{T}_\eta$ is a function of the chain up to its last leaf $B_{\eta-1}$, not of the blocks of the upgrade epoch alone, since the pool-root fields (6, 7, 12, 13, 15 and 16) are roots of final treestates, which depend on every note commitment since the activation of the pool (Definition 6.3; Construction 6.10; ZIP

221, “Tree Node specification”; ZIP 258, “Changes to ZIP 221”). Any party that holds that chain computes them; what is unspecified is the message.

Remark 7.11 (Transcript). The transcript is open. Construction 5.27 derives the sampling randomness from a hash of the head (ePrint 2019/226, Section 6.2). For Zcash no ZIP or specification section fixes the header bytes, the hash, the domain separation or the map from output to weight points across upgrade epochs, so Proposition 5.29 has no Zcash instance.

Remark 7.12 (Security analysis). The security analysis is open. Zcash retargets after every block from an averaging window with median-time spans, damping and asymmetric clamps, which is not Definition 4.5. The blocks of the upgrade epochs from Heartwood through NU6.3 are governed by a 17-block window at a 75-second target spacing (protocol specification, §“Difficulty adjustment”); NU7 as specified uses 102 blocks at 25 seconds (ZIP 218, Draft, “Averaging window”; *Consensus Guide*, §“Difficulty in motion”). Neither regime is covered by Theorem 5.32, and the paper contains no derivation of (c, L) from either (Remark 4.13). The published analysis of a Zcash instantiation (Perazzo and Capecchi, arXiv 2604.26736, Section IV-A, Definition IV.1) assumes its adversarial work budget rather than deriving it from the retarget rule, so it does not close the gap; it is recorded here because the open classification must account for it. ZIP 221, “Security and Privacy Considerations”, records that the paper leaves such chains with only heuristic guarantees, that “The use of these fields has not been analysed in the academic security literature”, and that “A more in-depth security analysis of FlyClient should be performed before designing a FlyClient-based light client ecosystem for Zcash”. No ZIP addresses the gap.

Remark 7.13 (NU7). ZIP 218 (Draft) changes the target spacing from 75 to 25 seconds, the averaging window from 17 to 102 blocks and the expected stale rate (ZIP 218, “Block target spacing”, “Averaging window”, “Stale block rate”). These bear on the (c, L) that an instantiation could assume, not on Theorem 5.21, which is stated in c and δ . Transition checks must use the height-dependent spacing and window, span two regimes across one more upgrade boundary (Corollary 6.26), and meet first adjustments that ZIP 218, “Effect on difficulty adjustment”, expects to be limited by PoWMaxAdjustDown (*Consensus Guide*, §“NU7: 25-second blocks and bounded shielded work”).

Remark 7.14 (Compact-block scanning). Composition with compact-block scanning is open. A FlyClient replaces trust in the header chain, not in the channel through which a wallet detects its transactions (ZIP 307, Draft).